UNIVERSIDAD DE CASTILLA-LA MANCHA

GRADO EN INGENIERÍA INFORMÁTICA

Tecnologías de la Información

TRABAJO FIN DE GRADO

Desarrollo de una aplicación de Realidad Virtual para ejercitación de pacientes renales durante sesiones de Hemodiálisis

Pedro Antonio Trujillo Felipe

Escuela Superior de Informática



Ciudad Real, julio de 2026



UNIVERSIDAD DE CASTILLA-LA MANCHA
ESCUELA SUPERIOR DE INFORMÁTICA

Tecnologías y Sistemas de Información

Tecnologías de la Información

TRABAJO FIN DE GRADO

Desarrollo de una aplicación de Realidad Virtual para ejercitación de pacientes renales durante sesiones de Hemodiálisis

Autor: Pedro Antonio Trujillo Felipe

Tutor: Javier Alonso Albusac Jiménez

Cotutora: Vanesa Herrera Tirado

julio, 2026

Desarrollo de una aplicación de Realidad Virtual para ejercitación de pacientes renales durante sesiones de Hemodiálisis

© Pedro Antonio Trujillo Felipe, 2026

Este documento se distribuye con licencia CC BY-NC-SA 4.0. El texto completo de la licencia se puede obtener en https://creativecommons.org/licenses/by-nc-sa/4.0/. La copia y distribución de esta obra está permitida en todo el mundo, sin regalías y por cualquier medio, siempre que esta nota sea preservada. Se concede permiso para copiar y distribuir traducciones de este libro desde el español original a otro idioma, siempre que la traducción sea aprobada por el autor del libro y tanto el aviso de copyright como esta nota de permiso, sean preservados en todas las copias.



Este texto ha sido preparado con la plantilla LaTeX para Trabajo Fin de Estudios en Ingeniería Informática para la UCLM publicada por Jesús Salido en el repositorio público Zenodo, DOI: 10.5281/zenodo.4561708.

A Pedro Antonio Trujillo Olivares
Por guiarme, cuidarme y apoyarme toda mi infancia

TRABAJO FIN DE GRADO - ESCUELA SUPERIOR DE INFORMÁTICA (UCLM)

Resumen

Pedro Antonio Trujillo Felipe
Ciudad Real, julio 2026

Resumen

La enfermedad renal crónica obliga a los pacientes a someterse a largas sesiones de hemodiálisis, lo que provoca sedentarismo, atrofia muscular y un profundo desgaste emocional. Aunque la cinesiterapia intradialítica mitiga estos efectos físicos, la realización de ejercicios en camilla suele ser monótona, lo que deriva en una alta tasa de abandono. Este Trabajo Fin de Grado aborda esta problemática proponiendo una solución tecnológica basada en la gamificación para fomentar el ejercicio físico moderado y mejorar el estado anímico del paciente durante su tratamiento.

Para dar respuesta a estas necesidades, se ha diseñado y desarrollado un ecosistema de *software* compuesto por aplicaciones inmersivas de Realidad Virtual (RV) y una plataforma web de gestión clínica. Utilizando visores autónomos Meta Quest 3 y el motor Unity, se han programado dos "juegos serios" terapéuticos adaptados a las severas restricciones de movilidad de la diálisis: un módulo para el tren superior (*ArkanoidVR*) y otro rítmico para el tren inferior (*Piernas VRítmicas*). Estos módulos operan bajo un patrón *Offline-First*, recopilando telemetría biomecánica del paciente de forma transparente. Posteriormente, una arquitectura web distribuida (desarrollada en Spring Boot y Angular) permite al personal sanitario ingerir y visualizar estos datos mediante cuadros de mando interactivos, operando mediante seudónimos como medida de seguridad y minimización de datos, en cumplimiento del Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD).

El ecosistema ha sido sometido a pruebas de experimentación y validado con éxito en el Hospital General Universitario de Ciudad Real (HGUCR). Los resultados concluyen que el sistema es ergonómico, seguro y altamente usable para los pacientes. La solución dota al personal de nefrología de métricas objetivas de rendimiento físico intradialítico, estableciendo una base tecnológica robusta para futuras investigaciones sobre el impacto del ejercicio en la calidad de vida de los enfermos renales.

Palabras clave: Escuela Superior de Informática, UCLM, TFG, Realidad Virtual, Juegos serios, Hemodiálisis.

BACHELOR DISSERTATION - ESCUELA SUPERIOR DE INFORMÁTICA (UCLM)

VR Application for Hemodialysis patients

Pedro Antonio Trujillo Felipe
Ciudad Real, julio 2026

Abstract

Chronic kidney disease requires patients to undergo long hemodialysis sessions, leading to physical inactivity, muscle atrophy, and profound emotional distress. Although intradialytic kinesitherapy mitigates these physical effects, performing exercises in a hospital bed is often monotonous, resulting in a high dropout rate. This Bachelor's Thesis addresses this issue by proposing a technology-driven solution based on gamification to encourage moderate physical exercise and improve the patient's mood during treatment.

To address these needs, a *software* ecosystem comprising immersive Virtual Reality (VR) applications and a clinical management web platform has been designed and developed. Using standalone Meta Quest 3 headsets and the Unity engine, two therapeutic "serious games" adapted to the severe mobility restrictions of dialysis have been programmed: an upper-body module (*ArkanoidVR*) and a rhythmic lower-body module (*Piernas VRítmicas*). These modules operate under an *Offline-First* pattern, seamlessly collecting the patient's biomechanical telemetry. Subsequently, a distributed web architecture (developed in Spring Boot and Angular) allows healthcare staff to ingest and visualize this data through interactive dashboards, operating through pseudonyms as a security and data minimization measure, in compliance with the General Data Protection Regulation (GDPR) and the Organic Law on Protection of Personal Data and Guarantee of Digital Rights (LOPDGDD).

The ecosystem has undergone experimental testing and was successfully validated at the Hospital General Universitario de Ciudad Real (HGUCR). The results conclude that the system is ergonomic, safe, and highly usable for patients. The solution provides nephrology staff with objective metrics of intradialytic physical performance, establishing a robust technological foundation for future research on the impact of exercise on the quality of life of renal patients.

Keywords: Escuela Superior de Informática, UCLM, TFG, Realidad Virtual, Juegos serios, Hemodiálisis.

Agradecimientos

Quiero comenzar agradeciendo a mi familia, tanto mi padre, Juan Antonio; mi madre, María del Prado; mi hermana Andrea y mis tíos y primos que siempre me han apoyado y ayudado en cuanto fuese necesario, sin importar el tiempo o el dinero que les costase.

A mis amigos, por estar siempre atentos y hacerme pasar unos buenos cuatro años de carrera, más los que sigan, además de conseguir que me sienta en casa cuando estoy con vosotros.

Y sobre todo, dar también gracias a mis tutores Javier y Vanesa por todo lo que se han preocupado por ayudarme con este Trabajo Fin de Grado; por todo lo que han confiado en mí y su sumo interés en conseguir que aprenda lo máximo posible durante la realización del trabajo, como conseguir que sea de la mayor calidad posible. Puede que yo no fuera el alumno que más fácil ha puesto las cosas respecto a los tiempos de realización, pero aún así os habéis esforzado en que pueda terminar a tiempo, y eso lo considero digno de admirar.

Pedro Antonio Trujillo Felipe
Ciudad Real, 2026

Notación y acrónimos

LISTA DE ACRÓNIMOS

Lista ordenada alfabéticamente con los acrónimos empleados en el texto.

ADB	:	*Android Debug Bridge*
AI	:	*Artificial Intelligence*
AIR	:	*Artificial Intelligence and Rendering*
API	:	*Application Programming Interface*
BPM	:	*Beats Per Minute*
CPU	:	*Central Processing Unit*
DoF	:	*Degrees of Freedom*
FOV	:	*Field of View*
HGUCR	:	Hospital General Universitario de Ciudad Real
HMD	:	*Head-Mounted Display*
HTTP	:	*Hypertext Transfer Protocol*
HUD	:	*Head-Up Display*
ID	:	Identificador
IoT	:	*Internet of Things*
JPA	:	*Java Persistence API*
JSON	:	*JavaScript Object Notation*
JWT	:	*JSON Web Token*
LOPDGDD	:	Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales
PC	:	*Personal Computer*
QA	:	*Quality Assurance*
RGPD	:	Reglamento General de Protección de Datos
ROM	:	*Range of Movement*
RV	:	Realidad Virtual
SDK	:	*Software Development Kits*
SQL	:	*Structured Query Language*
TFG	:	Trabajo Fin de Grado
UI	:	*User Interface*
USB	:	*Universal Serial Bus*
UX	:	*User Experience*
VFX	:	*Visual Effects*
VR	:	*Virtual Reality*
XR	:	*Extended Reality*

Índice general

Resumen vii

Abstract ix

Agradecimientos xi

Notación y acrónimos xiii

Índice de figuras xvii

Índice de tablas xix

Índice de listados xxi

1 Introducción 1
- 1.1 Contexto . . . 1
- 1.2 Motivación . . . 1
- 1.3 Propuesta . . . 2
- 1.4 Contexto disciplinar y tecnológico . . . 3
- 1.5 Estructura del documento . . . 4

2 Objetivos 5
- 2.1 Objetivo general . . . 5
- 2.2 Objetivos específicos . . . 5

3 Estado del Arte 7
- 3.1 Historia de la Realidad Virtual . . . 7
- 3.2 Fundamentos tecnológicos de la RV . . . 12
- 3.3 Fundamentación médica: cinesiterapia y ejercicio intradialítico . . . 14
- 3.4 Gamificación y juegos serios en el ámbito clínico . . . 15
- 3.5 Tecnologías de desarrollo y arquitectura de sistemas . . . 16
- 3.6 Juegos serios de rv en el contexto de la salud y la rehabilitación . . . 19
- 3.7 Marco legal: privacidad desde el diseño y protección de datos . . . 22

4 Metodología de trabajo 25
- 4.1 Comparativa de metodologías de desarrollo software . . . 25
- 4.2 Metodología ágil: enfoque iterativo e incremental . . . 25
- 4.3 Planificación y cronograma de desarrollo . . . 27
- 4.4 Recursos hardware y software . . . 29

4.5 Estudio de costes 30

5 Desarrollo y resultados **33**
5.1 Requisitos y análisis del sistema 33
5.2 Arquitectura del sistema 38
5.3 Modelo de datos, persistencia y telemetría 39
5.4 Implementación del sistema 42
5.5 Iteración 0: toma de contacto y definición de requisitos 42
5.6 Iteración 1: configuración del entorno, arquitectura base y seguridad . . . 43
5.7 Iteración 2: API REST, controladores y LOPDGDD 45
5.8 Iteración 3: prototipado del *Arkanoid*, ergonomía y telemetría 47
5.9 Iteración 4: retroalimentación sensorial y dificultad 55
5.10 Iteración 5: cierre del núcleo y calibración asimétrica 58
5.11 Iteración 6: refinamiento de UX y *Arkanoid* versión final 62
5.12 Iteración 7: desarrollo del tren inferior: juego de ritmo 64
5.13 Iteración 8: ecosistema web e ingesta de datos 74
5.14 Iteración 9: experimentación, pruebas y validación del sistema 81
5.15 Recopilación, publicación y estructura de datos biométricos 83
5.16 Acceso a los repositorios del proyecto 85

6 Conclusiones **87**
6.1 Objetivos alcanzados 87
6.2 Justificación de competencias adquiridas 88
6.3 Trabajos derivados y futuros 89
6.4 Valoración personal 89

Bibliografía **91**

A Informe de Colaboración y Validación del HGUCR **97**

Índice de figuras

3.1 El Sensorama de Morton Heilig, considerado el precursor analógico de la RV [7]. 8

3.2 Usuario interactuando con el prototipo de *la Espada de Damocles*. Nótese el brazo mecánico superior utilizado como hardware para el seguimiento espacial (*tracking*). (Fuente: https://proyectoidis.org/espada-de-damocles/). 9

3.3 Traje *DataSuit* desarrollado por VPL Research (circa 1989), equipado con sensores para registrar el movimiento de extremidades y torso. (Fuente: https://en.wikipedia.org/wiki/VPL_Research). 10

3.4 Ejemplos de intentos de comercialización de RV durante la década de 1990 que sufrieron problemas de cinetosis. (Fuentes: https://vandal.elespanol.com/loquepudoser/sega-vr y https://es.wikipedia.org/wiki/Virtual_Boy). 11

3.5 Escena interactiva de la aplicación clínica *SnowWorld*, donde el paciente se distrae lanzando bolas de nieve. (Fuente: R&D World [19]). 20

3.6 Interacción del usuario mediante seguimiento de manos (*hand tracking*) en la versión virtual del *Box and Block Test* (VR-BBT), desarrollada bajo la arquitectura *Rehab-Immersive*. Se aprecia el cambio de color visual como retroalimentación al realizar un agarre correcto. (Fuente: [9]). 22

4.1 Diagrama de Gantt mostrando las iteraciones y sus duraciones a lo largo del proyecto. 29

5.1 Diagrama general de casos de uso del ecosistema, separado por entorno inmersivo y plataforma web. 37

5.2 Diagrama de contenedores (modelo C4) ilustrando el desacoplamiento de la aplicación de RV y el flujo de subida mediante el cliente web. 40

5.3 Diagrama integrado del modelo de datos relacional y el almacenamiento estructurado en ficheros de telemetría y configuración. 41

5.4 Vista en primera persona del paciente proyectando el nivel de Arkanoid sobre el entorno relajante. 49

5.5 Esquema cinemático del centrado ergonómico adaptativo mediante centrado de vista. 52

5.6 Secuencia de retroalimentación visual al colisionar con un bloque. Se aprecia el destello blanco transitorio inyectado por la Corrutina y la posterior degradación de color del material. 56

5.7 Panel de notificación flotante (*Lazy HUD*) informando al paciente sobre los controles ergonómicos de centrado. 63

5.8 Arquitectura de componentes del sistema. Se ilustra la importación y reutilización de los *scripts* e interfaces del núcleo común para la construcción de los dos módulos terapéuticos independientes. 66
5.9 Vista inmersiva del túnel dinámico en el entorno "Gimnasio Zen", adaptado a la línea de visión del paciente y mostrando los indicadores de ritmo, puntuación y vida. 67
5.10 Colocación del mando derecho con el uso de la cinta elástica en la cara anterior del muslo, para el juego *Piernas VRítmicas*. 69
5.11 Modelos tridimensionales de las notas rítmicas. Se observa la reutilización del modelo de flecha mediante rotación y la codificación de colores asignada a cada postura motriz. 71
5.12 Vista del menú selector de niveles, con el nombre de la canción, el nombre del artista, la duración y el récord personal del jugador. 74
5.13 Mecánicas del módulo rítmico. En la fila superior, secuencia de aproximación de una nota hacia la zona de colisión. En la fila inferior, desplazamiento del cursor analógico en respuesta a la inclinación física de la pierna del paciente para lograr el encaje. 75
5.14 Interfaz del panel de administración en Angular para la gestión de enfermeros y privilegios de acceso. 77
5.15 Módulo de privacidad y LOPDGDD, mostrando la tabla dinámica de expedientes seudonimizados. 77
5.16 Vista principal del personal clínico, mostrando el listado de expedientes y la opción de nuevo registro. 78
5.17 Interfaz de subida de archivos mediante zonas de arrastre (*Drag & Drop*) segmentadas por tipo de dato. 79
5.18 Panel interactivo del expediente clínico, mostrando las gráficas de evolución generadas mediante Chart.js y la tabla de datos condicional. 80
5.19 Sesión de experimentación clínica y validación ergonómica llevada a cabo en las instalaciones del Hospital General Universitario de Ciudad Real. . . 82
5.20 Códigos QR y enlaces directos para el acceso a los *datasets* clínicos alojados en Zenodo. 85
5.21 Códigos QR para el acceso directo a los repositorios de código del ecosistema clínico. 85

Índice de tablas

4.1 Comparativa de características entre metodologías de desarrollo predictivas y ágiles.[20] . . . 26
4.2 Presupuesto estimado del proyecto. . . . 31

5.1 Tareas planificadas y completadas durante la configuración del entorno de desarrollo y la inicialización del servidor. . . . 44
5.2 Tareas planificadas y completadas durante la implementación de la seguridad JWT y el registro clínico. . . . 46
5.3 Tareas planificadas y completadas para el prototipado del juego de tren superior en RV. . . . 48
5.4 Tareas planificadas y completadas para la estabilización del primer minijuego inmersivo. . . . 55
5.5 Tareas planificadas y completadas para la consolidación del núcleo lógico y la telemetría biomecánica. . . . 59
5.6 Tareas planificadas y completadas para el cierre y pulido de la versión final del módulo de tren superior. . . . 62
5.7 Tareas planificadas y completadas para el desarrollo del ecosistema inmersivo del tren inferior. . . . 65
5.8 Tareas planificadas y completadas durante el desarrollo del ecosistema web y la refactorización del servidor. . . . 76

Índice de listados

5.1 Algoritmo de interpolación clínica para el movimiento virtual de la pala basado en el ROM del paciente. 50
5.2 Algoritmo predictivo para el cálculo anticipado de la zona de aterrizaje de la pelota simulando rebotes espaciales. 54
5.3 Algoritmo de seguimiento ocular suavizado (Lazy HUD) utilizando interpolación matemática escalar y rotacional. 63
5.4 Algoritmo de centrado ergonómico (*Túnel Dinámico*) basado en la proyección del vector direccional de la cabeza. 68
5.5 Algoritmo de detección biomecánica mediante la proyección vectorial del desplazamiento (Producto Escalar). 70
5.6 Algoritmo de cuantización y grabación de notas en tiempo real para el editor de niveles personalizado. 73

CAPÍTULO 1
Introducción

Este capítulo tiene como propósito explicar, de manera introductoria, la naturaleza de este proyecto. Primero, se dará a conocer el contexto en el que se enmarca, dando pie a describir la necesidad que este proyecto busca satisfacer. Por último, se elaborará una concepción general de la solución propuesta que abordaremos con detalle a lo largo de este documento.

1.1. CONTEXTO

La enfermedad renal crónica (ERC) constituye un importante problema de salud pública caracterizado por la pérdida progresiva e irreversible de la función de los riñones, cuyo propósito principal es la eliminación de toxinas y el exceso de líquido en el cuerpo [13].

Entre las causas más comunes que derivan en este daño renal destacan la diabetes, la hipertensión, la inflamación del riñón (glomerulonefritis) y diversas enfermedades hereditarias como la poliquistosis renal. Esta patología, a menudo silenciosa en sus primeras etapas, puede empeorar a lo largo de los años hasta alcanzar lo que se denomina enfermedad renal terminal (ERT).

Cuando la función renal desciende a niveles críticos, resulta necesario recurrir a terapias de sustitución para garantizar su supervivencia. Entre ellas, la hemodiálisis es un procedimiento médico vital que asume la función de los riñones sanos.

El tratamiento consiste en conectar al paciente a una máquina dializadora mediante un acceso vascular, que suele realizarse a través de una fístula en el brazo o un catéter venoso. La sangre circula a través de un filtro externo que elimina las toxinas y el agua retenida, devolviéndola limpia al organismo.

A pesar de que este tratamiento alarga y mejora la esperanza de vida de los pacientes, no constituye una cura y conlleva un gran impacto en su día a día. Las sesiones de hemodiálisis son necesarias prácticamente a diario o en días alternos, con una duración de 4 a 6 horas. Como consecuencia directa de esta rutina clínica, emergen dos problemáticas fundamentales que originan este proyecto: por un lado, una alta inactividad física al verse el paciente postrado en una camilla; y por otro, un bajo estado emocional, derivado del aburrimiento, la pasividad y el profundo desgaste físico y psicológico.

1.2. MOTIVACIÓN

Para hacer frente a esta inactividad y desgaste emocional, la introducción de ejercicio físico controlado durante las sesiones de hemodiálisis (ejercicio intradialítico) ha demostrado ser una herramienta terapéutica altamente beneficiosa. Físicamente, el ejercicio aeróbico y de fuerza reduce el riesgo cardiovascular, combate la atrofia muscular mejorando la capacidad

funcional e incrementa el aclaramiento de toxinas al aumentar el flujo sanguíneo [6]. A nivel psicológico y social, la realización de estas actividades disminuye la sintomatología de ansiedad y depresión, mejorando la calidad del sueño y la calidad de vida general del paciente [6].

Sin embargo, realizar ejercicios repetitivos en una camilla resulta monótono, lo que causa que un porcentaje significativo de los pacientes abandone el tratamiento [10]. De esta problemática surge la iniciativa del presente Trabajo Fin de Grado (TFG). El objetivo principal es reducir tanto la tasa de abandono de las sesiones como el malestar generado, integrando para ello una tecnología novedosa que fomente el ejercicio de forma lúdica: la Realidad Virtual (RV).

En la medicina ya existen múltiples usos de gran interés público y probada eficacia. Destacan, por ejemplo, las aplicaciones de RV orientadas a la distracción del dolor en pacientes sometidos a quimioterapia o curas de quemaduras graves, logrando lo que se conoce como una "analgesia virtual" a través de la inmersión [27]. A un nivel más cercano a nuestro entorno, cabe destacar la aplicación para la rehabilitación de miembros superiores en pacientes con lesión en la médula espinal, desarrollada por el grupo de investigación AIR en colaboración con el Hospital Nacional de Parapléjicos de Toledo. Dicho proyecto utiliza la Realidad Virtual para detectar el agarre de distintos objetos simulados, además de evaluar la capacidad de mantener dicho agarre al realizar diferentes movimientos, generando así estadísticas que permiten realizar un registro objetivo de la evolución del paciente.

A raíz de los beneficios observados en este tipo de intervenciones, surge la oportunidad de extrapolar estas soluciones interactivas a otras áreas clínicas con necesidades similares de evasión mental y estimulación física, como es el caso de la nefrología. Es en este punto donde convergen la necesidad médica de amenizar las salas de hemodiálisis y la experiencia tecnológica previa en "juegos serios" para la salud.

De este modo, este proyecto se enmarca dentro de una línea de investigación y colaboración conjunta entre el anteriormente mencionado grupo de investigación AIR y el Hospital General Universitario de Ciudad Real.

1.3. PROPUESTA

Para lograr este propósito, se ha optado por desarrollar un entorno inmersivo y atractivo diseñado para utilizarse de forma simultánea a la recepción del tratamiento médico. Las sesiones de RV planteadas tendrán una duración aproximada de 45 minutos, con el doble fin de ejercitar físicamente al paciente y ofrecerle entretenimiento, mejorando así su estado de ánimo y mitigando la inactividad. Cabe destacar que la duración de estas sesiones de "juego" ha sido acotada intencionadamente. Por un lado, se busca evitar la aparición de sensación de mareo inducido por la Realidad Virtual (conocido como *cybersickness*) en exposiciones prolongadas [18]; y por otro, se adapta a las limitaciones de autonomía de las propias gafas de Realidad Virtual.

Para cumplir con los objetivos mencionados, se ha estructurado el desarrollo de la aplicación en cuatro bloques de trabajo principales. Los dos bloques intermedios representan el núcleo de la carga de desarrollo y comparten similitudes en su arquitectura técnica.

El primer bloque está constituido por una experiencia visual de carácter introductorio y relajante. Su finalidad es ayudar al paciente a reducir la tensión y la ansiedad en los momentos previos al tratamiento, con el objetivo de reducir la percepción del dolor durante

la punción inicial necesaria para la hemodiálisis, además de servir de explicación para el uso durante la sesión de las gafas, y sus juegos.

El segundo bloque consiste en el desarrollo de un "juego serio" enfocado en la ejercitación del tren superior. El diseño de este módulo supone un reto técnico de accesibilidad, ya que debe adaptarse a restricciones físicas severas: algunos pacientes solo dispondrán de movilidad en un brazo (debido a la vía de la diálisis) y se encontrarán en una camilla con una inclinación de entre 45 y 90 grados. Para dar respuesta a estas limitaciones, se ha proyectado un entorno inspirado en las mecánicas del clásico *Arkanoid*, garantizando así un enfoque original respecto a los desarrollos previos realizados por el grupo AIR.

El tercer bloque se asemeja en gran parte al segundo en el hecho de que también consiste en el desarrollo de un "juego serio", pero se diferencia en que la parte que se busca ejercitar es el tren inferior. Hereda a su vez las mismas limitaciones de posición del paciente, además de incluir la necesidad de introducir maneras novedosas de sujetar los mandos de Realidad Virtual con las piernas, para que el juego pueda detectar los movimientos adecuadamente y registrar métricas del comportamiento del paciente de manera aceptable. Para esta solución, se ha planteado la implementación de un juego de ritmo musical, en el que se pide adoptar distintas posturas con las piernas según el instante de la canción en reproducción.

El cuarto y último bloque engloba el desarrollo de una aplicación web para los enfermeros a través de la cuál podrán consultar estadísticas de uso y todos los demás datos de interés recogidos por las gafas de Realidad Virtual, permitiendo a su vez mantener un registro de estas sesiones de uso.

1.4. CONTEXTO DISCIPLINAR Y TECNOLÓGICO

Para llevar a cabo los bloques descritos y dar respuesta a las estrictas restricciones del entorno clínico, el proyecto se fundamenta en la integración de tres áreas tecnológicas principales: hardware inmersivo, motores gráficos y desarrollo web estructurado.

En primer lugar, en el ámbito del hardware, se prescinde de los tradicionales sistemas de Realidad Virtual dependientes de un ordenador externo (*tethered VR*). En su lugar, el proyecto emplea visores autónomos (*standalone*), concretamente el modelo Meta Quest 3, equipados con sistemas de posicionamiento *inside-out*. Esta elección es indispensable en una sala de hemodiálisis, ya que la ausencia total de cableado y de balizas de seguimiento externas garantiza la seguridad del paciente y no interfiere con el instrumental médico ni con la movilidad del personal de enfermería.

En segundo lugar, para el diseño de las experiencias inmersivas y los juegos serios, se utiliza el motor gráfico Unity. Esta plataforma facilita el renderizado tridimensional y el cálculo de físicas en tiempo real, permitiendo además una integración nativa con los SDKs (*Software Development Kits*) específicos del visor. Gracias a esto, es posible trasladar con alta precisión los movimientos físicos del usuario al entorno virtual, un requisito crítico para evaluar el rendimiento motriz del paciente durante las partidas.

Finalmente, para dotar al proyecto de una herramienta de gestión clínica eficiente, se emplea un ecosistema de desarrollo web basado en una arquitectura cliente-servidor distribuida. La lógica de negocio y el procesamiento de las métricas biomecánicas y de uso generadas por las gafas se gestionan mediante un *Backend* robusto desarrollado en Java con el *framework* Spring Boot. Por su parte, la interfaz de usuario orientada al personal sanitario se despliega como una Aplicación de Página Única (SPA) construida con Angular,

ofreciendo un panel de control fluido, modular y seguro para la supervisión analítica de los tratamientos.

1.5. ESTRUCTURA DEL DOCUMENTO

1. **Introducción**. Este capítulo establece el contexto clínico y tecnológico que motiva la realización del Trabajo Fin de Grado, exponiendo la problemática actual de los pacientes de hemodiálisis y justificando la idoneidad de la Realidad Virtual como herramienta terapéutica.
2. **Objetivos**. En este capítulo se define de forma clara el propósito general del proyecto y se desglosan las metas técnicas y funcionales específicas necesarias para la construcción del ecosistema completo.
3. **Estado del Arte**. Se presenta una revisión bibliográfica y técnica de las tecnologías implicadas. Abarca desde la evolución del *hardware* inmersivo hasta el marco metodológico de los juegos serios orientados a la rehabilitación, estableciendo los precedentes que fundamentan las decisiones arquitectónicas del sistema.
4. **Metodología de trabajo**. En este capítulo se describe y justifica la metodología ágil adoptada para la organización del proyecto. Se detalla la planificación temporal, la división en iteraciones, los recursos empleados y se incluye una estimación económica del desarrollo.
5. **Desarrollo y resultados**. Constituye el núcleo técnico del documento. Se documenta cronológicamente el ciclo de vida del *software*: desde el análisis de requisitos y el diseño de la arquitectura distribuida, hasta la implementación técnica de las experiencias en Unity, el desarrollo de la plataforma web en Angular y la consolidación de la telemetría, culminando con los resultados obtenidos en las pruebas de validación en entorno real.
6. **Conclusiones**. Se realiza una evaluación crítica del grado de cumplimiento de los objetivos iniciales. Asimismo, se detallan las competencias académicas adquiridas, se proponen futuras líneas de investigación y se incluye una reflexión sobre el impacto personal y formativo del proyecto.
7. **Bibliografía**. Recopilación exhaustiva de todas las referencias académicas, artículos científicos, documentaciones técnicas y recursos digitales citados a lo largo del documento.
8. **Anexos**. Apartado destinado a incluir material complementario que respalda el desarrollo del proyecto, como el informe oficial de validación clínica emitido por el hospital.

CAPÍTULO 2
Objetivos

En este capítulo se define de manera concreta y detallada el objetivo del presente Trabajo Fin de Grado, acompañado, posteriormente, del conjunto de objetivos específicos necesarios para cumplir con el objetivo general.

2.1. OBJETIVO GENERAL

El objetivo general de este Trabajo de Fin de Grado es el diseño y desarrollo de un ecosistema de software compuesto por aplicaciones de Realidad Virtual (RV) inmersivas y una plataforma web de gestión, destinado a mitigar la inactividad y mejorar el estado anímico de los pacientes durante las sesiones clínicas de hemodiálisis. La solución busca ofrecer alternativas lúdico-terapéuticas adaptadas a las severas restricciones de movilidad de los usuarios en camilla, garantizando al mismo tiempo la recolección, persistencia local y gestión de métricas biomecánicas a través de seudónimos, en estricto cumplimiento con la normativa de protección de datos (LOPDGDD).

2.2. OBJETIVOS ESPECÍFICOS

La consecución del objetivo general propuesto requiere una descomposición natural y bien justificada del proyecto en elementos más simples, concretos y medibles. Dada la amplitud del ecosistema a desarrollar y la diversidad de las tecnologías implicadas, el diseño y desarrollo se ha estructurado en los siguientes objetivos específicos:

- **Análisis y adaptación a restricciones clínicas:** determinar los requisitos funcionales y no funcionales del sistema teniendo en consideración las limitaciones físicas y espaciales inherentes al tratamiento de hemodiálisis. Esto implica adaptar la interfaz y las mecánicas de la Realidad Virtual para usuarios que se encuentran recostados en una camilla con una inclinación de entre 45 y 90 grados, y que pueden presentar una inmovilización temporal o molestias en una de sus extremidades superiores debido al acceso vascular.
- **Desarrollo del módulo inmersivo de relajación:** diseñar e implementar una experiencia audiovisual introductoria, con la ayuda del Hospital General Universitario de Ciudad Real, cuyo propósito primordial sea disminuir la tensión y el estrés del paciente en los instantes previos a la sesión clínica, buscando mitigar de forma específica la percepción de dolor y la ansiedad asociadas a la punción inicial, además de servir como explicación de la sesión de uso de las gafas de Realidad Virtual.
- **Diseño e implementación del juego serio para la ejercitación del tren superior:** desarrollar un módulo interactivo basado en las mecánicas del clásico videojuego *Arkanoid*, adaptado a un entorno tridimensional. Este subobjetivo engloba la creación

de un sistema de calibración dinámica asimétrica que permita mapear el alcance de los movimientos de forma individualizada para cada brazo, garantizando la accesibilidad mediante esquemas de control mono-manuales o bi-manuales.

- **Diseño e implementación del juego serio para la ejercitación del tren inferior:** construir un juego terapéutico de ritmo musical que exija al paciente adoptar diversas posturas predefinidas (tales como reposo, extensión, inclinación lateral y elevación). Para lograrlo, se formulará e integrará un modelo matemático basado en la proyección de vectores espaciales que permita traducir con precisión el movimiento analógico de un único controlador acoplado a la pierna del paciente, evitando problemas de acoplamiento de ejes geométricos.
- **Desarrollo del núcleo de telemetría y persistencia local de datos:** programar un módulo subyacente encargado de la recolección no intrusiva de métricas biomecánicas durante las partidas, tales como el rango de movimiento, índices de simetría, detección de fatiga y tiempo de reacción cognitivo-motor. El diseño debe asegurar la persistencia local de la información en el propio hardware de las gafas mediante archivos estructurados (`.json` y `.csv`), garantizando un funcionamiento autónomo del entorno virtual con independencia del acceso a internet provisto a las gafas, que en hospitales puede ser negado.
- **Desarrollo de la plataforma web de gestión clínica e integración de privacidad desde el diseño:** diseñar e implementar una arquitectura de software distribuida (Frontend y Backend) que permita al personal sanitario supervisar las sesiones y analizar las estadísticas clínicas. Este objetivo conlleva el diseño de una base de datos relacional y una lógica de negocio adaptadas para operar exclusivamente mediante identificadores a modo de seudónimos generados aleatoriamente, desvinculando de forma irreversible cualquier dato de carácter personal de los historiales de uso, cumpliendo de forma estricta con la normativa vigente en materia de protección de datos (LOPDGDD).

CAPÍTULO 3
Estado del Arte

La concepción y el desarrollo de un ecosistema de software orientado al ámbito hospitalario exige un análisis profundo que trascienda la mera revisión informática. Por ello, el presente capítulo no constituye únicamente un recorrido por la evolución de la tecnología inmersiva, sino que plantea un estudio del estado del arte multidisciplinar donde la bioingeniería, la práctica clínica y el diseño interactivo convergen para dar respuesta a una problemática médica real.

Para fundamentar la viabilidad y pertinencia de la solución propuesta en este Trabajo de Fin de Grado, la revisión bibliográfica e investigativa se ha estructurado en torno a tres pilares fundamentales:

En primer lugar, se abordará el **marco tecnológico**, trazando la evolución histórica de la Realidad Virtual hasta llegar a la consolidación del actual espectro de la Realidad Extendida (XR) y los visores autónomos. Se analizarán los fundamentos físicos de la inmersión, el seguimiento espacial y las arquitecturas de red necesarias para operar en entornos restringidos.

En segundo lugar, se establecerá el **marco clínico**, analizando el contexto de la enfermedad renal crónica, las severas limitaciones impuestas por el tratamiento de hemodiálisis y, de forma crítica, la fundamentación médica que avala los beneficios fisiológicos y psicológicos del ejercicio intradialítico activo.

Finalmente, se explorará el **marco metodológico e interaccional**, profundizando en los principios de la gamificación y el diseño de "juegos serios" (*Serious Games*) orientados a la salud. Se estudiarán los antecedentes científicos de éxito en la rehabilitación motora y el manejo cognitivo del dolor (analgesia virtual), demostrando cómo un software de entretenimiento puede actuar de manera simultánea como una herramienta de telemetría pasiva.

La intersección coherente de estas tres disciplinas proporcionará la justificación científica y arquitectónica que sustenta las decisiones de ingeniería adoptadas en la fase de desarrollo del proyecto.

3.1. HISTORIA DE LA REALIDAD VIRTUAL

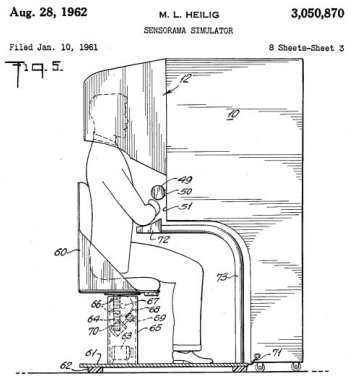
3.1.1. El origen de la inmersión: Sensorama (1957)

En la búsqueda de una experiencia cinematográfica totalmente inmersiva, el inventor y cineasta Morton Heilig diseñó en 1957 (obteniendo su patente en 1962) la que es ampliamente considerada como la primera máquina de RV de la historia: el Sensorama [7]. A diferencia de los sistemas de RV contemporáneos, que basan su funcionamiento en el procesamiento

computacional en tiempo real, el Sensorama era un dispositivo mecánico y analógico con el aspecto exterior de una cabina o máquina recreativa tradicional.



(a) Máquina Sensorama original.



(b) Plano de la patente (1962).

Figura 3.1: El Sensorama de Morton Heilig, considerado el precursor analógico de la RV [7].

El propósito principal de Heilig era trascender los límites audiovisuales del cine convencional y estimular de forma simultánea múltiples sentidos del usuario. Para lograr esta inmersión, la máquina estaba equipada con un visor que albergaba una pantalla estereoscópica a color, proporcionando un efecto tridimensional y visión periférica. Sin embargo, su mayor innovación tecnológica residía en su enfoque multisensorial: el dispositivo integraba un sistema de sonido estéreo direccional, ventiladores para simular la fricción del viento, un asiento móvil que replicaba las vibraciones del entorno y un emisor de aromas sincronizado con la acción de la pantalla [7].

Para dotar de contenido a su invención, Heilig desarrolló de forma paralela una cámara estereoscópica con la que grabó seis cortometrajes específicos, destacando entre ellos la simulación de conducción de una motocicleta. Además, el equipo incorporaba un panel de control que introducía un grado de interacción rudimentaria. Esta combinación de visualización 3D, estimulación táctil, olfativa y capacidad de interacción sentó los cimientos conceptuales de lo que hoy definimos como RV, alcanzando un nivel de inmersión sensorial que gran parte de los visores comerciales actuales aún no han estandarizado de forma nativa [7].

3.1.2. El nacimiento del seguimiento espacial: la Espada de Damocles (1968)

Si bien el Sensorama fue pionero en la estimulación sensorial inmersiva, el salto definitivo hacia la RV computacional se produjo en 1968. En la Universidad de Harvard, los investigadores Ivan Sutherland y David Cohen desarrollaron el primer prototipo funcional de un visor de RV y aumentada conectado a un ordenador, formalmente conocido como HMD (*Head-Mounted Display*) [22]. Este hito tecnológico se fundamentó en la integración de dos avances previos: los programas interactivos bidimensionales desarrollados por Sutherland en el MIT (empleando el ordenador transistorizado TX-2) y los pioneros simuladores de vuelo tridimensionales basados en gráficos vectoriales programados por Cohen.

A nivel de hardware, el dispositivo, apodado “la Espada de Damocles”, utilizaba dos tubos de rayos catódicos (CRT) en miniatura. Mediante un complejo sistema óptico de espejos, estos componentes eran capaces de sobreimpresionar imágenes gráficas tridimensionales directamente sobre la visión de la escena real del usuario. Sin embargo, su innovación más trascendental para la historia de la RV fue la resolución del problema del posicionamiento

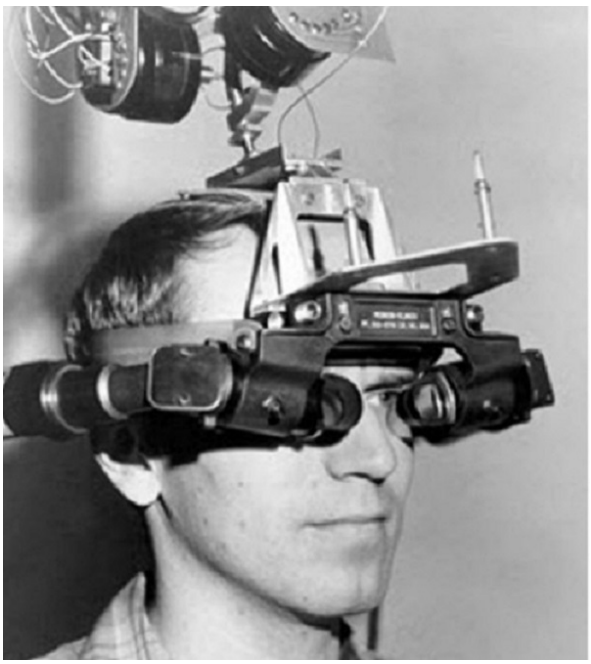


Figura 3.2: Usuario interactuando con el prototipo de *la Espada de Damocles*. Nótese el brazo mecánico superior utilizado como hardware para el seguimiento espacial (*tracking*). (Fuente: https://proyectoidis.org/espada-de-damocles/).

espacial (*tracking*). Debido al peso masivo de los componentes, el visor debía estar suspendido del techo mediante un brazo mecánico articulado. Este brazo no cumplía únicamente una función de soporte estructural, sino que actuaba como un sensor mecánico en tiempo real que registraba la posición y orientación exacta de la cabeza del usuario.

Esta rudimentaria lectura de coordenadas sentó las bases de los actuales sistemas de grados de libertad (DoF). Gracias a este brazo sensorizado, el ordenador podía recalcular la perspectiva de los gráficos vectoriales en tiempo real. Como resultado, el usuario podía percibir la estructura de un objeto tridimensional flotando en mitad del laboratorio y contemplar sus distintas caras de forma coherente a medida que se desplazaba físicamente por la habitación.

Aunque la Espada de Damocles no superó la fase de prototipo de laboratorio debido a sus obvias limitaciones ergonómicas y computacionales, estableció el paradigma técnico definitivo. El sistema aglutinó por primera vez la práctica totalidad de los pilares sobre los que se sustenta la arquitectura de la Realidad Virtual y Aumentada contemporánea: el uso de gráficos tridimensionales renderizados en tiempo real, la visualización estereoscópica y, fundamentalmente, la lectura de sensores de posición para permitir la navegación espacial alrededor de un objeto virtual [22].

3.1.3. La acuñación del término y la interacción: VPL Research (década de 1980)

Si bien los desarrollos de las décadas anteriores sentaron las bases de la visualización inmersiva y el seguimiento espacial, la RV carecía de una nomenclatura unificada y de periféricos orientados a la interacción directa con el entorno simulado. Esta carencia fue abordada en la década de 1980 por Jaron Lanier, fundador de la empresa VPL Research, a quien se le atribuye la popularización del término “Realidad Virtual” para agrupar todas estas tecnologías emergentes [31].

El aporte más significativo de VPL Research a la evolución del hardware fue la transición de la RV como una tecnología puramente visual a un medio interactivo. La compañía desarrolló y comercializó los primeros periféricos de entrada vestibles, destacando el *DataGlove* (un guante instrumentado capaz de registrar la flexión de los dedos y la posición de la mano) y visores HMD como el *EyePhone* [31].



Figura 3.3: Traje *DataSuit* desarrollado por VPL Research (circa 1989), equipado con sensores para registrar el movimiento de extremidades y torso. (Fuente: https://en.wikipedia.org/wiki/VPL_Research).

La introducción del *DataGlove* supuso un hito en la interfaz hombre-máquina. Por primera vez, el usuario no solo podía observar un mundo tridimensional generado por ordenador, sino que podía manipular objetos virtuales utilizando los movimientos naturales de sus manos. Esta innovación estableció el paradigma de la interacción basada en el seguimiento de extremidades, un concepto que resulta el pilar fundamental sobre el que operan los modernos controladores espaciales empleados en el desarrollo de los módulos de ejercitación física del presente proyecto.

3.1.4. El salto comercial y el "invierno" de la Realidad Virtual (década de 1990)

Durante la década de 1990, la RV experimentó su primer gran intento de democratización y llegada al mercado de consumo masivo. Compañías líderes en la industria del entretenimiento, como Sega y Nintendo, invirtieron fuertemente en el desarrollo de visores orientados al usuario doméstico, destacando prototipos como el Sega VR y el lanzamiento comercial de la consola *Virtual Boy*. Paralelamente, comenzaron a desplegarse costosas máquinas recreativas (*arcades*) que ofrecían experiencias inmersivas de corta duración en centros de entretenimiento.

Sin embargo, esta etapa (a menudo denominada en la literatura como el "invierno de la Realidad Virtual") culminó en un rotundo fracaso comercial y un estancamiento temporal de la industria. Las ambiciones de la época chocaron frontalmente con las limitaciones tecnológicas del hardware de la época. Los equipos carecían de la potencia de procesamiento y renderizado necesaria para generar gráficos tridimensionales de alta fidelidad, y el coste de manufactura impedía el acceso del gran público a componentes de calidad.



(a) Prototipo del visor Sega VR. **(b)** Consola Nintendo Virtual Boy.

Figura 3.4: Ejemplos de intentos de comercialización de RV durante la década de 1990 que sufrieron problemas de cinetosis. (Fuentes: https://vandal.elespanol.com/loquepudoser/sega-vr y https://es.wikipedia.org/wiki/Virtual_Boy).

El factor técnico más crítico que sentenció a esta generación de dispositivos fue la elevada latencia y la baja tasa de refresco de las pantallas. Esta precariedad técnica provocaba un desfase entre el movimiento de la cabeza del usuario y la actualización de la imagen, induciendo una profunda desorientación espacial, náuseas y fatiga visual; un síndrome clínicamente conocido como cinetosis o *cybersickness*.

Analizar estas carencias históricas resulta fundamental para la fundamentación de este Trabajo de Fin de Grado. El fracaso de los años 90 subraya la importancia crítica de mantener una alta fluidez gráfica sostenida en entornos de salud, y justifica empíricamente la decisión de acotar las sesiones de RV para los pacientes de hemodiálisis a un máximo de 45 minutos, con el fin de preservar su bienestar físico.

3.1.5. El renacimiento moderno y la era autónoma (2012 - Actualidad)

El resurgimiento de la RV como una tecnología viable y comercialmente exitosa se produjo a partir de 2012, liderado inicialmente por la campaña de financiación de Oculus Rift. A diferencia de los intentos fallidos de la década de 1990, esta nueva generación se benefició enormemente de la economía de escala y los avances de la industria de la telefonía móvil. La disponibilidad de pantallas de alta densidad de píxeles, procesadores eficientes y, sobre todo, unidades de medición inercial (IMU, compuestas por giroscopios y acelerómetros microscópicos) de bajo coste, permitió reducir drásticamente la latencia entre el movimiento físico y la actualización gráfica (*motion-to-photon latency*), mitigando de forma efectiva los mareos y la cinetosis.

En sus primeras iteraciones modernas (como HTC Vive u Oculus Rift CV1), los sistemas de alta fidelidad requerían estar conectados físicamente a un ordenador de alto rendimiento (*tethered VR*). Además, para lograr los seis grados de libertad (6DoF) necesarios para el movi-

miento espacial, dependían de un seguimiento *outside-in*, es decir, requerían la instalación de sensores o cámaras externas en las paredes de la habitación para rastrear la posición del usuario.

El verdadero cambio de paradigma, y el que hace tecnológicamente viable este Trabajo de Fin de Grado, ocurrió con la introducción de los visores autónomos o *standalone* (como la línea Meta Quest empleada en este proyecto). Estos dispositivos eliminan por completo la dependencia de ordenadores externos al integrar arquitecturas de procesamiento móvil (típicamente SoCs de la familia Qualcomm Snapdragon) y baterías directamente en el chasis del visor.

A nivel de seguimiento espacial, esta generación sustituyó las balizas externas por el sistema de *tracking inside-out*. Mediante un conjunto de cámaras infrarrojas integradas en la propia carcasa y algoritmos avanzados de Visión Computacional y SLAM (*Simultaneous Localization and Mapping*), el visor es capaz de mapear su entorno y calcular su posición exacta en tiempo real.

La adopción de tecnología *standalone* con seguimiento *inside-out* no es una mera elección de conveniencia para este proyecto, sino una exigencia arquitectónica impuesta por el entorno clínico. En una sala de hemodiálisis, la presencia de cables de conexión a ordenadores o la necesidad de calibrar sensores en las paredes resulta incompatible con el trasiego del personal médico, las camillas y las propias máquinas dializadoras. Por tanto, el uso de gafas como las Meta Quest 2 permite ofrecer una inmersión completa y un registro preciso de los movimientos del paciente de forma segura, higiénica y no intrusiva.

3.2. FUNDAMENTOS TECNOLÓGICOS DE LA RV

3.2.1. El espectro de la Realidad Extendida (XR) y niveles de inmersión

A medida que las tecnologías de visualización han evolucionado, la línea que separa el mundo físico del digital se ha difuminado. Para comprender y categorizar adecuadamente las herramientas disponibles en el ámbito clínico, la literatura académica se apoya en el modelo del "Continuo de la Realidad-Virtualidad" propuesto por Paul Milgram y Fumio Kishino en 1994. Este marco teórico establece una escala lineal que abarca desde el entorno físico real (sin alteraciones digitales) hasta el entorno virtual puro [17].

En la actualidad, la industria y la comunidad investigadora han adoptado el término paraguas Realidad Extendida (XR, por sus siglas en inglés, *Extended Reality*) para englobar todas las tecnologías que operan dentro de este continuo [24]. Atendiendo a su grado de inmersión y aislamiento del mundo real, el espectro XR se clasifica en los siguientes niveles tecnológicos:

- **Realidad Virtual no inmersiva (Desktop VR):** el usuario interactúa con un entorno tridimensional a través de una pantalla plana convencional (monitor o televisión), utilizando periféricos tradicionales (ratón, teclado o mandos).
  - *Ventajas:* alta accesibilidad, coste reducido y nula incidencia de cinetosis.
  - *Desventajas:* la inmersión y la sensación de presencia son mínimas. El usuario es plenamente consciente de su entorno físico en todo momento.
- **Realidad Mixta (MR) y Realidad Aumentada (AR):** estas tecnologías superponen e integran elementos digitales tridimensionales sobre la visión del mundo físico del usuario en tiempo real (mediante cristales transparentes o cámaras de *Passthrough*).

  - *Ventajas:* permite al usuario mantener la conciencia espacial, interactuar con otras personas (como el personal médico) y desplazarse por la habitación sin riesgo de colisión con objetos físicos.
  - *Desventajas:* el usuario permanece visual y cognitivamente "anclado" a la realidad física de su entorno inmediato.
- **Realidad Virtual Inmersiva:** mediante el uso de visores cerrados (HMD), se ocluye por completo el campo de visión del usuario, sustituyendo el mundo real por un entorno digital tridimensional renderizado en tiempo real.
  - *Ventajas:* genera un nivel máximo de presencia psicológica y aislamiento (escapismo).
  - *Desventajas:* elimina la conciencia espacial, generando un riesgo físico de choque o caída si el usuario se desplaza sin supervisión. Requiere hardware de alta fidelidad para evitar mareos inducidos por la latencia.

Justificación de la elección tecnológica en el contexto clínico

La decisión de desarrollar el presente Trabajo de Fin de Grado utilizando Realidad Virtual Inmersiva, descartando enfoques como la Realidad Mixta (a pesar de estar soportada nativamente por las gafas Meta Quest 3), obedece a una justificación estrictamente médica y psicológica.

El objetivo principal de este ecosistema ludificado no es únicamente la ejercitación motora, sino la mitigación del estrés, el aburrimiento y el dolor inherentes a las sesiones de hemodiálisis de hasta cuatro horas de duración. Si se implementara una solución en Realidad Mixta, el paciente observaría los bloques del juego *Arkanoid* flotando sobre la sala del hospital; seguiría viendo la máquina dializadora, el tránsito de enfermeros, el entorno aséptico de la planta y, críticamente, la vía venosa canalizada en su propio brazo. Este anclaje visual a la realidad médica destruiría la "analgesia virtual", impidiendo que el paciente logre abstraerse de su condición de enfermo.

Al optar por la Realidad Virtual Inmersiva, se aísla al paciente de los estímulos estresantes del hospital, transportándolo por completo a entornos relajantes y estimulantes (como un bosque o un gimnasio zen). Por otro lado, la principal desventaja de este enfoque (el riesgo físico de colisión al estar cegado al mundo real) queda totalmente anulada en este escenario clínico, dado que el paciente se encuentra inmovilizado y seguro sobre una camilla de tratamiento. Por tanto, la inmersión total se consolida como la tecnología óptima y de menor riesgo para alcanzar el estado de flujo cognitivo (*flow*) y fomentar el ejercicio intradialítico.

3.2.2. Inmersión y presencia

La RV contemporánea se define como una simulación interactiva generada por ordenador que sumerge al usuario en un entorno tridimensional computacional. Para analizar el estado actual de esta tecnología, la literatura científica exige distinguir entre dos conceptos fundamentales: la inmersión y la presencia [23].

Por un lado, la **inmersión** es una métrica objetiva ligada puramente a la arquitectura de hardware y software. Describe la capacidad del sistema para aislar al usuario del mundo físico y sustituir sus estímulos sensoriales por equivalentes sintéticos. El grado de inmersión está dictaminado por las especificaciones técnicas del visor: la resolución por ojo, el campo

de visión (FOV, *Field of View*), la tasa de refresco (medida en Hercios) y la latencia del sistema (*motion-to-photon latency*).

Por otro lado, la **presencia** es el fenómeno psicológico y subjetivo resultante de una alta inmersión. Mel Slater la subdivide en la "ilusión de estar allí" (*Place Illusion*) y la ilusión de que los eventos que ocurren en el entorno virtual son reales (*Plausibility Illusion*) [23]. En el entorno clínico de este proyecto, alcanzar un alto grado de presencia es el requisito ineludible para desencadenar la "analgesia virtual"; un mecanismo cognitivo que satura las vías de procesamiento sensorial del cerebro, distrayendo efectivamente al paciente del dolor y la incomodidad inherentes a la hemodiálisis.

3.2.3. Grados de libertad (DoF) y posicionamiento espacial (*Tracking*)

La capacidad de un sistema inmersivo para mapear el movimiento físico del usuario y trasladarlo al entorno tridimensional se cuantifica mediante los Grados de Libertad (DoF, *Degrees of Freedom*).

Los visores multimedia operan tradicionalmente con **3 DoF**, registrando únicamente las rotaciones geométricas de la cabeza sobre sus propios ejes espaciales (cabeceo, guiñada y alabeo). Sin embargo, el estado del arte actual para aplicaciones de gamificación biomecánica exige el estándar **6 DoF**. Un sistema 6 DoF procesa matrices de rotación y vectores de traslación espacial (movimiento en los ejes X, Y, Z), calculando la posición absoluta tanto del visor como de los controladores hápticos [14].

Como se adelantó en el recorrido histórico, el ecosistema *Standalone* actual (como el empleado en este proyecto) resuelve el cálculo 6 DoF mediante la tecnología ***Tracking Inside-Out***. A diferencia de los sistemas ópticos tradicionales que dependen de balizas externas (*Outside-In*), los visores modernos integran Unidades de Medición Inercial (IMU) de alta frecuencia y múltiples cámaras infrarrojas en su chasis. Mediante la aplicación concurrente de algoritmos de Visión Computacional orientados a la extracción de características (*Feature Extraction*) y técnicas de Localización y Mapeo Simultáneos (SLAM), el procesador del visor es capaz de reconstruir un mapa de profundidad de la habitación y triangular su posición exacta en milisegundos, sin depender de hardware perimetral.

3.3. FUNDAMENTACIÓN MÉDICA: CINESITERAPIA Y EJERCICIO INTRADIALÍTICO

La aplicación de la RV en este proyecto no persigue un fin puramente lúdico, sino que se fundamenta en los principios de la cinesiterapia: el uso del movimiento corporal con fines terapéuticos. En el contexto de la enfermedad renal crónica, la literatura médica actual avala ampliamente la introducción de rutinas de ejercicio durante las sesiones de hemodiálisis (ejercicio intradialítico) para mitigar el deterioro muscular y mejorar la eficiencia del tratamiento [11].

A nivel fisiológico, la hemodiálisis limpia las toxinas de la sangre, pero su eficacia está limitada por la velocidad a la que estas sustancias (como la urea o el fósforo) se desplazan desde los tejidos musculares hacia el compartimento vascular. El ejercicio físico controlado aumenta el flujo sanguíneo y abre lechos capilares en los músculos en movimiento [11]. Este incremento en la perfusión tisular facilita un mayor arrastre de toxinas hacia la sangre que está siendo filtrada por la máquina dializadora, mejorando de forma medible la adecuación de la diálisis y el aclaramiento de solutos [11].

Asimismo, intervenciones clínicas han demostrado que el ejercicio aeróbico y de fuerza-resistencia aplicado en las primeras horas de la diálisis revierte parcialmente la miopatía urémica y reduce marcadores de inflamación crónica [6]. Estas evidencias justifican la necesidad de desarrollar módulos de ejercitación específicos para el tren superior e inferior, utilizando dinámicas de juego para garantizar que el paciente mantenga un nivel de actividad sostenido que reporte beneficios clínicos reales.

3.4. GAMIFICACIÓN Y JUEGOS SERIOS EN EL ÁMBITO CLÍNICO

3.4.1. Diseño orientado a la rehabilitación (*Serious Games*)

En la disciplina de la informática en salud, resulta crítico establecer la frontera entre el software de entretenimiento comercial y el concepto de "juego serio" (*Serious Game*). Según la definición formal establecida por Michael Zyda, los juegos serios aplican dinámicas lúdicas a simulaciones interactivas cuyo propósito final trasciende el entretenimiento, abarcando áreas formativas y terapéuticas [32].

En el subsector de los *Games for Health*, la arquitectura del software se subordina a estrictas restricciones clínicas. Mientras que un videojuego tradicional calibra sus motores de físicas y colisiones para maximizar el desafío (pudiendo generar frustración), un juego serio clínico prioriza la accesibilidad, la seguridad y la rehabilitación motora. En el contexto de este trabajo fin de grado, las mecánicas desarrolladas no buscan medir la destreza competitiva del paciente, sino forzar la adopción de posturas isométricas y movimientos articulares dentro de un rango seguro para combatir la atrofia muscular [30].

3.4.2. El estado de flujo y la telemetría médica

La gamificación se define metodológicamente como la extracción y aplicación de mecánicas de juego en contextos ajenos a este, con el fin de modificar el comportamiento del usuario [5]. En la nefrología, donde la falta de adherencia al tratamiento por aburrimiento y pasividad es un factor de riesgo crítico, la gamificación se emplea como herramienta de choque.

La implementación de bucles de retroalimentación positiva (*feedback loops*), sistemas de puntuación dinámica y recompensas audiovisuales facilita que el paciente alcance un "estado de flujo" (*flow*) [16]. Este estado de inmersión cognitiva mitiga la vulnerabilidad sentida en la camilla de diálisis. Además, el estado del arte de los juegos serios actuales no se limita a la estimulación visual, sino que actúan como herramientas de telemetría pasiva. Las sesiones ludificadas permiten la recolección de datos biomecánicos en tiempo real (tiempos de reacción, rango de movimiento, temblores), transformando el tiempo de ocio en una sesión de análisis clínico objetivo.

3.4.3. Fundamentación biomecánica de las interacciones desarrolladas

El diseño de las mecánicas lúdicas de este ecosistema se ha subordinado a los principios de la cinesiterapia activa asistida. En lugar de adaptar videojuegos comerciales genéricos, la carga de desarrollo en el motor Unity se centró en programar interacciones que fuercen movimientos articulares terapéuticos específicos, mitigando la atrofia muscular propia de los pacientes en hemodiálisis.

Para el tren superior, el desarrollo se fundamenta en las mecánicas de un juego clásico de bloques (tipo *Arkanoid*), lo que en la literatura de rehabilitación neurológica y motora se clasifica como "tareas de alcance interactivo" (*Interactive Reaching Tasks*) [15]. Al obligar al paciente a desplazar una pala virtual para interceptar un objeto en movimiento, el *software*

fomenta extensiones de codo y abducciones de hombro repetitivas. La ventaja clínica de implementar esta mecánica en RV es que el motor gráfico permite manipular dinámicamente la zona de aparición de los elementos, forzando al paciente a explorar los límites de su rango de movimiento (ROM) máximo sin percibir el esfuerzo como un ejercicio clínico tedioso [15].

Para el tren inferior, la carga de programación se orientó hacia un juego de ritmo musical. Esta decisión arquitectónica se apoya en el principio clínico de la estimulación rítmica auditiva (RAS, por sus siglas en inglés) [26]. La sincronización de posturas isométricas de las piernas con marcadores visuales y pulsos musicales activa redes neuronales auditivo-motoras que facilitan la coordinación del movimiento y reducen la fatiga percibida. El algoritmo de proyección de vectores espaciales desarrollado para este módulo se encarga de verificar que la amplitud del movimiento del muslo coincide exactamente con la cadencia exigida por el patrón rítmico, proporcionando una métrica objetiva de la resistencia y control motor del paciente a lo largo de los 45 minutos de sesión.

3.5. TECNOLOGÍAS DE DESARROLLO Y ARQUITECTURA DE SISTEMAS

3.5.1. Motores de desarrollo para Realidad Extendida (XR)

El desarrollo de aplicaciones inmersivas requiere el uso de motores gráficos avanzados capaces de gestionar el renderizado tridimensional, las físicas y la captura de sensores en tiempo real. En el panorama actual de la investigación y desarrollo en Realidad Extendida (XR), las dos grandes alternativas de la industria son Unreal Engine (desarrollado por Epic Games) y Unity (desarrollado por Unity Technologies).

Por un lado, Unreal Engine destaca por su fotorrealismo, su motor de iluminación global y su potente sistema de programación visual (*Blueprints*), siendo el estándar en producciones de alto presupuesto y simulaciones de alta fidelidad. Sin embargo, su curva de aprendizaje es pronunciada y el peso computacional de su motor de renderizado resulta menos eficiente para dispositivos autónomos de RV (*Standalone VR*) que cuentan con recursos de procesamiento móvil limitados[25].

Por otro lado, Unity ofrece una arquitectura más ligera y accesible basada en el lenguaje de programación C#. En la actualidad, Unity se ha consolidado como la plataforma predominante en el ecosistema inmersivo, estimándose que impulsa el 60 % de las experiencias de RV y Aumentada desarrolladas para visores móviles como la familia Meta Quest[2].

Tras analizar ambas alternativas, se seleccionó Unity como el motor principal para este Trabajo de Fin de Grado debido a las siguientes ventajas tecnológicas y logísticas:

Soporte oficial y SDK nativo: Unity cuenta con una integración profunda, madura y consolidada con el *Meta XR Core SDK*. Este soporte nativo garantiza un acceso seguro a los *Building Blocks* oficiales de Meta, reduciendo drásticamente el riesgo técnico durante el desarrollo e implementación de mecánicas complejas como el rastreo espacial (*Tracking*) y la lectura de los controladores.

Arquitectura orientada a componentes: a diferencia de proyectos masivos que requieren el nuevo modelo orientado a datos ECS (*Entity-Component-System* / DOTS) para procesar miles de entidades simultáneas, este proyecto aprovecha la arquitectura clásica y sumamente estable de Unity basada en `GameObjects` y componentes `MonoBehaviour`. Este paradigma resulta idóneo para el prototipado clínico, permitiendo acoplar lógicas específicas (como la interpolación de movimiento del paciente o la telemetría) a objetos tridimensionales de forma modular y altamente escalable.

Gestión nativa de Prefabs: Unity facilita la creación de niveles repetitivos mediante su sistema de *Prefabs*. Un *Prefab* actúa como una plantilla de objeto tridimensional preconfigurada y reutilizable que permite su instanciación dinámica en memoria durante el tiempo de ejecución. Esta característica fue clave para diseñar la arquitectura escalable de los niveles de Arkanoid y el generador asíncrono de notas del juego rítmico.

Ecosistema de paquetes y comunidad: Unity destaca por poseer la mayor comunidad de desarrolladores de XR del mundo, lo que se traduce en una extensa documentación técnica que mitiga los bloqueos de desarrollo. Asimismo, la madurez de su motor de físicas (*PhysX*) y el acceso a la *Unity Asset Store* permitieron integrar modelos, materiales y paquetes de terceros de forma inmediata, acelerando las fases de diseño para focalizar el esfuerzo del TFG en la ingeniería *software* y la lógica terapéutica.

3.5.2. Arquitectura web, persistencia y tecnologías de desarrollo

Para complementar el ecosistema inmersivo, el proyecto requiere de una plataforma web capaz de centralizar los expedientes, procesar la telemetría masiva y exponer cuadros de mando clínicos. En el diseño de este tipo de sistemas de salud (*eHealth*), confluyen decisiones críticas sobre la topología de red, la arquitectura de *software* y las herramientas de desarrollo empleadas.

Arquitectura cliente-servidor y patrón Offline-First

En la industria actual, existe una tendencia hacia el uso de arquitecturas orientadas a microservicios e ingestas de telemetría en tiempo real (*streaming*) procedentes de dispositivos IoT. No obstante, el despliegue de estas topologías en entornos hospitalarios reales presenta severos inconvenientes logísticos. Las salas de hemodiálisis suelen contar con infraestructuras de red inalámbrica restringidas (por políticas de seguridad perimetral) o inestables, lo que provocaría pérdidas irreparables de datos clínicos si los visores dependieran de una conexión continua al servidor.

Por consiguiente, para este proyecto se ha adoptado una arquitectura de ingesta de datos bajo el patrón *Offline-First* (sincronización diferida). El visor de RV opera de manera completamente autónoma, persistiendo los historiales y la telemetría en archivos locales inmutables (JSON y CSV). El flujo de vida de los datos se completa mediante un proceso asíncrono y seguro: el personal sanitario extrae los archivos físicamente vía USB tras la sesión y los sube a la plataforma web. Este enfoque elimina la dependencia de la red, garantiza la integridad absoluta de los datos biométricos y blinda a los visores frente a posibles ciberataques de red[1].

Para soportar este flujo, se ha descartado la hiper-segmentación por microservicios en favor de una arquitectura clásica de tres capas (cliente, servidor de aplicaciones y base de datos relacional). Este modelo monolítico modular resulta ser la solución óptima y más fundamentada científicamente para el desarrollo de un Producto Mínimo Viable (PMV) clínico, ya que simplifica drásticamente el despliegue (*deployment*) en los servidores locales (*on-premise*) del hospital, reduce la latencia interna y facilita el mantenimiento a largo plazo por parte de los equipos de soporte informático del centro[12].

Comparativa y selección de tecnologías Backend

El Servidor de Aplicaciones es el responsable de procesar la ingesta masiva de archivos físicos, autenticar al personal y servir la información al cliente web. Para su desarrollo, se evaluaron tres grandes alternativas del mercado: *Node.js* (con *Express*), *Django* (Python) y *Spring Boot* (Java).

Aunque *Node.js* ofrece un excelente rendimiento en operaciones de entrada/salida (*I/O*) gracias a su naturaleza orientada a eventos, su arquitectura de un solo hilo (*single-threaded*) resulta ineficiente para tareas intensivas de CPU, como el *parseo* simultáneo de archivos CSV masivos de telemetría. Por su parte, *Django* permite un prototipado extremadamente rápido, pero el tipado dinámico de Python puede derivar en errores en tiempo de ejecución, un riesgo inasumible al manipular registros médicos.

Finalmente, se seleccionó **Spring Boot**. Su ecosistema basado en Java proporciona un tipado estricto, una gestión multihilo robusta para la ingesta paralela de documentos pesados y una integración nativa e impecable con *Spring Security*, fundamental para implementar el filtro de autenticación por *tokens* JWT necesario para cumplir con los requisitos de seguridad de la LOPDGDD [4].

Para la capa de persistencia, se descartaron bases de datos NoSQL (como MongoDB) debido a que los datos clínicos están fuertemente estructurados. Se optó por el motor relacional **MySQL**, el cual garantiza las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) indispensables en salud, y permite modelar la compleja herencia polimórfica de las partidas (*@Inheritance JOINED*) mediante el ORM *Hibernate*.

Comparativa y selección de tecnologías Frontend

En cuanto a la interfaz de usuario para el personal de enfermería, se analizó la triada de tecnologías dominantes: *React*, *Vue.js* y *Angular*.

React (desarrollado por Meta) es la librería más popular por su flexibilidad, pero carece de estructura predefinida, obligando a ensamblar dependencias de terceros para el enrutamiento o el manejo de estados. *Vue.js* ofrece una curva de aprendizaje suave, pero su adopción en entornos empresariales altamente estructurados es menor.

La elección final recayó sobre **Angular** (desarrollado por Google). Al ser un *framework* completo y de opiniones fuertes (*opinionated*), proporciona una arquitectura MVC estandarizada, inyección de dependencias y enrutamiento nativo desde el primer momento. Además, su adopción obligatoria de *TypeScript* previene errores de interfaz durante el desarrollo, y su profunda integración con la librería reactiva *RxJS* ha sido crucial para orquestar con éxito la compleja subida asíncrona de múltiples archivos biométricos de forma paralela mediante el operador `forkJoin`.

3.5.2.1. Evolución frente a los sistemas de información hospitalaria (HIS) tradicionales

Históricamente, los hospitales han dependido de sistemas de información hospitalaria (HIS, por sus siglas en inglés) monolíticos, diseñados principalmente para la gestión administrativa y el almacenamiento estático de historiales médicos electrónicos (EHR). Sin embargo, la integración de dispositivos *wearables* y visores de RV ha introducido un nuevo paradigma: la ingesta masiva de telemetría continua de alta frecuencia, como variaciones angulares o tiempos de reacción medidos en milisegundos [21].

Los sistemas monolíticos tradicionales carecen de la elasticidad necesaria para procesar estos flujos de datos sin comprometer el rendimiento general de la red hospitalaria. Es en este contexto donde la arquitectura distribuida adoptada en este proyecto demuestra su idoneidad técnica.

El uso de Spring Boot en el *backend* aprovecha la robustez de la Máquina Virtual de Java (JVM) para la gestión eficiente de múltiples hilos de ejecución (*multithreading*), per-

mitiendo procesar y seudonimizar peticiones concurrentes de telemetría provenientes de múltiples visores Meta Quest 3 simultáneamente. Además, su ecosistema integral (mediante módulos como Spring Security) facilita la implementación estandarizada de protocolos de autenticación sin estado, críticos para aislar la red clínica de posibles vulnerabilidades en los dispositivos periféricos [21].

En la capa de presentación, la elección de Angular (basado en TypeScript) frente a librerías de interfaz gráfica convencionales se justifica por su tipado estricto. En el desarrollo de *software* médico, garantizar que las estructuras de datos de las constantes vitales y métricas biomecánicas no muten inesperadamente es una exigencia crítica de seguridad. Angular impone una arquitectura basada en componentes fuertemente tipados y servicios inyectables, lo que previene errores de ejecución en el navegador del personal de enfermería y asegura que las gráficas de evolución del paciente representen fielmente los datos almacenados en el servidor [8].

3.6. JUEGOS SERIOS DE RV EN EL CONTEXTO DE LA SALUD Y LA REHABILITACIÓN

Hasta este punto, se han analizado los fundamentos tecnológicos y teóricos que sustentan la inmersión, así como el potencial de los "juegos serios" y la telemetría pasiva. Sin embargo, la viabilidad de la RV como herramienta terapéutica no es una mera hipótesis de diseño, sino una disciplina consolidada a través de décadas de investigación clínica y desarrollo de software médico. Analizar los antecedentes existentes resulta fundamental para contextualizar la idoneidad y el enfoque de este Trabajo de Fin de Grado.

3.6.1. Manejo del dolor y analgesia virtual: SnowWorld

Uno de los mayores exponentes mundiales del uso clínico de la RV es el tratamiento del dolor agudo en las unidades de quemados. El proceso de curación y cuidado de heridas (como la retirada de grapas o la limpieza de tejido muerto) en víctimas de quemaduras es extremadamente doloroso, y los fármacos narcóticos suelen resultar insuficientes para enmascarar ese nivel de dolor intenso.

Para combatir este desafío médico, los investigadores Hunter Hoffman y David Patterson, del Human Interface Technology Laboratory de la Universidad de Washington, desarrollaron SnowWorld. Se trata de una experiencia inmersiva de RV diseñada específicamente para distraer a los pacientes con quemaduras durante sus intervenciones [19].

- **Mecánicas y diseño:** en la aplicación, los usuarios interactúan y suman puntos lanzando bolas de nieve a pingüinos, muñecos de nieve, peces voladores y mamuts. La temática del hielo y la nieve fue elegida intencionadamente por ser la antítesis del fuego; un entorno diametralmente opuesto que ayuda a que los pacientes fuertemente medicados eviten revivir recuerdos del accidente (y un posible estrés postraumático) durante el cuidado de las heridas. Para facilitar la accesibilidad, el control se realiza mediante el simple movimiento de la cabeza y la pulsación de un único botón [19].
- **Mecanismo de acción neurocognitiva:** el éxito clínico de la aplicación se fundamenta en la saturación atencional. Aproximadamente una cuarta parte del cerebro (el córtex visual) se dedica a procesar información visual. Al sumergir al paciente en un entorno virtual inmersivo, se agotan numerosos recursos cognitivos, lo que deja al cerebro con una capacidad de atención limitada para poder procesar las señales de dolor entrantes. Los escáneres cerebrales respaldan empíricamente esta teoría,

revelando una reducción real en la actividad cerebral vinculada directamente al dolor [19].

- **Eficacia y resultados:** la evidencia científica de su uso es contundente. Un estudio publicado en 2007 con 88 sujetos demostró que añadir esta distracción de RV resultó en una reducción del 20 % en la intensidad del dolor máximo subjetivo, una disminución del 26 % en la sensación de desagrado y una reducción del 37 % en el tiempo que el paciente pasaba pensando en el dolor [19].
- **Adaptación a hidroterapia:** en 2008, el hardware de *SnowWorld* se adaptó usando tecnología de fibra óptica para poder ser empleado en tanques de hidroterapia, donde se ablanda el tejido cicatricial. Este estudio evidenció que los pacientes lograban rebajar sus valoraciones de "dolor severo" a "dolor moderado", alcanzando incluso niveles de dolor "leve" en aquellos pacientes que experimentaban la mayor ilusión inmersiva de estar dentro del entorno virtual [19].



Figura 3.5: Escena interactiva de la aplicación clínica *SnowWorld*, donde el paciente se distrae lanzando bolas de nieve. (Fuente: R&D World [19]).

Originalmente, el hardware necesario para estas intervenciones era altamente privativo, con visores que costaban alrededor de 35.000 dólares. En la actualidad, la evolución del procesamiento móvil y los videojuegos ha permitido adaptar la experiencia para su uso clínico con visores comerciales accesibles, como Oculus Rift o HTC Vive [19].

3.6.2. Rehabilitación neurológica y telemetría: el ecosistema MindMaze

Mientras que aplicaciones como *SnowWorld* se centran en la distracción cognitiva frente al dolor agudo, otras vertientes de la RV clínica se enfocan en la recuperación física y la rehabilitación motora. En este ámbito destaca *MindMaze*, una compañía biotecnológica que ha desarrollado un ecosistema líder de neuroterapia de precisión centrado en la salud y recuperación cerebral [28].

A diferencia de las terapias físicas tradicionales, la plataforma de *MindMaze* propone una combinación innovadora de software interactivo, dispositivos de hardware y análisis masivo de datos para el tratamiento y gestión de enfermedades neurológicas [28]. Su enfoque

comercial y clínico resulta de especial interés para los fundamentos de este Trabajo de Fin de Grado debido a dos pilares metodológicos:

- **Telemetría clínica objetiva:** el sistema no se limita a ofrecer un entorno virtual inmersivo, sino que actúa como una herramienta de recolección intensiva de métricas biomecánicas. Se estima que la plataforma procesa aproximadamente 100 millones de puntos de datos mensuales [28]. Esta capacidad de monitorización valida académicamente la necesidad de integrar sistemas de telemetría y *dashboards* web en proyectos de salud ludificados, permitiendo a los profesionales médicos evaluar la evolución del paciente sin depender exclusivamente de la observación visual en planta.
- **Entrenamiento autónomo y adherencia:** uno de los grandes hitos de la plataforma es el fomento del tiempo de entrenamiento activo sin supervisión directa, el cual ha experimentado un crecimiento interanual del 43 % [28]. Este dato subraya la eficacia de los "juegos serios" para mantener la adherencia al tratamiento físico, empoderando al paciente para realizar sus rutinas de forma autónoma gracias a la retroalimentación inmediata (*feedback*) que proporciona el propio software [28].

La viabilidad tecnológica y la fiabilidad de este tipo de ecosistemas en la medicina moderna quedan respaldadas por su extensa adopción global. Hasta la fecha, las soluciones terapéuticas de *MindMaze* cuentan con 12 aprobaciones regulatorias, han sido validadas mediante más de 40 ensayos clínicos y han facilitado más de 200.000 sesiones de terapia a un volumen superior a los 35.000 pacientes [28].

3.6.3. Rehabilitación motora en lesiones medulares: Rehab-Immersive

Como antecedente directo y base tecnológica fundamental de este Trabajo de Fin de Grado, es necesario destacar el proyecto *Rehab-Immersive* [9]. Este *framework* de software ha sido desarrollado por el grupo de investigación Inteligencia Artificial y Representación (AIR) de la Universidad de Castilla-La Mancha, en estrecha colaboración con el personal clínico del Hospital Nacional de Parapléjicos de Toledo (HNPT) [9].

El propósito principal de *Rehab-Immersive* es proporcionar una arquitectura base para el desarrollo de aplicaciones clínicas de RV inmersiva orientadas a la rehabilitación del tren superior. Su diseño está específicamente enfocado en pacientes que sufren trastornos motores derivados de lesiones en la médula espinal (SCI), tales como la tetraplejia [9]. El sistema aborda las limitaciones físicas de estos pacientes mediante tres pilares de diseño clínico e interaccional:

- **Personalización clínica del entorno:** el sistema incluye una capa de configuración que permite adaptar el escenario virtual a las necesidades y limitaciones motoras específicas de cada paciente. Esta arquitectura permite ajustar la distancia y el tamaño de los objetos virtuales. Asimismo, incorpora un modo de agarre automático (*autogrip*) destinado a aquellos pacientes que, en las fases iniciales de su rehabilitación, aún no poseen la capacidad física para realizar pinzas o agarres funcionales [9].
- **Interacción sin controladores físicos:** para facilitar el uso clínico, el *framework* emplea tecnología comercial de bajo coste y prescinde de los controladores físicos tradicionales, apoyándose en el seguimiento de manos (*hand tracking*) integrado en visores como las Oculus/Meta Quest 2. El uso de mandos hápticos se descartó intencionadamente, ya que añaden un peso extra que dificulta el ejercicio. Además, los pacientes con este tipo de lesiones medulares suelen carecer de la fuerza necesaria para sujetarlos y presentan una sensibilidad táctil muy reducida, por lo que el sistema

opta por ofrecer retroalimentación puramente audiovisual. Con esta tecnología, el software es capaz de detectar múltiples tipos de agarres, incluido el agarre de tenodesis, característico de los pacientes con movilidad limitada en los dedos [9].

- **Telemetría y detección de compensaciones:** el sistema integra un módulo de persistencia de datos en la nube que almacena variables cinemáticas fundamentales para la evaluación médica. Entre los parámetros registrados se encuentran la velocidad de la mano, la fuerza de torsión de la muñeca y, de especial relevancia, la posición tridimensional de la cabeza. La monitorización de la cabeza permite al personal médico identificar si el paciente está realizando movimientos compensatorios incorrectos con el tronco para alcanzar un objeto, lo cual podría entorpecer su proceso de rehabilitación [9].

Para validar clínicamente esta arquitectura, el grupo de investigación desarrolló una versión inmersiva del *Box and Block Test* (VR-BBT), una prueba médica estandarizada para evaluar la destreza manual unilateral, cuyo entorno y mecánicas de interacción visual pueden observarse en la Figura 3.6 [9]. Las metodologías de interacción, adaptación a la movilidad reducida y persistencia de métricas establecidas por *Rehab-Immersive* sientan las bases conceptuales sobre las que se construyen los juegos serios de tren superior e inferior del presente proyecto orientados a la sala de hemodiálisis.

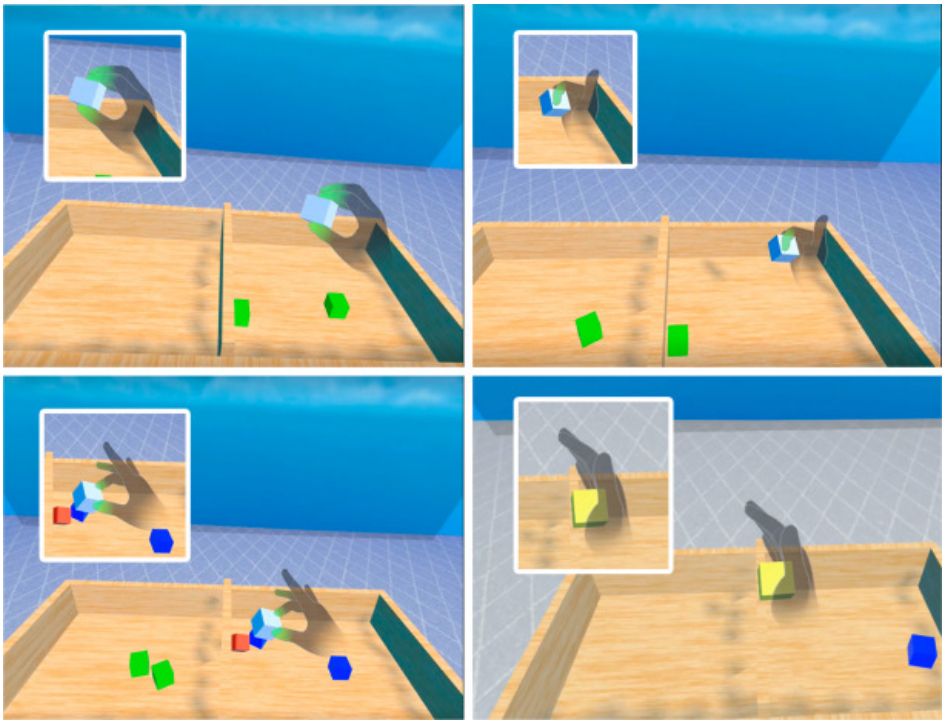


Figura 3.6: Interacción del usuario mediante seguimiento de manos (*hand tracking*) en la versión virtual del *Box and Block Test* (VR-BBT), desarrollada bajo la arquitectura *Rehab-Immersive*. Se aprecia el cambio de color visual como retroalimentación al realizar un agarre correcto. (Fuente: [9]).

3.7. MARCO LEGAL: PRIVACIDAD DESDE EL DISEÑO Y PROTECCIÓN DE DATOS

El desarrollo de cualquier sistema de software destinado a operar en un entorno clínico, aunque se encuentre en fase de prototipado, está sujeto a un estricto marco normativo. En el ámbito europeo y nacional, el tratamiento de la telemetría generada por los pacientes se

rige por el Reglamento General de Protección de Datos (RGPD) de la Unión Europea [29] y la Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [4].

Los datos relativos a la salud y la biometría (como el rango de movimiento o el tiempo de reacción registrados por los visores de RV) están tipificados como datos de categoría especial. Por ello, la arquitectura de este proyecto incorpora el principio legal de "privacidad desde el diseño" (*Privacy by Design*).

Este principio se implementa a través de la seudonimización y el aislamiento de la información. El sistema omite de forma deliberada la recolección de nombres, historiales médicos o datos de identificación personal directa (*PII, Personally Identifiable Information*). En su lugar, el *backend* genera de forma automatizada identificadores alfanuméricos aleatorios de seis caracteres para cada usuario, los cuales actúan como única clave de enlace con sus métricas. De este modo, se garantiza un canal seguro de telemetría hospitalaria que, aun no estando todavía desplegado en un entorno de producción real, previene teóricamente cualquier fuga de información sensible.

CAPÍTULO 4
Metodología de trabajo

Este capítulo detalla el marco metodológico adoptado para la planificación, diseño e implementación de las soluciones tecnológicas que conforman este Trabajo de Fin de Grado. De este modo, se exponen y justifican las decisiones tomadas respecto al ciclo de vida del software, adaptándolas a las necesidades de un entorno clínico y a las exigencias académicas.

4.1. COMPARATIVA DE METODOLOGÍAS DE DESARROLLO SOFTWARE

En el ámbito de la Ingeniería del Software, las metodologías de desarrollo proporcionan un conjunto de reglas, prácticas y procesos estandarizados para gestionar el ciclo de vida de un proyecto de forma eficiente [20]. Tradicionalmente, la industria del desarrollo se ha dividido en dos grandes vertientes metodológicas: los enfoques clásicos (o predictivos) y los enfoques ágiles (o adaptativos) [20].

Las metodologías tradicionales, como el modelo en Cascada (*Waterfall*), imponen un flujo de trabajo estrictamente secuencial. En este paradigma, no es posible avanzar a la siguiente fase (por ejemplo, la implementación del código) sin haber cerrado y documentado exhaustivamente la fase previa (análisis de requisitos y diseño). Por el contrario, las metodologías ágiles (tales como Scrum o Kanban) fragmentan el proyecto en ciclos de desarrollo cortos, priorizando la flexibilidad frente a los cambios, la mejora continua y la entrega constante de valor funcional [20].

A continuación, en la tabla 4.1, se exponen las diferencias fundamentales entre ambos enfoques para contextualizar la decisión metodológica adoptada en este proyecto:

4.2. METODOLOGÍA ÁGIL: ENFOQUE ITERATIVO E INCREMENTAL

Tras evaluar las características expuestas en la tabla anterior, para el desarrollo de este ecosistema tecnológico se ha optado por adoptar una metodología ágil fundamentada en un modelo de ciclo de vida iterativo e incremental. En lugar de intentar construir la totalidad del sistema de forma monolítica, el trabajo se fragmenta en iteraciones o ciclos temporalmente acotados [20].

Bajo este modelo, cada iteración funciona como un bloque autónomo que atraviesa sus propias fases de análisis, diseño, implementación y pruebas. Al finalizar un ciclo, se obtiene un incremento o prototipo que aporta valor real. Si este prototipo supera las pruebas de validación, sirve como base sólida para la siguiente iteración, integrándose con los desarrollos posteriores y añadiendo progresivamente nuevas características funcionales hasta alcanzar la versión final y completa del producto [20].

Metodologías tradicionales	Metodologías ágiles
Estructura secuencial y lineal. Cada fase debe finalizar por completo antes de iniciar la siguiente.	Estructura iterativa e incremental. Se avanza mediante ciclos cortos de desarrollo (iteraciones).
Baja tolerancia al cambio. Las modificaciones en etapas avanzadas suponen un alto coste temporal y estructural.	Alta flexibilidad. Los requisitos pueden evolucionar y adaptarse de forma dinámica a lo largo del proyecto.
Exige una documentación exhaustiva y un plan inamovible antes de iniciar la fase de programación.	Prioriza el software funcional sobre la documentación extensa, documentando lo estrictamente necesario y útil.
El producto se entrega al cliente de forma íntegra únicamente al finalizar todo el ciclo del proyecto.	Se realizan entregas parciales y funcionales al término de cada iteración, permitiendo pruebas tempranas.
La interacción con el cliente se limita, por lo general, a la toma de requisitos inicial y a la entrega final.	Fomenta una colaboración y validación constante con los clientes o los usuarios finales del sistema.

Tabla 4.1: Comparativa de características entre metodologías de desarrollo predictivas y ágiles.[20]

4.2.1. Justificación de la metodología adoptada

La adopción de este enfoque ágil no responde a una elección de conveniencia arbitraria, sino que está estrictamente alineada con la naturaleza de este Trabajo de Fin de Grado y las circunstancias particulares de su ejecución. La justificación de esta elección recae en dos pilares fundamentales:

- **Compatibilidad con el entorno académico:** el desarrollo de este proyecto se lleva a cabo de forma concurrente a las responsabilidades inherentes al último curso del Grado en Ingeniería Informática. Una metodología ágil permite modular la carga de trabajo de forma sumamente eficiente. El uso de iteraciones cortas facilita concentrar el mayor esfuerzo de programación en los periodos de menor carga lectiva, mientras que las semanas destinadas a exámenes pueden reservarse para tareas de menor exigencia técnica, como el diseño de prototipos de interfaz, documentación o revisión bibliográfica. Esta flexibilidad de planificación garantiza un avance ininterrumpido sin comprometer el rendimiento académico [20].
- **Naturaleza experimental y variabilidad de requisitos:** este proyecto se enmarca en un contexto clínico muy particular (una sala de hemodiálisis) y demanda la integración de múltiples ecosistemas tecnológicos complejos: visores autónomos Meta Quest 3, el motor de físicas Unity y un entorno *Full-Stack* de gestión compuesto por un *Backend* en Spring Boot y un *Frontend* analítico en Angular. A diferencia de un software de gestión estándar, el diseño de "juegos serios" para pacientes en camilla conlleva una alta incertidumbre funcional. Es altamente probable que parámetros como la sensibilidad de las colisiones, la accesibilidad de las mecánicas o el formato de las métricas biomecánicas deban ser reajustados continuamente tras las pruebas iniciales. El desarrollo incremental asegura que estas mecánicas experimentales se prueben en

fases tempranas, asumiendo los cambios arquitectónicos no como una desviación del plan original, sino como una evolución necesaria para garantizar la utilidad médica del sistema final [20].

4.2.2. Gestión y codificación de tareas

Para organizar el trabajo dentro de cada ciclo de desarrollo, los requisitos del sistema se han desglosado en tareas técnicas de menor granularidad. Dado que se ha optado por una metodología ágil adaptada a un único desarrollador, se ha prescindido de ceremonias formales propias de marcos de trabajo complejos como *Scrum* (tales como reuniones diarias o retrospectivas formales). La carga de trabajo se ha gestionado de forma pragmática mediante listas de tareas priorizadas asignadas a cada iteración.

Para asegurar la coherencia metodológica y mantener la trazabilidad entre el diseño del sistema y la implementación técnica documentada en capítulos posteriores, cada tarea ha sido clasificada mediante un código identificativo único. Este prefijo alfanumérico agrupa las tareas según su dominio tecnológico o clínico dentro del proyecto:

- **ENV / ARCH:** configuración del entorno de desarrollo y arquitectura base.
- **BACK / FRONT:** desarrollo de la capa lógica del servidor (Spring Boot) y de la interfaz web clínica (Angular).
- **SEC / LOPDGDD:** tareas relativas a la ciberseguridad, criptografía y cumplimiento normativo de privacidad.
- **VR / GAME:** integración de *hardware* inmersivo, programación de físicas y desarrollo de las mecánicas de los juegos serios.
- **UX / AUDIO:** diseño de interfaz de usuario, ergonomía en Realidad Virtual y diseño sonoro.
- **DATA / CLIN:** módulos de persistencia, telemetría masiva y algoritmos de validación médica (fatiga, tiempos de reacción, ROM).
- **FIX / TOOL:** corrección de errores lógicos (*bugs*), refactorización y creación de herramientas de autoría internas.

4.3. PLANIFICACIÓN Y CRONOGRAMA DE DESARROLLO

Para ejecutar el ciclo de vida iterativo e incremental establecido en la metodología, el desarrollo del ecosistema tecnológico se dividió en una serie de iteraciones. La planificación original se estructuró a partir de los requisitos extraídos en las reuniones iniciales con el equipo de tutorización y los responsables del Hospital General Universitario de Ciudad Real. En estas reuniones se definieron las limitaciones clínicas fundamentales: los pacientes reciben la hemodiálisis en camillas con una inclinación de entre 45 y 90 grados, a menudo con un brazo inmovilizado por la vía venosa, y el tiempo máximo de uso inmersivo no debe superar los 45 minutos para evitar cinetosis.

Con estos requerimientos como base, el trabajo se organizó en las siguientes iteraciones funcionales:

Iteración 0: toma de contacto y definición de requisitos. Fase de investigación preliminar y estado del arte. Se realizó la comparativa técnica de tecnologías (Unity vs. Unreal, Spring Boot, Angular) y se definió la arquitectura clínica del proyecto, optando por un modelo de tres capas y un patrón *Offline-First* para garantizar la seguridad en el entorno hospitalario.

Iteración 1: configuración del entorno, arquitectura base y seguridad (semanas 1 a 2). Preparación de la estación de trabajo local y los repositorios de control de versiones. Se inicializó el servidor Spring Boot, modelando las entidades clínicas base y estableciendo los cimientos de la seguridad mediante el algoritmo criptográfico BCrypt y el aislamiento de credenciales por variables de entorno.

Iteración 2: API REST, controladores y LOPDGDD (semanas 3 a 4). Implementación de la seguridad perimetral de la API mediante el estándar JSON Web Tokens (JWT) y control de accesos por roles (RBAC). Se desarrollaron los controladores estandarizados y el algoritmo de generación de identificadores alfanuméricos no ambiguos para asegurar el cumplimiento del RGPD.

Iteración 3: prototipado del *Arkanoid*, ergonomía y telemetría (semanas 4 a 6). Inicio del desarrollo en Realidad Virtual. Se programaron las mecánicas base de *Arkanoid*, la proyección en sala secreta y la adaptación asimétrica del Rango de Movimiento (ROM). Se instauró el *Monitor Clínico* para la captura de fatiga (*jitter*), tiempo de reacción y el centrado de vista en camilla.

Iteración 4: retroalimentación sensorial y dificultad (semanas 6 a 8). Integración de estelas de movimiento (*Trail Renderer*), efectos de sonido independientes y un sistema dinámico de dificultad que altera las variables físicas del juego. El esfuerzo arquitectónico se centró en empaquetar todo el código transversal en un "núcleo común" reutilizable.

Iteración 5: cierre del núcleo y calibración asimétrica (semanas 8 a 9). Rediseño de la persistencia local, creando carpetas estructuradas por paciente y volcados de biometría en crudo a 10 Hz con detección de tirones. Se implementó una pantalla curva envolvente de 120 grados, el patrón arquitectónico *Audio Bridge* y la calibración independiente de lateralidad.

Iteración 6: refinamiento de UX y *Arkanoid* versión final (semanas 10 a 11). Fase de aseguramiento de calidad (QA). Se programó un sistema de notificaciones flotantes adaptativas (*Lazy HUD*), textos dinámicos de puntuación y se solucionaron conflictos de renderizado (*Z-Ordering*) y *clipping*, culminando con la versión candidata de lanzamiento del módulo de brazos.

Iteración 7: desarrollo del tren inferior: juego de ritmo (semanas 11 a 13). Despliegue de un nuevo proyecto Unity heredando el núcleo común. Se desarrolló un algoritmo basado en el producto escalar para rastrear la pierna sin sufrir *Gimbal Lock*, se construyó un editor de niveles nativo con cuantización y se integraron composiciones musicales generadas por Inteligencia Artificial.

Iteración 8: ecosistema web e ingesta de datos (semanas 13 a 15). Construcción del *Frontend* web para el personal sanitario. Desarrollo de una Single Page Application (SPA) para gestionar las identidades, ingerir los historiales clínicos mediante *Drag & Drop* y visualizar la progresión a través de paneles (*Dashboards*) apoyados en Chart.js.

Iteración 9: experimentación, pruebas y validación del sistema (semanas 15 a 17). Fase dedicada a la evaluación del ecosistema en un entorno real. Se ejecutaron pruebas de viabilidad en las instalaciones del Hospital General Universitario de Ciudad Real (HGUCR), certificando la ergonomía del proyecto con el equipo médico y liberando los repositorios bajo licencia de código abierto.

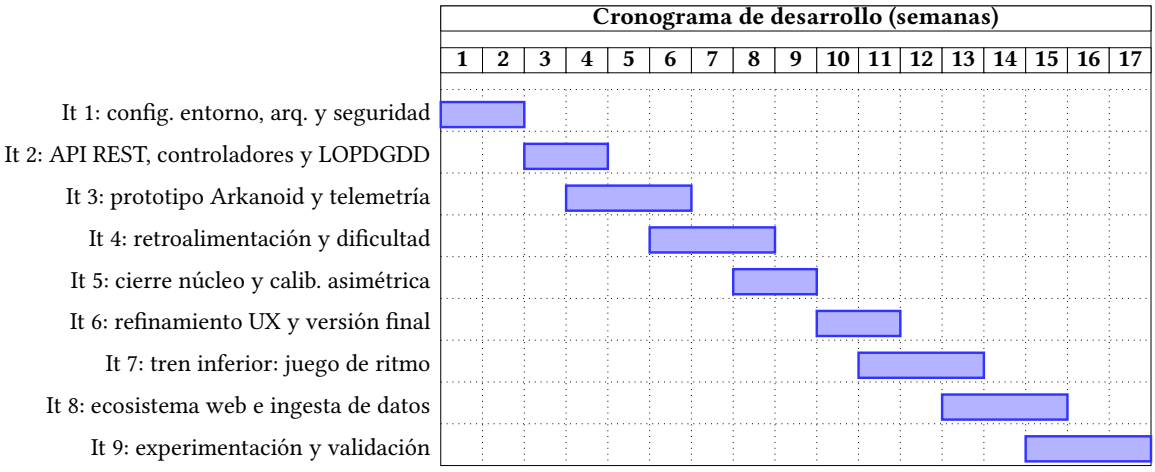


Figura 4.1: Diagrama de Gantt mostrando las iteraciones y sus duraciones a lo largo del proyecto.

4.4. RECURSOS HARDWARE Y SOFTWARE

Para la correcta ejecución de las iteraciones definidas en el cronograma, ha sido necesaria la utilización de diversas herramientas físicas y lógicas. A continuación, se detallan los recursos empleados y la justificación de su uso dentro del ecosistema del proyecto.

4.4.1. Recursos hardware

- **Visor de realidad virtual (Meta Quest 3):** dispositivo principal del proyecto. Su elección se justifica por su naturaleza *standalone* (autónoma), su sistema de posicionamiento *inside-out* y su capacidad nativa para el seguimiento preciso de manos (*hand tracking*), elementos indispensables para el uso clínico en la sala de hemodiálisis sin requerir cableado ni balizas externas.
- **Estación de trabajo (PC de desarrollo):** equipo informático empleado para la programación de los *scripts*, el renderizado de los entornos tridimensionales y el despliegue local de las pruebas del servidor. Las pruebas de RV se han agilizado utilizando el protocolo *Meta Quest Link*, permitiendo visualizar los cambios en el motor gráfico sin necesidad de compilar la aplicación en cada iteración.

4.4.2. Recursos software

- **Unity:** motor gráfico seleccionado para el desarrollo de los módulos inmersivos y juegos serios. Proporciona el entorno de físicas tridimensionales y se integra directamente con el *Meta XR Core SDK* para interpretar los datos biométricos del visor.
- **Spring Boot y Angular:** *frameworks* empleados para el desarrollo de la arquitectura web. Spring Boot (Java) se ha utilizado para levantar una API REST robusta en el *backend*, mientras que Angular (TypeScript) ha permitido construir la interfaz de usuario de la página única (SPA) en el *frontend*.
- **Entornos de Desarrollo Integrado (IDE):** se ha utilizado *Visual Studio Code* para la programación de los *scripts* en C# (Unity) y TypeScript (Angular), y *IntelliJ IDEA* para el desarrollo del *backend* en Java.
- **Git y GitHub:** sistema de control de versiones distribuido empleado para salvaguardar el progreso del código, gestionar las distintas ramas de las iteraciones y mantener copias de seguridad en la nube a lo largo de todas las iteraciones.

- **Suno AI:** herramienta de Inteligencia Artificial empleada para la generación procedimental de las pistas de audio del juego de ritmo, garantizando la ausencia de conflictos por derechos de autor (*copyright*).

4.5. ESTUDIO DE COSTES

Todo proyecto de ingeniería requiere de una evaluación económica que estime el valor real de su desarrollo, independientemente de si su fin es académico o comercial. Para este estudio de costes se han contemplado tres partidas presupuestarias fundamentales: costes de personal, costes de hardware y costes de software.

1. **Costes de personal:** representa la partida más significativa. Se ha estimado una dedicación aproximada de 300 horas de trabajo, correspondientes a los 12 créditos ECTS del Trabajo de Fin de Grado (a razón de 25 horas por crédito). Asumiendo el rol de un Ingeniero de Software Junior, se ha estipulado un coste por hora de 15 €[1].
2. **Costes de hardware:** incluye el precio de adquisición del visor Meta Quest 3 de 128 GB[2]. Asimismo, se ha presupuestado el coste íntegro de adquisición de una estación de trabajo (PC de desarrollo) apta para el renderizado de Realidad Virtual, la cual requiere de especificaciones técnicas de gama media-alta (procesador multinúcleo, RAM de alta capacidad y tarjeta gráfica dedicada)[3].
3. **Costes de software:** esta partida tiene un coste nulo (0 €). Todas las herramientas y servicios empleados (Unity Personal, Spring Boot, Angular, VS Code, Git) disponen de licencias *Open Source* o versiones educativas que cubren íntegramente las necesidades del proyecto. Del mismo modo, la generación procedimental de audio mediante Inteligencia Artificial se ha resuelto satisfactoriamente utilizando la capa gratuita de la plataforma Suno AI.

En la Tabla 4.2 se detalla el presupuesto total estimado para la ejecución del proyecto:

[1] Estimación basada en el salario medio interprofesional de un Ingeniero de Software *Junior* en España, consultado a través de portales de análisis salarial como Glassdoor.

[2] Precio oficial del visor Meta Quest 3 (128 GB) consultado en la tienda corporativa de Meta: https://www.meta.com/es/quest/quest-3/

[3] Precio medio estimado para un equipo informático de escritorio orientado a desarrollo 3D y RV, consultado en proveedores tecnológicos nacionales.

Concepto	Cantidad	Coste Unitario	Coste Total
Costes de personal			
Ingeniero de Software Junior	300 horas	15,00 €/h	4.500,00 €
Costes de hardware			
Visor Meta Quest 3 (128 GB)	1 unidad	549,00 €	549,00 €
Estación de trabajo (PC de desarrollo)	1 unidad	1.200,00 €	1.200,00 €
Costes de software y servicios			
Licencias de desarrollo y herramientas	N/A	0,00 €	0,00 €
		Subtotal	**6.249,00 €**
		IVA (21 %)	1.312,29 €
		TOTAL PRESUPUESTO	**7.561,29 €**

Tabla 4.2: Presupuesto estimado del proyecto.

CAPÍTULO 5
Desarrollo y resultados

Este capítulo detalla la fase de implementación tecnológica del proyecto, describiendo cómo se han materializado la arquitectura y tecnologías definidas en el marco metodológico. Se documentará el proceso completo de ingeniería: desde la extracción de requisitos y el modelado del sistema, hasta el diseño estructural de la base de datos, el desarrollo de las aplicaciones inmersivas y la construcción de la plataforma web de gestión clínica. Finalmente, se expondrán los resultados obtenidos tras las pruebas de validación.

5.1. REQUISITOS Y ANÁLISIS DEL SISTEMA

El desarrollo de un ecosistema de *software* orientado al ámbito hospitalario exige un proceso de ingeniería de requisitos exhaustivo. La convivencia de una plataforma web de gestión clínica y un entorno inmersivo de Realidad Virtual (RV) obliga a definir con precisión las fronteras operativas y los flujos de datos. A continuación, se detallan los perfiles de usuario identificados y el catálogo extendido de requisitos funcionales y no funcionales que rigen el comportamiento del sistema.

5.1.1. Identificación de actores

Para modelar las interacciones con el sistema, se han identificado tres perfiles de actores, cada uno de ellos con un grado de responsabilidad y un contexto de uso diferenciado:

- **Paciente:** actor central de la fase de experimentación y usuario final de los módulos inmersivos. Su interacción se produce exclusivamente a través del visor de RV mientras recibe el tratamiento durante sesiones de hemodiálisis (en sillón o camilla). El sistema debe garantizar que este actor no requiera conocimientos tecnológicos previos para navegar por las experiencias propuestas.
- **Personal sanitario:** actor que opera con la plataforma web (*frontend*) desde el panel de gestión clínica o mediante dispositivos móviles del hospital. Sus responsabilidades abarcan la gestión del ciclo de vida de la sesión terapéutica: da de alta a los pacientes bajo estrictos protocolos de seudonimización, configura los parámetros previos al juego y examina las gráficas de evolución clínica (telemetría) una vez finalizadas las sesiones.
- **Administrador:** actor con privilegios de mantenimiento técnico e infraestructura. Gestiona el acceso al *backend*, administra las cuentas del personal sanitario, registra nuevos visores adquiridos por el centro hospitalario y ejerce las funciones de control de datos, pudiendo ejecutar el borrado definitivo de registros para el cumplimiento legal.

5.1.2. Requisitos funcionales del panel de gestión clínica

Este bloque agrupa las funcionalidades exigidas para la plataforma web, encargada de la administración del sistema y el análisis de datos (interfaz entre el personal sanitario y la base de datos).

RF-01. Autenticación y control de acceso basado en roles: el sistema deberá proporcionar una pasarela de entrada segura que requiera usuario y contraseña. Dependiendo de si las credenciales pertenecen a un administrador o a un enfermero, el sistema web alterará dinámicamente la interfaz de usuario, restringiendo el acceso a los paneles de borrado de datos y gestión del personal para los roles no autorizados.

RF-02. Registro y seudonimización de pacientes: en estricto cumplimiento con el Reglamento General de Protección de Datos (RGPD), el sistema no dispondrá de campos para introducir el nombre, apellidos o número de la seguridad social de los pacientes. Al solicitar una nueva alta, el sistema generará de forma automatizada un identificador (ID) aleatorio de seis caracteres alfanuméricos.

RF-03. Exclusión de caracteres ambiguos: derivado del requisito anterior y para evitar errores humanos de lectura en la planta del hospital, el algoritmo de generación de identificadores excluirá explícitamente caracteres visualmente confusos (como la letra 'O' mayúscula frente al número '0', o la letra 'l' minúscula frente al número '1') [20].

RF-04. Visualización de telemetría y dashboard analítico: la plataforma proporcionará un panel de control interactivo donde el personal sanitario podrá visualizar el historial de los pacientes registrados. El sistema renderizará gráficas y tablas que expondrán métricas extraídas de las sesiones inmersivas: el índice de fatiga, los tiempos de reacción ante estímulos, el uso porcentual asimétrico de extremidades y la amplitud de los movimientos articulares.

RF-05. Gestión de sesiones y límites de exposición: desde el panel web, el sanitario podrá establecer las directrices operativas de las gafas de RV asociadas. Especialmente, el sistema permitirá limitar la duración de las sesiones (con una recomendación estricta de no superar los 45 minutos) para prevenir la cinetosis o fatiga visual extrema en los pacientes sometidos a diálisis.

5.1.3. Requisitos funcionales de la experiencia inmersiva (RV)

Este bloque detalla las funcionalidades de la aplicación desplegada en el visor Meta Quest 3, la cual es ejecutada directamente por el paciente.

RF-06. Calibración asimétrica del rango de movimiento: dado que los pacientes de hemodiálisis suelen presentar una vía venosa conectada a uno de sus brazos (o limitaciones temporales de movilidad), la aplicación exigirá un proceso de calibración inicial. El sistema medirá y guardará el alcance máximo de cada extremidad de forma totalmente independiente, alterando la posición de los objetivos virtuales para que el paciente solo se esfuerce dentro de su rango seguro y asimétrico.

RF-07. Centrado dinámico del entorno virtual: debido a que la hemodiálisis se administra en camillas articuladas, el paciente se encontrará en ángulos variables de entre 45° y 90°. El *software* deberá incluir una funcionalidad que permita recentrar todo el eje visual y la interfaz interactiva para adaptarla instantáneamente a la inclinación del torso y la línea de visión del usuario.

RF-08. Interacción inmersiva para el tren superior (*Arkanoid*): el sistema de juego de bloques para extremidades superiores se controlará mediante los mandos físicos de las gafas Meta Quest 3. El *software* deberá registrar con alta precisión el posicionamiento espacial (6DoF) de los controladores para trasladar de forma fluida el movimiento del paciente a las palas virtuales, garantizando un cálculo exacto de las colisiones y de las físicas de rebote dentro del entorno tridimensional.

RF-09. Proyección espacial para el tren inferior (ritmo): para el módulo de ejercitación de piernas, el visor rastreará el movimiento mediante controladores adheridos. Para resolver las limitaciones matemáticas del *hardware* (como el bloqueo de ejes o *gimbal lock*), el sistema aplicará cálculos de producto escalar sobre vectores proyectados, logrando una medición precisa de las posturas rítmicas del paciente en sincronía con el compás de la música.

RF-10. Persistencia autónoma de datos locales: para evitar la pérdida de información crítica ante interrupciones de la conectividad WiFi del hospital, la aplicación de RV actuará de forma autónoma. Todas las métricas de la partida y configuraciones de calibración se guardarán internamente en el almacenamiento del visor en formatos estructurados (CSV para métricas tabulares y JSON para jerarquías complejas), quedando a la espera de ser procesadas por el personal sanitario.

5.1.4. Requisitos no funcionales (RNF)

Los requisitos no funcionales definen los atributos de calidad, rendimiento y restricciones tecnológicas bajo las cuales debe operar la arquitectura del sistema propuesto.

RNF-01. Arquitectura desacoplada (*Stateless*): el diseño de comunicación entre las gafas de RV, el cliente web y el servidor debe ser inherentemente sin estado. No se mantendrán canales bidireccionales en vivo (como WebSockets) entre las gafas y el *backend*, garantizando la escalabilidad de la API REST y evitando fallos críticos si la red clínica sufre latencias.

RNF-02. Criptografía y protección de credenciales: todas las contraseñas pertenecientes al personal médico almacenadas en la base de datos se cifrarán de manera unidireccional utilizando el algoritmo de *hashing* BCrypt. Asimismo, la autorización de llamadas a la API se validará mediante el uso de *JSON Web Tokens* (JWT) con un tiempo de expiración riguroso.

RNF-03. Rendimiento gráfico y mitigación de cinetosis: el renderizado del entorno virtual en el motor Unity deberá sostener una tasa de refresco mínima y constante (entre 72 y 90 fotogramas por segundo) y asegurar una latencia *motion-to-photon* marginal. Para maximizar el confort, se exigirá la inclusión de ayudas visuales como estelas de movimiento (*trail renderers*) y un diseño de iluminación sin parpadeos excesivos.

RNF-04. Independencia de derechos de autor (*Copyright*): con el fin de evitar conflictos legales derivados de la distribución o demostración del *software*, todos los recursos artísticos y auditivos (especialmente las pistas musicales empleadas en el juego de ritmo) se generarán mediante sistemas de inteligencia artificial (p. ej., Suno AI) o se extraerán de repositorios bajo licencias de dominio público (CC0).

5.1.5. Modelo de casos de uso

El modelo de casos de uso representa de manera formal y estructurada las interacciones entre los actores descritos anteriormente y los límites del ecosistema tecnológico. Dado

el diseño arquitectónico desacoplado, impuesto por las restricciones clínicas y de red, los casos de uso se dividen funcionalmente en dos subsistemas principales: el entorno inmersivo operado desde el visor de RV y la plataforma web de gestión clínica.

En la Figura 5.1 se expone el diagrama general del sistema, evidenciando la separación de responsabilidades y la ausencia de cruce de flujos de trabajo entre el paciente y el personal del hospital.

5.1.5.1. Casos de uso del entorno inmersivo (paciente)

Las interacciones en este bloque se ejecutan de manera autónoma en el visor Meta Quest 3, almacenando los resultados de forma local.

- **CU-01. Calibrar rango de movimiento:** antes de iniciar cualquier actividad, el paciente interactúa con el sistema para registrar su alcance máximo asimétrico, permitiendo que la aplicación adapte la posición espacial de los elementos virtuales.
- **CU-02. Realizar sesión de tren superior (*Arkanoid*):** el paciente ejecuta movimientos de extensión y flexión de los brazos para interceptar objetos tridimensionales. El sistema procesa el *hand tracking* y calcula las colisiones en tiempo real.
- **CU-03. Realizar sesión de tren inferior (ritmo):** el paciente interactúa moviendo sus piernas en sincronía con estímulos audiovisuales, mientras el sistema evalúa la proyección de sus vectores espaciales.
- **CU-04. Recentrar entorno virtual:** el paciente o el personal de apoyo acciona el comando para pivotar todo el entorno 3D, ajustándolo instantáneamente a su grado de inclinación en la camilla de hemodiálisis.

5.1.5.2. Casos de uso de la plataforma web clínica (sanitario y administrador)

Este bloque abarca la gestión y visualización de la telemetría, interactuando directamente con la API REST y la base de datos centralizada.

- **CU-05. Autenticarse en el sistema:** el personal sanitario o el administrador introducen sus credenciales para obtener un *token* de acceso (JWT), el cual determina las vistas y menús que se cargarán en la interfaz.
- **CU-06. Registrar paciente (ID a modo de seudónimo):** el enfermero inicia el flujo de alta de un nuevo usuario. El sistema genera y devuelve un código seguro de seis caracteres sin letras ambiguas, evitando la recogida de cualquier dato personal vinculante.
- **CU-07. Consultar historial y telemetría:** el enfermero accede al panel de control (*dashboard*) de un paciente utilizando su identificador a modo de seudónimo, visualizando las gráficas de rendimiento, lateralidad, tiempo de reacción e índice de fatiga recopilados en las sesiones anteriores.
- **CU-08. Gestionar personal sanitario:** el administrador utiliza su panel exclusivo para dar de alta a nuevos enfermeros, modificar sus permisos o revocar su acceso al sistema.
- **CU-09. Borrar registros clínicos (LOPDGDD):** el administrador ejerce la función de control de datos, eliminando de forma definitiva e irreversible de la base de datos el identificador de un paciente y toda su telemetría asociada.

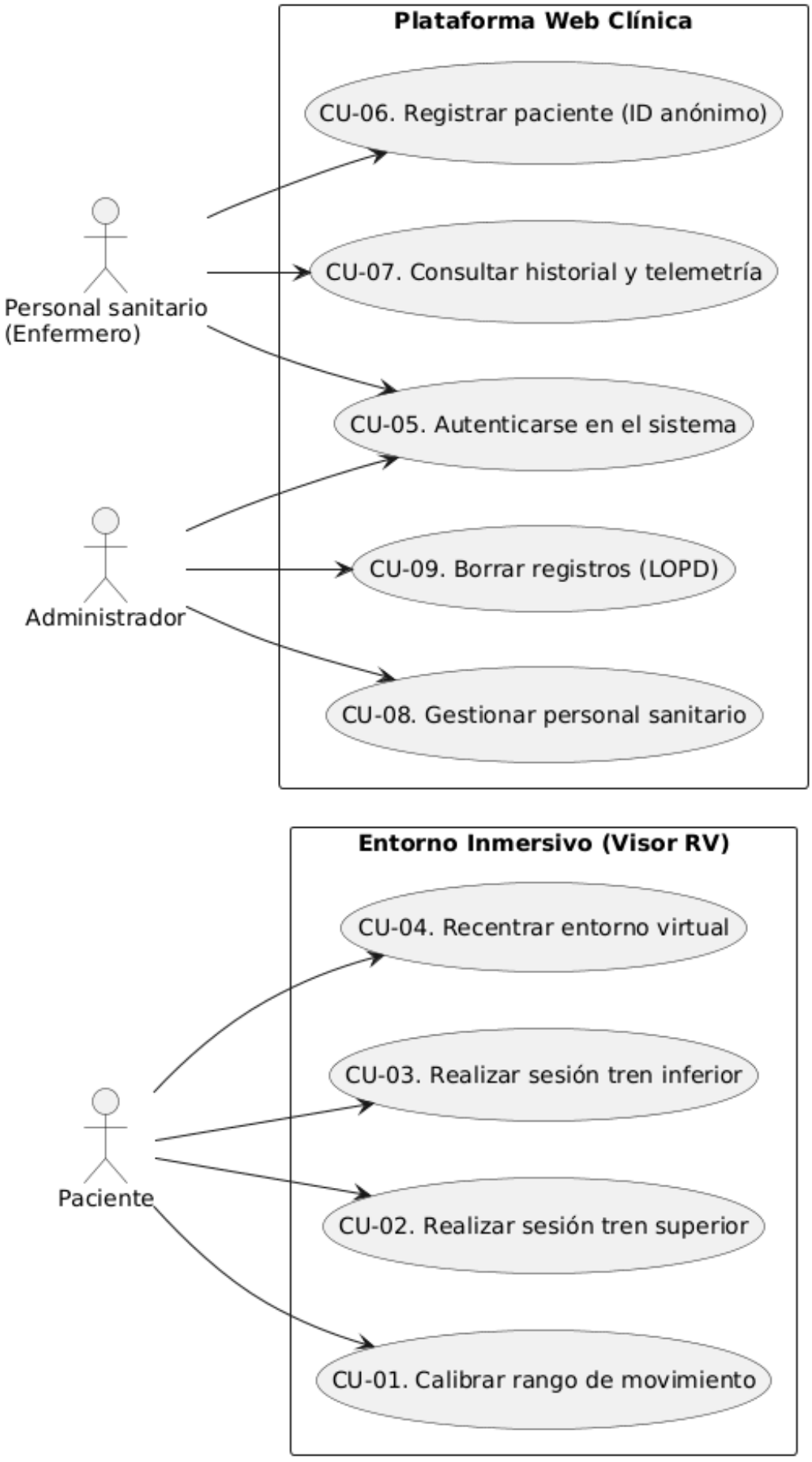


Figura 5.1: Diagrama general de casos de uso del ecosistema, separado por entorno inmersivo y plataforma web.

5.2. ARQUITECTURA DEL SISTEMA

Para modelar y documentar la arquitectura de este proyecto, se ha adoptado el modelo C4 [3]. Este enfoque estandarizado permite visualizar la estructura del *software* en distintos niveles de abstracción, facilitando la comprensión técnica del ecosistema por parte de todos los actores involucrados. Dada la naturaleza de la solución desarrollada, el análisis arquitectónico se centrará en el nivel 2 del modelo (diagrama de contenedores), el cual desglosa las aplicaciones y repositorios de datos que componen el sistema global.

En la Figura 5.2 se ilustra la topología del ecosistema, caracterizado por un desacoplamiento físico y lógico total entre el entorno inmersivo y el servidor. Esta decisión de diseño garantiza la operatividad ininterrumpida de las gafas de RV en el entorno hospitalario, prescindiendo por completo de la necesidad de conectividad a la red clínica.

5.2.1. Descripción de los contenedores

El ecosistema tecnológico se divide en cuatro contenedores principales, implementados con distintas tecnologías y con responsabilidades estrictamente delimitadas:

- **Clientes de RV:** desarrollados utilizando el motor gráfico Unity y el lenguaje C#. Este contenedor se despliega de forma nativa en el visor Meta Quest 3 (*standalone*). Su arquitectura interna opera completamente fuera de línea (*offline*): procesa la lógica de los juegos serios y recopila la telemetría biomecánica del paciente. Todos los datos generados se persisten exclusivamente en el almacenamiento local del visor (en formatos estructurados JSON y CSV), sin establecer comunicación alguna con servidores externos. Ambos clientes contienen un núcleo en común, mejorando su escalabilidad.
- **Panel de control clínico (cliente web *frontend*):** aplicación de página única (SPA) desarrollada con el *framework* Angular y TypeScript. Actúa como la interfaz visual para el personal de enfermería y los administradores. Además de renderizar gráficas estadísticas y gestionar rutas protegidas, este contenedor asume la responsabilidad de actuar como pasarela de subida: permite al personal sanitario adjuntar los archivos previamente extraídos del visor y enviarlos al servidor mediante peticiones HTTP.
- **Servidor central (API REST *backend*):** desarrollado en Java utilizando el *framework* Spring Boot. Constituye el núcleo lógico de la plataforma distribuida. Provee una API RESTful segura que actúa como intermediaria entre el cliente web y la base de datos. Sus responsabilidades incluyen la validación de credenciales (JWT), la seudonimización de pacientes (generación de IDs alfanuméricos) y el procesamiento de los archivos de telemetría subidos a través de la aplicación web.
- **Base de datos relacional:** implementada sobre MySQL. Este contenedor de persistencia es gestionado internamente por el servidor Spring Boot a través de JPA/Hibernate. Almacena las credenciales cifradas del personal, la jerarquía de las partidas y las rutas estáticas hacia los archivos de telemetría alojados en el sistema de ficheros del servidor.

5.2.2. Flujo de sincronización y extracción de datos

El diseño de las comunicaciones entre los componentes elimina cualquier dependencia de red por parte del visor de RV. El flujo de vida de los datos sigue un proceso asíncrono y mediado por intervención humana, estructurado en las siguientes fases:

1. **Recolección local:** durante la sesión de hemodiálisis, la aplicación inmersiva registra las métricas (calculadas a partir de los datos cinemáticos) y las almacena internamente en la memoria de las Meta Quest 3, vinculadas al identificador del paciente.
2. **Extracción física:** una vez finalizadas las sesiones de la jornada, el personal sanitario o técnico conecta el visor a un equipo informático mediante una conexión USB, extrayendo manualmente los directorios generados (archivos CSV y JSON).
3. **Sincronización web:** a través del panel de control clínico (Angular), el enfermero autenticado selecciona los archivos extraídos y ejecuta la subida al sistema. El *frontend* empaqueta estos datos y realiza una petición HTTP POST (*multipart/form-data*) hacia el *backend* (Spring Boot).
4. **Procesamiento y persistencia:** la API REST recibe los archivos, los ubica de forma segura en el almacenamiento del servidor, guarda las referencias relacionales en la base de datos MySQL y devuelve una respuesta de éxito, permitiendo que el *dashboard* se actualice y renderice de inmediato las nuevas gráficas de evolución clínica.

5.3. MODELO DE DATOS, PERSISTENCIA Y TELEMETRÍA

Para garantizar la trazabilidad clínica y el análisis posterior de las sesiones terapéuticas, el ecosistema cuenta con una sólida capa de persistencia dividida en dos grandes bloques tecnológicos: una base de datos relacional para el almacenamiento de metadatos consolidados y un sistema de ficheros estructurados para la telemetría biomecánica en crudo. En la Figura 5.3 se ilustra el diagrama general que unifica ambas estrategias de almacenamiento.

El soporte de persistencia centralizado se apoya en una base de datos relacional gestionada a través de MySQL e implementada en el servidor Spring Boot mediante el mapeo objeto-relacional (ORM) proporcionado por Hibernate. En este esquema, se han definido entidades primarias para la gestión hospitalaria, tales como el *Enfermero*, que custodia las credenciales encriptadas y los privilegios de administración, y el *Paciente*, representado exclusivamente por un identificador alfanumérico inmutable para garantizar la seudonimización y el cumplimiento de la LOPDGDD.

Para el registro de los juegos serios, se ha diseñado una arquitectura basada en herencia utilizando la estrategia *JOINED* del estándar *Java Persistence API* (JPA). Una clase abstracta denominada *Partida* centraliza los metadatos comunes a cualquier sesión, como la marca de tiempo, la duración y el identificador del paciente asociado. De esta clase base derivan entidades específicas como *PartidaArkanoid* y *PartidaRitmo*, las cuales almacenan en tablas separadas las métricas de alto nivel extraídas tras cada sesión, incluyendo el índice de fatiga calculado, los porcentajes de precisión rítmica, la racha máxima alcanzada, el tiempo de reacción cognitivo-motor y la lateralidad de los impactos.

Adicionalmente, el esquema relacional cuenta con la entidad *RegistroError*, diseñada para persistir cualquier anomalía técnica, fallo de red o excepción detallada capturada tanto en la lógica del servidor como en las propias gafas de RV.

Para evitar la saturación y degradación del rendimiento del servidor relacional, la base de datos no almacena directamente las extensas series temporales de movimiento generadas por el visor. En su lugar, la entidad base de la partida guarda únicamente un puntero lógico en forma de ruta de acceso hacia los archivos físicos alojados en el almacenamiento del sistema. Esta información descentralizada se estructura en dos formatos de archivo independientes generados localmente por el visor Meta Quest 3 durante su uso *offline*. Por un lado, se generan archivos de configuración en formato JSON encargados de persistir las preferencias

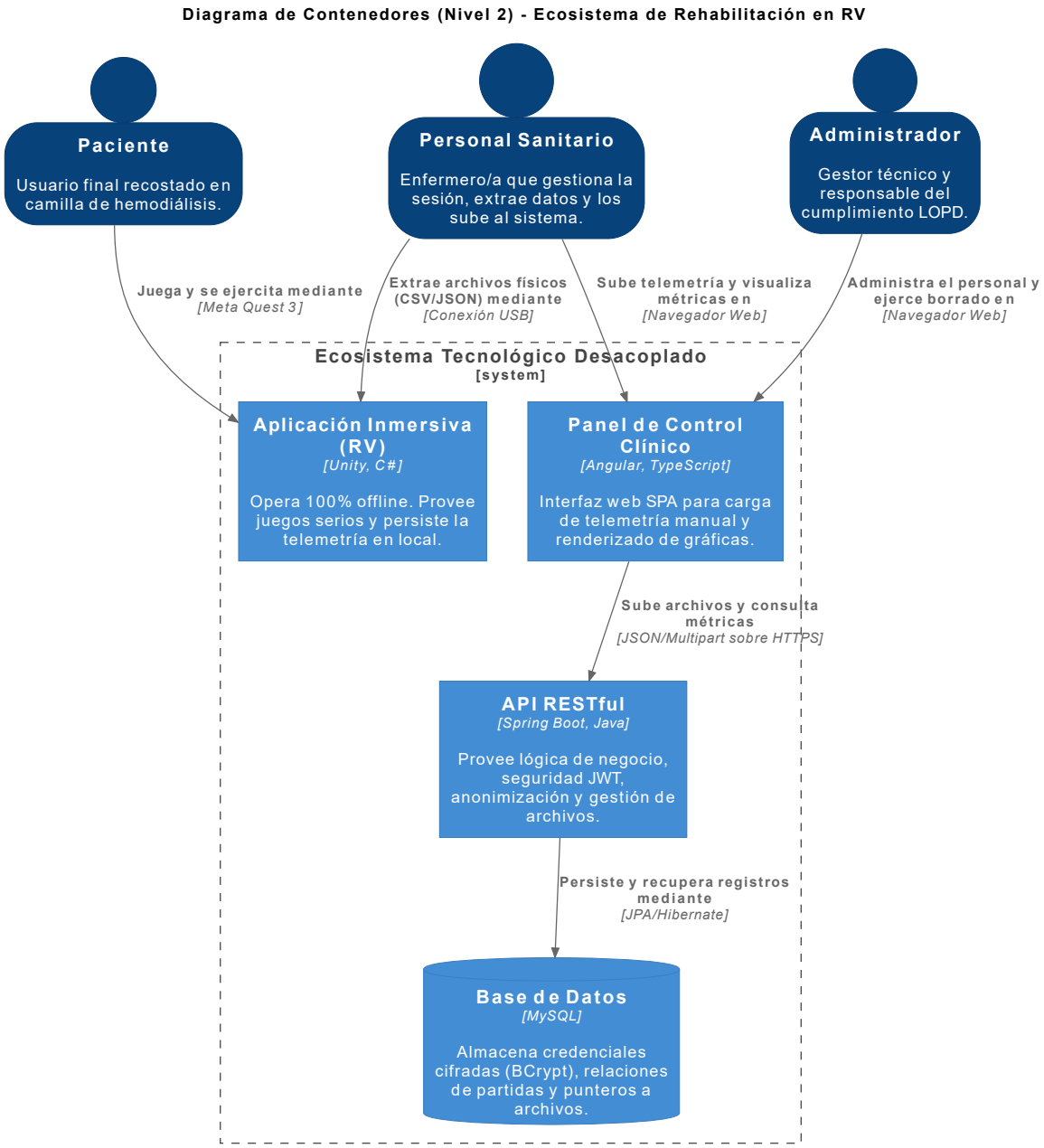
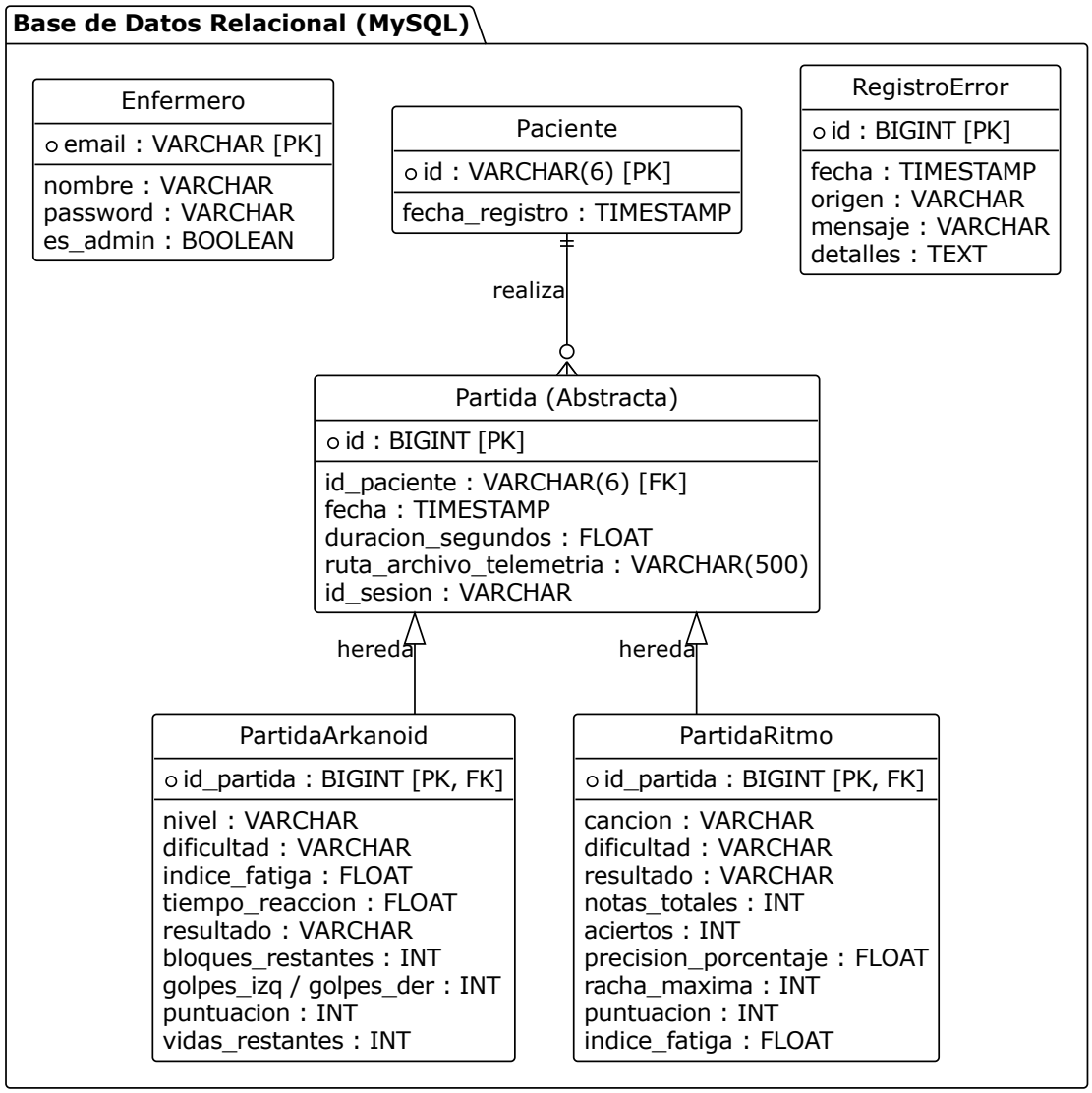
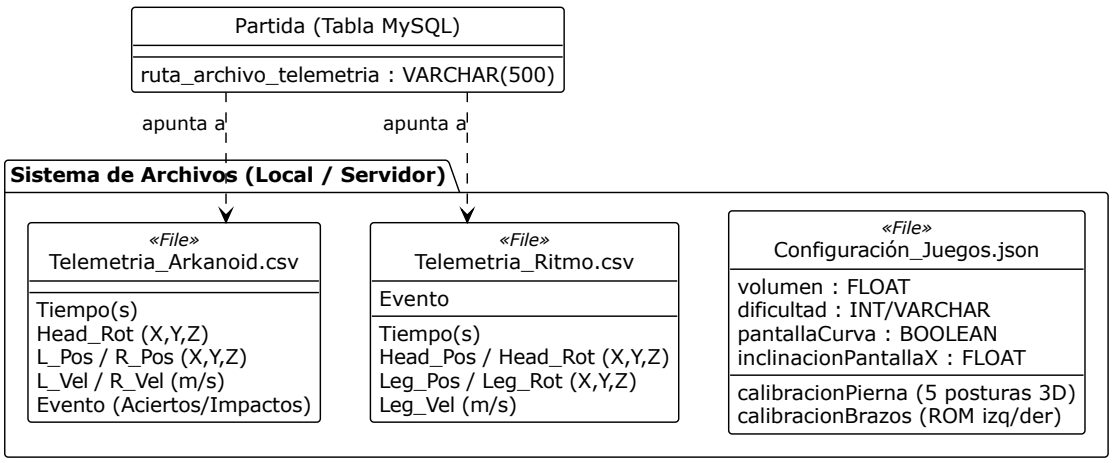


Figura 5.2: Diagrama de contenedores (modelo C4) ilustrando el desacoplamiento de la aplicación de RV y el flujo de subida mediante el cliente web.



(a) Modelo de datos relacional diseñado para la plataforma clínica.



(b) Estructura del sistema de archivos vinculada desde la base de datos.

Figura 5.3: Diagrama integrado del modelo de datos relacional y el almacenamiento estructurado en ficheros de telemetría y configuración.

ergonómicas del paciente en el entorno tridimensional, tales como la escala del menú, la inclinación dinámica de la pantalla para ajustarse a la camilla, la activación de paneles curvos y los vectores tridimensionales exactos resultantes de la calibración clínica. En el caso del tren superior, este archivo guarda el rango de alcance asimétrico paramétrico de los brazos, mientras que para el tren inferior consolida las matrices de rotación y traslación correspondientes a las cinco posturas anatómicas de la pierna.

Por otro lado, la telemetría pura del movimiento articular se exporta en archivos de valores separados por comas (CSV) capturados a alta frecuencia. Estos registros documentan periódicamente la evolución temporal de la sesión, almacenando fotograma a fotograma la marca de tiempo, la posición y rotación espacial de la cabeza del usuario, las coordenadas vectoriales de los controladores físicos acoplados a las extremidades, su velocidad lineal expresada en metros por segundo y una columna dedicada exclusivamente al registro de eventos discretos, como colisiones con bloques o aciertos de notas musicales. Esta separación estratégica entre metadatos relacionales ligeros y archivos biométricos pesados garantiza tiempos de respuesta óptimos en las consultas web del personal de enfermería, al tiempo que proporciona a los especialistas médicos un repositorio exhaustivo de datos en crudo para futuras investigaciones biomecánicas.

5.4. IMPLEMENTACIÓN DEL SISTEMA

Tras haber definido los requisitos funcionales y haber modelado la arquitectura teórica del sistema mediante el enfoque C4, en esta sección se detalla el proceso real de construcción del *software*. La implementación se ha llevado a cabo siguiendo el ciclo de vida iterativo e incremental descrito en la metodología, organizando la carga de trabajo en iteraciones funcionales.

A nivel arquitectónico general, el desarrollo se abordó desde un enfoque de sistema distribuido, requiriendo la inicialización de tres entornos de trabajo independientes pero interconectados: un motor gráfico para el entorno de RV, un entorno de ejecución en Java para el servidor central y un *framework* basado en TypeScript para la interfaz web. A continuación, se desgrana la evolución cronológica y técnica del ecosistema a través de las distintas fases de desarrollo.

5.5. ITERACIÓN 0: TOMA DE CONTACTO Y DEFINICIÓN DE REQUISITOS

El desarrollo práctico de este Trabajo de Fin de Grado comenzó formalmente con una fase preliminar, designada como iteración 0, llevada a cabo durante el mes de septiembre. Esta iteración inicial resultó crítica para la viabilidad del proyecto, ya que el punto de partida técnico requería una inmersión absoluta desde cero en el desarrollo con el motor gráfico Unity y la programación de experiencias inmersivas. Para suplir esta carencia inicial, las primeras semanas se dedicaron a reuniones de tutorización y al estudio de las herramientas fundamentales de la plataforma.

El hito más relevante de este periodo inicial fue la reunión conjunta mantenida entre el equipo de tutorización de la universidad, el propio alumno y el responsable del Hospital General Universitario de Ciudad Real. El objetivo de este encuentro fue fijar la arquitectura del sistema a la realidad física de una sala de hemodiálisis. Durante la sesión, se establecieron las severas restricciones ergonómicas que condicionarían todo el desarrollo posterior: los pacientes reciben el tratamiento recostados en camillas con una inclinación variable de entre 45 y 90 grados, con las piernas completamente rectas.

Además, se determinó que la exposición a la RV no debía superar los 45 minutos continuados para prevenir la aparición de mareos. A nivel motriz, se identificó una limitación crítica derivada del propio tratamiento, ya que la vía venosa canalizada en el brazo del paciente restringe severamente la movilidad de una de sus extremidades superiores.

Con estas premisas clínicas sobre la mesa, se esbozó la arquitectura general y el alcance del ecosistema de *software*. Se acordó dividir la solución inmersiva en tres grandes bloques interconectados. El primero consistiría en una experiencia audiovisual introductoria, orientada a explicar el uso del visor y a relajar al paciente para mitigar el dolor percibido durante la punción inicial. El segundo y tercer bloque conformarían el núcleo de la carga de trabajo, centrándose en el desarrollo de "juegos serios" terapéuticos.

Para el tren superior, se debatió la creación de interacciones de baja carga cognitiva que pudieran ser manejadas con un solo brazo, surgiendo ideas iniciales como un simulador de ping-pong o un *Whack-A-Mole* adaptado a los reflejos de pacientes de edad avanzada. Para el tren inferior, la arquitectura planteada requería una innovación de *hardware*, decidiéndose que los controladores físicos del visor Meta Quest debían acoplarse directamente a las piernas del paciente mediante cintas o velcros. Esta decisión buscaba facilitar mecánicas basadas en la elevación y rotación de las extremidades a través de un juego de ritmo musical.

Finalmente, para dotar de valor clínico a estas interacciones lúdicas, el diseño arquitectónico se cerró con la propuesta de una plataforma web externa. Esta aplicación independiente permitiría al personal de enfermería registrar el uso de los visores y visualizar de forma gráfica e intuitiva las estadísticas y telemetrías generadas por los pacientes durante sus sesiones de juego.

5.6. ITERACIÓN 1: CONFIGURACIÓN DEL ENTORNO, ARQUITECTURA BASE Y SEGURIDAD

Objetivo global de la iteración. El propósito fundamental de esta primera iteración de desarrollo fue establecer y configurar desde cero toda la estación de trabajo local, abarcando la instalación de las herramientas tecnológicas necesarias para el ciclo de vida del proyecto. Asimismo, se planteó como meta principal la construcción de los cimientos lógicos del servidor central, diseñando la arquitectura inicial del *backend*, conectándolo a la base de datos y estableciendo los primeros protocolos de seguridad criptográfica.

5.6.1. Planificación de tareas

Para guiar esta fase inicial de cimentación técnica, se definieron las tareas estructurales y de configuración requeridas. En la tabla 5.1 se expone la planificación para la iteración, detallando la prioridad de cada instalación y el estado de los primeros desarrollos de la API.

5.6.2. Preparación del entorno y control de versiones

La fase inicial requirió una configuración minuciosa desde cero de la estación de trabajo, estableciendo un ecosistema de desarrollo a medida. Para la gestión de la persistencia de datos, se llevó a cabo la instalación del motor relacional MySQL Server, acompañado de su interfaz gráfica MySQL Workbench; una decisión técnica orientada a facilitar la administración visual y el diseño estructurado de las tablas durante la fase de prototipado temprano.

De forma paralela, se configuró Visual Studio Code (VSCode) como el Entorno de Desarrollo Integrado (IDE) principal del proyecto. A este editor se le incorporaron las extensiones

ID	Descripción de la tarea	Prioridad	Estado
ENV-01	Configuración de la base de datos local (MySQL Server y Workbench).	Alta	Completada
ENV-02	Preparación del IDE (VSCode) y cliente de pruebas HTTP (Postman).	Alta	Completada
ENV-03	Instalación de Unity 6.3 LTS y configuración de Meta Developer.	Media	Completada
BACK-01	Inicialización del control de versiones (Git/GitHub) y Spring Boot.	Alta	Completada
BACK-02	Modelado de entidades clínicas base y repositorios JPA.	Alta	Completada
SEC-01	Integración de encriptación BCrypt y aislamiento por variables de entorno.	Alta	Completada

Tabla 5.1: Tareas planificadas y completadas durante la configuración del entorno de desarrollo y la inicialización del servidor.

necesarias para el desarrollo con Java y el *framework* Spring Boot, dejando a su vez el terreno preparado para la futura implementación del *frontend* en Angular. Con el objetivo de poder probar y validar de manera ágil e independiente las peticiones a la API que se iban a desarrollar, se procedió a la instalación del cliente HTTP Postman.

En el ámbito de la RV, aunque el desarrollo intensivo de los "juegos serios" se pospuso para iteraciones posteriores, se consideró vital realizar una primera toma de contacto técnica. Por ello, se instaló la plataforma de gestión Unity Hub y se descargó de manera específica la versión con soporte a largo plazo Unity 6.3 LTS (6000.3.6f1), lo que permitió comenzar a explorar la interfaz del motor gráfico mediante pequeños proyectos de prueba.

Además, se completó la configuración requerida para obtener una cuenta de *Meta Developer*, un requisito tecnológico indispensable para habilitar el modo de desarrollador en el visor Meta Quest 3. Este paso permitió autorizar la carga lateral (*sideloading*) de las aplicaciones no publicadas en la tienda oficial mediante conexión USB. Para asegurar la trazabilidad del código y disponer de copias de seguridad continuas, se inicializó el repositorio principal del servidor mediante el sistema de control de versiones Git, alojándolo en GitHub bajo la nomenclatura `backend-tfg-pedroantonio`.

5.6.3. Arquitectura base y persistencia de datos

Una vez configurada toda la infraestructura de *software*, el trabajo de programación comenzó con la inicialización del proyecto Spring Boot. En primer lugar, se programaron los modelos de datos base, creando las entidades fundacionales del ecosistema clínico. Se modeló la clase `Enfermero`, incluyendo campos clave como el correo electrónico, nombre, contraseña y un indicador booleano de privilegios de administrador. De igual modo, se prepararon los esqueletos para las entidades de pacientes y sesiones clínicas.

Para dotar a estas entidades de persistencia real en el disco, se implementaron sus respectivas interfaces `Repository`. Estas clases, apoyadas en el estándar JPA (*Java Persistence API*), son las encargadas de abstraer las consultas complejas y gestionar el acceso directo a la base de datos MySQL sin necesidad de escribir sentencias SQL nativas de forma manual.

5.6.4. Seguridad criptográfica y resolución de problemas

En el ámbito crítico de la ciberseguridad, se creó la clase de configuración `SecurityConfig`. Fiel a una filosofía de diseño centrada en la robustez local, se integró el algoritmo de *hashing* BCrypt para garantizar que cualquier contraseña registrada por el personal sanitario fuera encriptada unidireccionalmente antes de su almacenamiento, impidiendo su lectura incluso ante una brecha interna de la base de datos.

Seguidamente, se programó la lógica de negocio mediante la creación de la clase `EnfermeroService` y el controlador REST `UserController`, habilitando la recepción de peticiones HTTP de tipo POST en el *endpoint* `/users/register`. Por último, se configuró la clase `WebConfig` para gestionar las reglas de acceso cruzado (CORS) y los filtros de las peticiones entrantes.

El principal obstáculo técnico durante esta fase temprana residió en aislar las credenciales de conexión de la base de datos. Para evitar de forma estricta la filtración de claves confidenciales en el repositorio público de GitHub, se recurrió al uso de variables de entorno locales. La correcta parametrización de los archivos `application.properties` para que el servidor Spring Boot pudiera leer dinámicamente estas credenciales ocultas requirió diversas pruebas de depuración en el sistema operativo.

5.6.5. Resultados finales

Al concluir esta primera iteración, se logró consolidar un entorno de desarrollo completamente funcional, intercomunicado y versionado. A nivel de *software*, la iteración culminó con un servidor *backend* desplegado de forma local y capaz de recibir peticiones seguras mediante Postman. El sistema demostró estar capacitado para procesar correctamente el registro de un nuevo enfermero, encriptar su contraseña mediante BCrypt y persistir de forma exitosa la información en MySQL, sentando de esta manera una base arquitectónica robusta y segura para la futura expansión de la API.

5.7. ITERACIÓN 2: API REST, CONTROLADORES Y LOPDGDD

Objetivo global de la iteración. La segunda iteración del desarrollo tuvo como meta principal dotar a la API REST de un sistema de autenticación robusto basado en *tokens*. Asimismo, el esfuerzo se dirigió a estandarizar todas las rutas de comunicación del servidor bajo los principios arquitectónicos REST. Finalmente, el objetivo clínico consistió en desarrollar la lógica de negocio orientada al registro seguro y seudonimizado de los pacientes, garantizando el cumplimiento legal.

5.7.1. Planificación de tareas

Para organizar la implementación de la capa de seguridad y la gestión de pacientes, se definieron las historias de usuario y tareas técnicas correspondientes. En la Tabla 5.2 se presenta la planificación para esta segunda iteración, destacando la prioridad de los algoritmos de seudonimización y la estandarización del código.

5.7.2. Seguridad perimetral y autenticación JWT

El trabajo técnico arrancó con la implementación de la capa de seguridad mediante el estándar *JSON Web Tokens* (JWT). Para gestionar este protocolo, se creó la clase `JWTService`, encargada de generar *tokens* de acceso con una validez temporal predefinida. La firma

ID	Descripción de la tarea	Prioridad	Estado
SEC-02	Implementación de autenticación mediante JSON Web Tokens (JWT).	Alta	Completada
SEC-03	Control de acceso basado en roles (RBAC) en los controladores.	Alta	Completada
BACK-03	Desarrollo del `PacienteController` y servicios de registro.	Alta	Completada
LOPDGDD-01	Algoritmo de generación de ID clínicos sin caracteres ambiguos.	Alta	Completada
FIX-01	Creación de un *script* de inyección SQL para el administrador base.	Media	Completada
FIX-02	Refactorización lingüística de los *endpoints* al estándar inglés.	Baja	Completada

Tabla 5.2: Tareas planificadas y completadas durante la implementación de la seguridad JWT y el registro clínico.

criptográfica de estos *tokens* se realiza utilizando una clave secreta fuerte de alta entropía, la cual se mantiene rigurosamente protegida dentro de las variables de entorno del servidor.

Para hacer efectiva esta capa de protección, se programó un filtro de red personalizado denominado `JwtAuthenticationFilter`. Este componente actúa como un escudo protector en la arquitectura de Spring Boot, interceptando absolutamente todas las peticiones entrantes al servidor. Su función es verificar la existencia, la firma y la caducidad de un *Bearer Token* en la cabecera HTTP de autorización antes de permitir el acceso a los controladores.

Una vez validado el *token* criptográfico, el sistema extrae el contexto de seguridad y otorga permisos dinámicos según el rol del usuario, permitiendo distinguir funcionalmente entre administradores y enfermeros estándar. Gracias a esta lógica de control de acceso basado en roles (RBAC), se logró restringir las operaciones sensibles de la API, otorgando el privilegio de registrar o eliminar nuevo personal médico de forma exclusiva a los usuarios con credenciales de administración.

5.7.3. Gestión clínica y cumplimiento del RGPD

Tras consolidar la seguridad perimetral de la API, el desarrollo viró hacia la lógica de negocio clínica con la creación del `PacienteController` y su servicio asociado. El principal desafío analítico en este punto era lograr registrar a los pacientes de forma unívoca en la base de datos, pero garantizando su seudonimización para cumplir escrupulosamente con el Reglamento General de Protección de Datos (RGPD) [29].

Para solventar esta exigencia legal, se descartó el almacenamiento de nombres, apellidos o correos electrónicos de los usuarios. En su lugar, se programó un algoritmo de generación automática apoyado en la clase `SecureRandom` de Java. Cada vez que un sanitario solicita un alta en el sistema, el servidor genera de manera autónoma un identificador único y aleatorio compuesto por seis caracteres alfanuméricos.

Pensando en la Experiencia de Usuario (UX) dentro del entorno hospitalario, este algoritmo se refinó meticulosamente para prevenir errores humanos. Se configuró el código para excluir de forma automática del generador todos los caracteres visualmente ambiguos,

tales como el número cero, la letra 'O' mayúscula, el número uno y las letras 'I' mayúscula y 'l' minúscula. De esta forma, se minimiza exponencialmente el riesgo de equivocaciones cuando el personal de enfermería deba transcribir manualmente este código desde la web hacia los visores de RV.

5.7.4. Resolución de conflictos y estandarización

Durante esta fase temprana, la evolución constante de los modelos de datos provocaba que la herramienta de mapeo objeto-relacional (Hibernate) reiniciara y vaciara la base de datos con extrema frecuencia. Esta situación eliminaba sistemáticamente a los usuarios administradores creados previamente para las pruebas, ralentizando de manera inasumible los testeos de inicio de sesión mediante Postman.

Para solucionar este bloqueo continuo de desarrollo, se elaboró un *script* de inicialización nativo en SQL (`data.sql`). Este archivo se ejecuta automáticamente cada vez que el servidor arranca, inyectando un usuario administrador base de manera persistente y garantizando un entorno de pruebas siempre disponible.

Adicionalmente, durante las auditorías de código se detectó una falta de consistencia técnica en la nomenclatura de los *endpoints*. En el diseño inicial de la API se habían mezclado términos en inglés y español (como, por ejemplo, la ruta `/pacientes/registrar`). Para subsanarlo, se aplicó una refactorización completa que estandarizó todas las rutas de comunicación hacia el idioma inglés, respetando las buenas prácticas universales del diseño REST.

Esta iteración concluyó con éxito, entregando un servidor securizado, con una interfaz de programación estandarizada y capacitado para gestionar el ciclo de vida de los pacientes en estricto cumplimiento del marco legal.

5.8. ITERACIÓN 3: PROTOTIPADO DEL *ARKANOID*, ERGONOMÍA Y TELEMETRÍA

Objetivo global de la iteración. Al haber unificado las iteraciones iniciales de prototipado con las fases posteriores de gamificación y retroalimentación, el objetivo de este ciclo de desarrollo se expandió de manera significativa. La meta principal se centró en lograr un prototipo funcional del juego de tren superior basado en la mecánica de *Arkanoid*, integrando desde sus cimientos una serie de mecánicas de accesibilidad clínica para pacientes inmovilizados. Asimismo, el esfuerzo se dirigió a implementar un sistema ergonómico de interfaz adaptativa y a consolidar un motor de telemetría local robusto, rigiéndose por principios de arquitectura de *software* limpio.

5.8.1. Planificación de tareas

Dada la magnitud de este ciclo de desarrollo inmersivo, fue necesario estructurar el trabajo en múltiples frentes paralelos, abarcando desde la configuración del motor físico hasta el diseño de interfaz. En la Tabla 5.3 se detalla la planificación, reflejando las prioridades asignadas para lograr un Producto Mínimo Viable (PMV) jugable en RV.

5.8.2. Configuración del hardware virtual y arquitectura visual

El enlace entre el motor gráfico Unity y el *hardware* físico de las gafas supuso uno de los retos de configuración más laboriosos de esta fase temprana. Tras numerosas pruebas de depuración para estabilizar la ejecución y compilación del código en tiempo real a través del protocolo *Meta Link*, se logró integrar con éxito el reconocimiento espacial del usuario.

ID	Descripción de la tarea	Prioridad	Estado
VR-01	Integración del SDK de Meta y estabilización del rastreo 6DoF.	Alta	Completada
VR-02	Arquitectura visual: diseño de la sala secreta y proyección tridimensional.	Alta	Completada
GAME-01	Programación asimétrica del ROM para las palas y físicas de la pelota.	Alta	Completada
GAME-02	Sistema de *prefabs* para la construcción escalable de escenarios.	Media	Completada
UX-01	Menú modular y centrado ergonómico de la interfaz para pacientes recostados.	Alta	Completada
DATA-01	Módulo SRP de telemetría (fatiga, *jitter*, tiempos de reacción y exportación CSV).	Alta	Completada

Tabla 5.3: Tareas planificadas y completadas para el prototipado del juego de tren superior en RV.

Para ello, en lugar de programar los rastreadores desde cero, se utilizaron los componentes prefabricados del SDK nativo conocidos como *Building Blocks*. Se introdujo en la jerarquía el *Camera Rig* para posicionar la cabeza del jugador en el entorno virtual con seis grados de libertad (*6DoF*), y el componente *Controller Tracking* para leer ininterrumpidamente los sensores espaciales de los mandos derecho e izquierdo.

Una vez estabilizado el rastreo del dispositivo, se diseñó la arquitectura visual del ecosistema. Para mantener el entorno relajante del paciente intacto y evitar que la interfaz lúdica rompiera la inmersión clínica, se ideó un sistema de proyección tridimensional. Se construyó una "sala secreta" de juego completamente aislada y desplazada fuera del campo de visión natural del paciente, ubicándola en unas coordenadas subterráneas (específicamente en el eje Y a -50 unidades). Dentro de esta sala oculta transcurre toda la física real y el procesamiento de colisiones del juego.

Utilizando una cámara secundaria paramétrica (*cámara_arkanoid*) apuntando directamente a esta sala, se captura la imagen del juego en tiempo real y se proyecta mediante una textura de renderizado (*Render Texture*) sobre un panel transparente (*pantalla_arkanoid*) situado frente al jugador en el bosque natural. Esta compleja decisión arquitectónica, ilustrada en la Figura 5.4, permite al sistema ajustar el tamaño, la inclinación y la posición del panel de juego de manera totalmente libre en el entorno tridimensional sin alterar las físicas subyacentes ni la gravedad del simulador.

A nivel de recursos gráficos, para agilizar el desarrollo visual se importaron diversos paquetes de la tienda oficial *Unity Asset Store*. El entorno natural se construyó utilizando los modelos optimizados del *Low Poly Simple Nature Pack*[1]. Asimismo, para el acabado visual de la pelota, se integró el paquete de libre uso *Yughues Free Metal Materials*[2], dotando a la esfera de un material metálico altamente pulido que facilita su seguimiento ocular frente

[1]Paquete 3D *Low Poly Simple Nature Pack*: https://assetstore.unity.com/packages/3d/environments/landscapes/low-poly-simple-nature-pack-162153

[2]Paquete de materiales metálicos de *Yughues*: https://assetstore.unity.com/packages/2d/textures-materials/metals/yughues-free-metal-materials-12949



Figura 5.4: Vista en primera persona del paciente proyectando el nivel de Arkanoid sobre el entorno relajante.

al fondo, mientras que los materiales definitivos de las palas se mantuvieron con texturas planas temporales durante esta fase de prototipado.

5.8.3. Mecánicas de accesibilidad bi-manual y físicas de la pelota

El núcleo interactivo se controla a través del *script* `ControladorPalaVR`, el cual vincula la posición de la pala virtual a las coordenadas espaciales del mando físico. Teniendo en consideración las severas limitaciones del tratamiento de hemodiálisis, donde es habitual que uno de los brazos del paciente se encuentre inmovilizado por la canalización de la vía venosa, se desarrolló un sistema de accesibilidad modular y dinámico.

Se programó el esquema de control centralizando la lógica en un objeto padre denominado `Sistema_Palas`, el cual gestiona la instanciación y activación de dos palas físicas independientes (`Pala_Derecha` y `Pala_Izquierda`). Desde el menú de opciones, el paciente o el enfermero puede seleccionar en tiempo real el brazo habilitado para jugar. Como añadido terapéutico, si el usuario selecciona el modo bi-manual, aparecen ambas palas en pantalla, exigiendo una coordinación motriz independiente de cada extremidad que eleva drásticamente el valor de rehabilitación del ejercicio.

Para prevenir lesiones, el sistema prescinde de un mapeo de movimiento uno a uno. En su lugar, el *script* implementa un cálculo basado en el Rango de Movimiento (ROM) máximo, calibrado al inicio de la sesión. Tal y como se detalla en el Listado 5.1, el algoritmo interpola la distancia entre el centro anatómico y el mando, convirtiéndola en un porcentaje absoluto de alcance. Esto garantiza que el paciente pueda cubrir el cien por cien del ancho de la pantalla virtual realizando movimientos físicos muy cortos, mitigando cualquier riesgo de sobreesfuerzo en los hombros.

A nivel de diseño de físicas, se sustituyó el colisionador tradicional de la pala por un *CapsuleCollider* horizontal, otorgando al jugador la capacidad de alterar estratégicamente el ángulo de rebote al impactar la pelota con los bordes curvos. Además, se delimitaron los bordes del área de juego mediante un marco físico con un material emisivo para que el paciente perciba claramente los límites, incluso a través de la transparencia holográfica.

Listado 5.1: Algoritmo de interpolación clínica para el movimiento virtual de la pala basado en el ROM del paciente.

```
private float CalcularPosicionVirtual(float posFisicaX, bool esIzq)
{
    if (!DatosUsuarioDisponibles()) return 0f;
    // Extracción simplificada de las referencias del paciente
    var config = GestorDatosUsuario.Instancia.configActual;
    float centro = esIzq ? config.centroL : config.centroR;
    float maxDer = esIzq ? config.maxDerL : config.maxDerR;
    if (posFisicaX >= centro)
    {
        float rangoFisico = maxDer - centro;
        // Mecanismo de seguridad para evitar división por cero
        if (rangoFisico <= 0.05f) rangoFisico = 0.05f;
        // Cálculo del porcentaje de estiramiento (0.0 a 1.0)
        float desplazamiento = posFisicaX - centro;
        float porcentaje = Mathf.Clamp(desplazamiento / rangoFisico, 0f, 1f);
        // Proyección al entorno virtual
        return porcentaje * limiteDerecho;
    }
    // (Lógica para el lado izquierdo omitida)
    return 0f;
}
```

Finalmente, la lógica geométrica del *script* `ComportamientoPelota` exigió trabajar estrictamente con coordenadas locales (`localPosition`), dado que toda la partida transcurre desplazada a -50 unidades en el eje Y. Se configuró un material físico interno (*Physic Material*) con fricción cero y rebote perfecto. Para evitar errores críticos de jugabilidad, se implementó un mecanismo de seguridad anti-bloqueo (*softlock*) dentro del ciclo `Update`, el cual inyecta una ligera fuerza de corrección si detecta que la pelota ha entrado en un bucle infinito de rebotes puramente horizontales a muy baja velocidad vertical.

5.8.4. Sistema de bloques y gamificación escalable

Para dotar al ecosistema con la capacidad de volver a jugar sin comprometer el rendimiento del visor, se desarrolló una arquitectura de creación de niveles basada íntegramente en *Prefabs*. El comportamiento individual de cada ladrillo se encapsuló dentro del *script* `BloqueArkanoid`, el cual es responsable de gestionar los puntos de vida y de actualizar dinámicamente su aspecto visual.

Cada vez que el bloque registra un impacto, el código reemplaza su material iterando sobre un vector de texturas holográficas. Esto proporciona al paciente una retroalimentación visual clara e inmediata del daño infligido a la estructura.

El flujo de trabajo establecido para la creación de nuevos escenarios resulta sumamente escalable para los desarrolladores. El proceso requiere instanciar un objeto contenedor vacío dentro de la sala secreta y poblarlo con los distintos *prefabs* de bloques, ajustando sus posiciones relativas para conformar el patrón deseado.

Una vez ensamblado el nivel, se renombra el objeto padre (por ejemplo, “Nivel 01”) y se almacena en el directorio del proyecto como un nuevo archivo inmutable. Finalmente, este

recurso se añade a la matriz de niveles del *Game Manager*, permitiendo cargar y destruir escenarios completos en tiempo de ejecución de manera altamente eficiente.

Con el objetivo de enriquecer la experiencia lúdica e incentivar la atención del paciente, se programó un sistema de bonificaciones o *power-ups*. Destaca la implementación de una mejora de tipo "multibola", cuya probabilidad de aparición fue diseñada para escalar matemáticamente en función de la dureza del bloque destruido.

En concreto, se estableció una probabilidad del 5 % para los bloques más débiles, un 10 % para los de dureza media y un 15 % para los más resistentes. Al liberarse, la mejora cae físicamente hacia el jugador; si este logra interceptarla con su pala virtual, el gestor duplica instantáneamente las pelotas activas en pantalla, invirtiendo el eje de movimiento horizontal de las nuevas esferas para fomentar un seguimiento ocular activo.

5.8.5. Gestor de partida y control asíncrono

El control absoluto del flujo de la partida (*Game Loop*) recae sobre el *script* `GestorArkanoid`, el cual fue estructurado bajo el patrón de diseño *Singleton*. Esta arquitectura garantiza la existencia de una única instancia global de la clase, permitiendo que cualquier otro componente interactúe con el núcleo del juego de forma directa y centralizada.

Este "árbitro" lógico asume responsabilidades críticas como la instanciación del *prefab* correspondiente al nivel seleccionado, la generación de la pelota y la monitorización constante de las variables de estado (como los bloques restantes y las vidas perdidas) para dictaminar el final de la partida.

Dado que el motor Unity ejecuta su ciclo de vida fotograma a fotograma de forma estrictamente secuencial, la gestión de secuencias temporales requirió un enfoque asíncrono. Por ejemplo, para orquestar la cuenta atrás inicial sin congelar el renderizado visual de las gafas, se hizo uso de Corrutinas y de la instrucción `WaitForSeconds`.

5.8.6. Interfaz modular y ergonomía cervical adaptativa

Para garantizar la escalabilidad a largo plazo del código visual, se diseñó la Interfaz de Usuario implementando el concepto arquitectónico de Máquina de Estados. El menú principal flotante (`Canvas_MenuVR`) se dividió en paneles modulares independientes, tales como Bienvenida, Selección de Niveles, Pausa y Ajustes de Control.

Esta estructura lógica asegura que los paneles se activen de forma mutuamente excluyente, evitando la sobrecarga de información gráfica. Esto permite al paciente navegar de manera intuitiva utilizando el rayo visual (`PunteroLaserVR`), el cual se acopla dinámicamente a su controlador activo.

En este punto del desarrollo, se implementó una de las funcionalidades clínicas más trascendentales para el bienestar físico del paciente: el sistema de centrado de vista. Los pacientes de hemodiálisis reciben su tratamiento recostados en camillas, manteniendo una inclinación lumbar que oscila habitualmente entre los 45 y los 90 grados.

Obligar a estos usuarios a interactuar con un horizonte virtual fijo y perpendicular al suelo les forzaría a mantener una postura cervical antinatural y dolorosa durante los 45 minutos de la sesión. Para solventarlo, se programó un botón de calibración ergonómica dentro del menú flotante.

Al accionar este comando, el sistema captura las coordenadas espaciales (*headAnchor.position*) y el vector de dirección frontal (*headAnchor.forward*) del visor en ese instante preciso. Uti-

lizando estas lecturas tridimensionales, el motor matemático teletransporta y rota tanto la pantalla holográfica gigante del juego como el propio menú flotante, alineándolos a la perfección con la línea de visión natural del paciente y respetando plenamente su inclinación de descanso, como detalla la Figura 5.5.

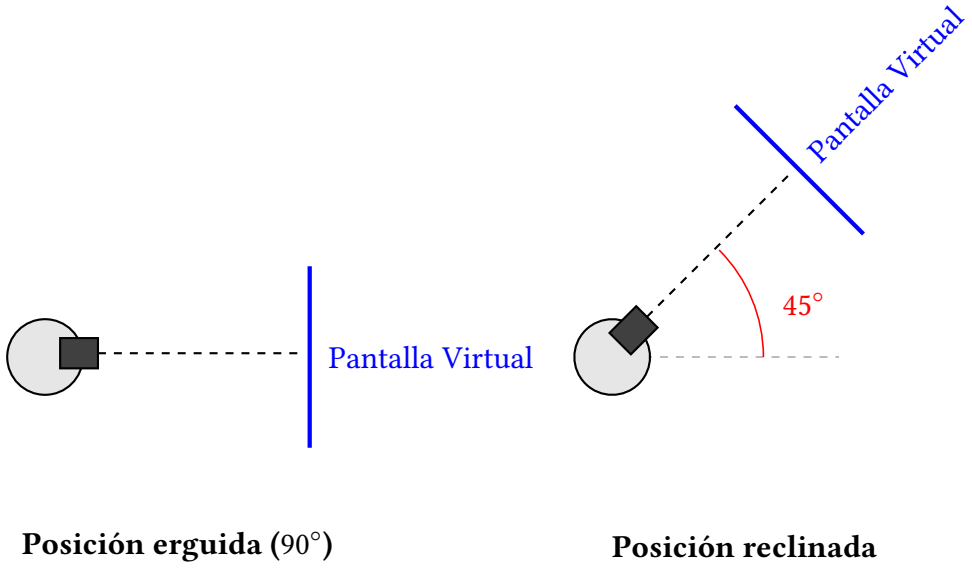


Figura 5.5: Esquema cinemático del centrado ergonómico adaptativo mediante centrado de vista.

5.8.7. Monitorización clínica y telemetría

Antes de ahondar en la arquitectura de recolección de datos, resulta esencial recordar que el propósito subyacente de este módulo interactivo basado en *Arkanoid* trasciende el mero entretenimiento visual del paciente durante sus horas de hemodiálisis. El objetivo clínico fundamental es la cinesiterapia: incentivar la realización de un ejercicio físico moderado y continuado de las extremidades del tren superior (los brazos) de una manera lúdica, enmascarando el esfuerzo terapéutico mediante mecánicas de juego. Por consiguiente, para poder evaluar la verdadera eficacia de esta actividad y proporcionar valor médico al personal sanitario, resulta imprescindible cuantificar de forma objetiva el comportamiento motor del usuario a lo largo de la partida.

Para garantizar la recolección de estadísticas clínicas de alto nivel sin penalizar el rendimiento gráfico del juego, el diseño del código se rigió estrictamente por el Principio de Responsabilidad Única (SRP). Toda la lógica de extracción, cálculo y almacenamiento de datos se separó de las mecánicas principales del juego y se delegó a un módulo gestor independiente, programado desde cero bajo el nombre de `MonitorClinico`.

Este sistema opera de manera completamente invisible en segundo plano, calculando métricas de valor para el personal de nefrología. En primer lugar, el monitor efectúa un registro continuo de lateralidad y uso bi-manual. El algoritmo registra el uso del mando izquierdo, del derecho o de ambos de forma simultánea. Esta métrica permitirá al personal clínico detectar si el paciente favorece un brazo de manera inconsciente, pero su valor más crítico radica en proporcionar métricas de rendimiento objetivas que cuantifican el volumen de ejercicio físico realizado. Esta recolección exhaustiva permitirá a los especialistas correlacionar posteriormente el esfuerzo intradialítico con las constantes vitales del

paciente medidas de forma externa, buscando establecer una relación clínica directa entre la ejercitación activa durante las sesiones y la mejora en su calidad de vida.

En segundo lugar, el gestor incorpora un algoritmo de detección de fatiga. Para ello, el sistema mide las micro-variaciones y los temblores musculares (*jitter*), calculando el delta de rotación espacial de las muñecas del paciente fotograma a fotograma. La acumulación de estas irregularidades a lo largo de los 45 minutos de partida genera un índice de cansancio.

Finalmente, el módulo evalúa el tiempo de reacción cognitivo-motor mediante un algoritmo predictivo de trazado geométrico. Para garantizar la fiabilidad de la métrica y evitar la sobrecarga del monitor, este cálculo se ejecuta exclusivamente cuando existe una única pelota activa en pantalla. El cronómetro interno se desencadena en el instante exacto en el que la esfera impacta contra un bloque y su vector de trayectoria resultante adopta un sentido descendente.

En esa misma fracción de segundo, el sistema proyecta una línea vectorial invisible que simula anticipadamente la caída de la pelota. Este cálculo matemático tiene en cuenta las colisiones cruzadas y los rebotes contra los muros laterales del nivel, avanzando hasta determinar la coordenada espacial exacta en el eje horizontal donde la pala deberá posicionarse para interceptarla de forma exitosa (la "zona de muerte" o altura de la pala), tal y como se detalla en el Listado 5.2.

El tiempo de reacción se detiene y se registra en la matriz de datos en el instante en que el paciente desplaza su pala virtual atravesando dicho punto de intercepción calculado. Como medida de depuración clínica, si la pala del jugador ya se encontraba posicionada en la coordenada de impacto en el momento exacto en el que la pelota rebotó en el bloque (generando un tiempo de respuesta de 0,00 segundos), el registro se descarta automáticamente, dado que obedece al azar posicional y no a un acto reflejo motor real. Al concluir la partida, el gestor extrae la media aritmética de todas las capturas válidas, obteniendo el tiempo de reacción consolidado de la sesión.

Para asegurar la preservación de toda esta información clínica, se programó un sistema de volcado automático apoyado en la clase `StreamWriter`. Al finalizar cada partida, el gestor consolida las métricas y genera un archivo en formato CSV alojado directamente en el almacenamiento persistente interno de las gafas Meta Quest. Se estableció un estándar de nomenclatura automatizado basado en la fecha y la hora exactas de la sesión, estructurando los datos de tal forma que faciliten su futura integración y envío hacia la API REST del servidor central.

5.8.8. Problemas encontrados y resultados finales

Durante el desarrollo de la interfaz de usuario en la sala secreta, la documentación oficial de Meta sugería el uso del componente *OVROverlayCanvas* para optimizar la nitidez de los textos. Sin embargo, al implementar esta tecnología, se descubrió que el componente renderizaba la interfaz directamente sobre las lentes del visor, saltándose la cámara de proyección de la sala secreta. Esto destruía por completo la ilusión de profundidad del holograma en el bosque natural, por lo que la idea fue descartada en favor de un *Canvas* en modo *World Space* estándar.

Por otro lado, la programación de la bonificación "multibola" presentó un error lógico crítico durante sus primeras pruebas en el motor gráfico. Al destruir los bloques, el objeto coleccionable no se instanciaba en la escena. Tras varias sesiones de depuración, se identificó

Listado 5.2: Algoritmo predictivo para el cálculo anticipado de la zona de aterrizaje de la pelota simulando rebotes espaciales.

```
private void CalcularPuntoAterrizaje(Vector3 direccion)
{
    Vector3 posActual = transform.position;
    Vector3 dirActual = direccion;
    float limiteX = 17.5f;
    float alturaPala = ↩
        ↪ ControladorPalaVR.ObtenerAlturaPalaActiva();

    int maxRebotes = 10; // Limite de seguridad anti-bucles
    while (posActual.y > alturaPala && maxRebotes > 0)
    {
        maxRebotes--;
        float distY = posActual.y - alturaPala;
        float tiempoHastaPala = distY / ↩
            ↪ Mathf.Abs(dirActual.y);
        float targetX = posActual.x + (dirActual.x * ↩
            ↪ tiempoHastaPala);

        if (targetX > limiteX) // Rebote en la pared derecha
        {
            float tPared = (limiteX - posActual.x) / ↩
                ↪ dirActual.x;
            posActual.x = limiteX;
            posActual.y += dirActual.y * tPared;
            dirActual.x = -dirActual.x; // Invertir vector X
        }
        else if (targetX < -limiteX) // Rebote en la izquierda
        {
            float tPared = (-limiteX - posActual.x) / ↩
                ↪ dirActual.x;
            posActual.x = -limiteX;
            posActual.y += dirActual.y * tPared;
            dirActual.x = -dirActual.x; // Invertir vector X
        }
        else // Aterrizaje directo sin chocar con más paredes
        {
            posActual.x = targetX;
            posActual.y = alturaPala;
            break;
        }
    }

    MonitorClinico.IniciarMedicionReaccion(posActual.x);
}
```

que las referencias a los *Prefabs* de las mejoras se habían desvinculado internamente en el panel de inspección de Unity, impidiendo que el código encontrara el modelo 3D a generar.

Solventados los desafíos técnicos y de interfaz, se logró compilar la versión Alfa `V0.2.3.apk`. Esta compilación ofreció la primera experiencia clínica completa, incluyendo el modo bimanual y el soporte para la adaptación cervical en camilla. Tras subsanar de forma definitiva el fallo de instanciación de las bonificaciones, la iteración concluyó con éxito mediante la compilación de la versión `V0.3.apk`, entregando un entorno terapéutico gamificado, accesible y capaz de exportar biometría compleja de manera autónoma.

5.9. ITERACIÓN 4: RETROALIMENTACIÓN SENSORIAL Y DIFICULTAD

Objetivo global de la iteración. La cuarta iteración del desarrollo se centró en pulir la experiencia de usuario del módulo de tren superior y estabilizar sus mecánicas principales. Los objetivos abarcaron la integración de retroalimentación audiovisual continua, la implementación de un sistema de dificultad paramétrico adaptado a distintos perfiles clínicos y el diseño arquitectónico de un núcleo de *software* reutilizable que sirviera como cimentación base para los futuros minijuegos terapéuticos del ecosistema.

5.9.1. Planificación de tareas

Para coordinar la estabilización de este primer módulo terapéutico, se planificó una batería de tareas enfocadas a la experiencia de usuario y a la refinación matemática del código. En la Tabla 5.4 se expone la planificación ejecutada durante este ciclo, destacando el desarrollo de la dificultad dinámica y la resolución lógica de estados.

ID	Descripción de la tarea	Prioridad	Estado
UX-02	Integración de *Trail Renderer* y *feedback* visual de colisión.	Alta	Completada
GAME-03	Ampliación de escenarios jugables (8 niveles modulares).	Media	Completada
FIX-03	Depuración lógica del conteo de estado de la mejora “multibola”.	Alta	Completada
CLIN-01	Sistema de dificultad paramétrico ligado a la exportación en CSV.	Alta	Completada
UX-03	Herramienta de recalibración dinámica del centro anatómico (ROM).	Media	Completada
ARCH-01	Aislamiento y empaquetado del “Núcleo Común” de *software* clínico.	Alta	Completada

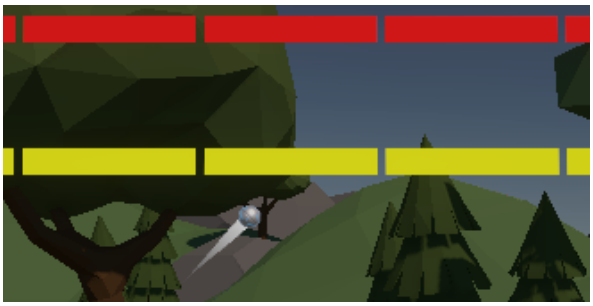
Tabla 5.4: Tareas planificadas y completadas para la estabilización del primer minijuego inmersivo.

5.9.2. Retroalimentación visual y corrección de estados lógicos

Para facilitar el seguimiento ocular de la pelota y mitigar posibles mareos o fatiga visual derivados de trayectorias muy rápidas en RV, se integró el componente de renderizado *Trail Renderer* nativo de Unity. Esta herramienta de *software* interpola las posiciones pasadas de la esfera para generar una estela tridimensional y persistente detrás de su núcleo geométrico.

Esto ayuda al paciente a predecir la trayectoria de los rebotes con un esfuerzo cognitivo y sacádico mucho menor.

Adicionalmente, se programó una corrutina específica en el *script* `BloqueArkanoid` para alterar de forma transitoria el material emisivo del ladrillo al recibir un impacto de físicas, secuencia que se ilustra en la Figura 5.6. Este código inyecta un destello blanco absoluto durante una fracción de segundo, proporcionando una confirmación visual instantánea de la colisión sin penalizar los recursos de la GPU con sistemas de partículas complejos. Para maximizar el contraste de estos estímulos, se ajustó el canal de opacidad (*Alpha*) del panel transparente que actúa como pantalla de juego a un valor de 200 (sobre 255), oscureciendo el fondo lo suficiente para destacar la interfaz lúdica sin perder la conexión ambiental con el bosque relajante.



(a) Aproximación



(b) Destello de impacto



(c) Cambio de estado

Figura 5.6: Secuencia de retroalimentación visual al colisionar con un bloque. Se aprecia el destello blanco transitorio inyectado por la Corrutina y la posterior degradación de color del material.

En el ámbito del diseño de recursos, se generó una iconografía clara e intuitiva para la bonificación "multibola" empleando herramientas de Inteligencia Artificial generativa. Todo ello se acompañó de una expansión del contenido clínico, utilizando la escalabilidad de

los *Prefabs* para diseñar, empaquetar e instanciar hasta un total de ocho niveles de juego independientes.

A nivel de ingeniería, para asegurar la estabilidad durante la ejecución masiva de estas entidades, fue necesario refactorizar la lógica de control de estados. Durante la depuración de variables en memoria, se detectó una duplicidad en el conteo de las entidades catalogadas como "pelota": el *script* gestor las registraba en su matriz interna tanto al ser instanciadas como clones, como al ejecutar su función de autoinicialización (`Start`). La centralización rigurosa de este conteo mediante un patrón de inyección de dependencias erradicó el error, evitando salidas prematuras al menú de fin de partida.

5.9.3. Sistema de dificultad clínica y calibración espacial dinámica

Con el propósito de ofrecer una terapia adaptable a pacientes con diferentes capacidades de movilidad o estadios de la enfermedad renal, se desarrolló un sistema dinámico de selección de dificultad. Este parámetro, ubicado estratégicamente en el panel de "Selección de Niveles" de la interfaz flotante, exige que el usuario o el enfermero predefina la dureza de la sesión antes de comenzar el ejercicio. La opción seleccionada altera de forma algorítmica las variables físicas y espaciales del entorno tridimensional.

En el modo "Fácil", la escala geométrica de las palas se incrementa un 50 %, otorgando al usuario un amplio margen de error posicional al interceptar. De manera paralela, el vector inicial de fuerza aplicado sobre la pelota se mitiga un 30 %, reduciendo la velocidad cinética de la partida, y la probabilidad programada para la aparición de mejoras ventajosas se multiplica por un factor de 1.5. Por el contrario, la selección del modo "Difícil" aplica una penalización del 25 % al tamaño de impacto de la pala, incrementa la aceleración de la esfera y recorta a la mitad las bonificaciones, forzando tiempos de reacción drásticamente más cortos.

Esta parametrización se encuentra estrictamente enlazada mediante eventos al módulo `MonitorClinico`. Consecuentemente, el nivel de dificultad elegido se estampa en la cabecera de metadatos del archivo CSV exportado. Esta trazabilidad de datos resulta indispensable para la investigación científica del equipo de nefrología, permitiéndoles cruzar de forma fidedigna los tiempos de reacción y las anomalías musculares detectadas con la curva de exigencia real a la que fue sometido el paciente en cada sesión.

Por último, dado que la posición física de descanso de los pacientes suele variar sutilmente a lo largo de las horas que dura el tratamiento en la camilla, se detectaron desajustes acumulativos en la asimetría de los brazos virtuales. Para subsanar este escollo ergonómico, se desarrolló una herramienta de recalibración asimétrica en tiempo real. Mediante un panel accesible en la interfaz, el sistema solicita al usuario que devuelva sus manos al eje central de su regazo durante una cuenta regresiva auditiva de tres segundos; acto seguido, el motor sobrescribe ese punto espacial y lo establece como el nuevo origen de coordenadas cero (*Origin Offset*) para recalcular la proyección del Rango de Movimiento (ROM).

5.9.4. Interfaz de usuario y diseño sonoro

La navegación y el control general del juego se enriquecieron integrando un sistema de interrupción táctil directamente accesible desde los controladores físicos. Se implementó un mapeo de entradas (*Input Mapping*) que vincula los botones *Start* y *X* en el mando izquierdo, así como el botón *A* en el derecho, a una rutina de interrupción global. Al registrarse la pulsación, el gestor congela la escala de tiempo del motor físico (`Time.timeScale = 0`) y despliega de inmediato el menú holográfico de pausa. Para optimizar la usabilidad y agilizar

el retorno a la acción clínica, se añadió un botón explícito de "Reanudar" en el panel virtual, evitando que el paciente sufra carga cognitiva intentando recordar combinaciones de botones para salir.

El diseño sonoro se abordó buscando un equilibrio estricto entre la relajación necesaria en el entorno hospitalario y el refuerzo positivo (*feedback*) indispensable en los juegos interactivos. Como hilo musical de fondo, se integró una pista instrumental de género *Lo-Fi Jazz*. Esta composición fue generada mediante Inteligencia Artificial y obtenida a través de la plataforma *Pixabay*, garantizando un ambiente auditivo calmado y completamente exento de restricciones por derechos de autor (*Royalty-Free*). Para otorgar control total al usuario sobre esta ambientación, se enlazó un deslizador paramétrico (*slider*) de volumen en el panel de ajustes de la interfaz.

Los efectos de sonido interactivos se implementaron utilizando recursos de alta fidelidad procedentes del portal para desarrolladores independientes *Kenney.nl*[3]. Se asignaron pistas de tipo *Sci-Fi* para enfatizar el inicio de la partida y la recolección de las mejoras. Asimismo, se integraron audios de interfaz nítidos para las cuentas atrás, y se aplicaron efectos de percusión acústica (emulando madera y cristal) extraídos del paquete *Impact Sounds* para diferenciar sonoramente los rebotes de la esfera contra los muros perimetrales, las palas virtuales y los bloques destructibles.

5.9.5. Arquitectura del núcleo común y próximos pasos

Uno de los hitos de ingeniería de *software* más relevantes de esta iteración fue la consolidación del "núcleo común" del proyecto. Siguiendo los principios de alta cohesión y bajo acoplamiento, se procedió a identificar, abstraer y aislar todos los módulos lógicos que no dependían de manera exclusiva de las mecánicas físicas del *Arkanoid*.

Elementos transversales como el registro seudonimizado de datos, el módulo gestor de telemetría (`MonitorClinico`), la arquitectura de menús flotantes basados en máquinas de estados, los sistemas de puntero láser interactivo y los perfiles de calibración asimétrica de extremidades, se empaquetaron en un bloque lógico independiente. Este diseño modular garantiza una reutilización absoluta del código fuente. De esta forma, se estableció una cimentación tecnológica robusta que permitirá acelerar de manera drástica el desarrollo de los futuros juegos terapéuticos enfocados al tren inferior, evitando cualquier duplicidad estructural en el ecosistema de RV.

Con la estabilidad matemática y visual del módulo de extremidades superiores plenamente garantizada, y tras implementar un panel de depuración en pantalla (*debug text*) funcional para facilitar las futuras pruebas internas sin conexión a un equipo informático, la iteración concluyó con éxito. Este cierre abrió paso a la investigación de nuevas mecánicas inmersivas complejas, planificando para la siguiente iteración el prototipado de pelotas con propiedades explosivas temporales y la experimentación geométrica con pantallas de proyección curvas capaces de envolver la visión periférica del paciente.

5.10. ITERACIÓN 5: CIERRE DEL NÚCLEO Y CALIBRACIÓN ASIMÉTRICA

Objetivo global de la iteración. Esta quinta iteración se dedicó a cerrar los objetivos clínicos planteados para el módulo del tren superior y a establecer de forma definitiva el "núcleo común" de *software*. La meta principal consistió en rediseñar la arquitectura

[3]Repositorio de recursos y materiales de dominio público *Kenney.nl*: https://kenney.nl/assets

de persistencia de datos para hacerla escalable, perfeccionar el algoritmo matemático de calibración de las palas y pulir la retroalimentación audiovisual del simulador.

5.10.1. Planificación de tareas

Para coordinar la refactorización profunda de los sistemas de guardado y la implementación de las nuevas mecánicas, se definieron las tareas descritas en la Tabla 5.5. En esta planificación destacan la intercepción de comandos de Android (*Intents*) y la programación de la pantalla curva adaptativa.

ID	Descripción de la tarea	Prioridad	Estado
DATA-02	Sistema de carpetas por paciente e intercepción de *Intents* (ADB).	Alta	Completada
DATA-03	Volcado de biometría en crudo a 10 Hz y detección de tirones.	Alta	Completada
CLIN-02	Rediseño asimétrico del algoritmo de rango de movimiento.	Alta	Completada
GAME-04	Mecánica de mejora "explosiva" y daño en área.	Media	Completada
AUDIO-01	Implementación del patrón *Audio Bridge* para sonidos persistentes.	Media	Completada
UX-04	Diseño paramétrico de pantalla envolvente curva (120 grados) y FOV.	Alta	Completada

Tabla 5.5: Tareas planificadas y completadas para la consolidación del núcleo lógico y la telemetría biomecánica.

5.10.2. Evolución de la persistencia y comunicación externa

Para establecer una cimentación lógica modular que fuera importable en futuros juegos de rehabilitación, se creó la clase central `GestorDatosUsuario` y se aplicó una refactorización severa al `MonitorClinico`. Se abandonó el enfoque primitivo de exportar archivos CSV sueltos y desordenados en el directorio raíz del visor tras cada partida. En su lugar, el sistema genera ahora una jerarquía de carpetas unificada, utilizando el identificador alfanumérico único del jugador como nombre del directorio raíz.

Para inyectar este identificador de forma automatizada en el entorno hospitalario, se implementó un puente de comunicación de bajo nivel empleando Reflexión en Java (*Java Reflection*). Este código intercepta los *Intents* del sistema operativo Android, permitiendo a la aplicación de Unity recibir el ID del usuario directamente desde comandos externos (ADB) o desde aplicaciones lanzadoras de terceros creadas por el equipo de redes. En el caso de no detectarse un ID externo, el sistema recurre a la instanciación de un perfil de "Invitado", generando una subcarpeta basada en la marca de tiempo exacta para garantizar que los expedientes de distintos pacientes jamás colisionen.

Dentro del directorio nativo de cada paciente, el gestor deposita un archivo `config.json`. Este documento almacena de forma persistente las preferencias clínicas del usuario (tales como el volumen, la dificultad seleccionada, los mandos habilitados, la distancia paramétrica de la pantalla y los vectores de calibración espacial). Al iniciarse una nueva sesión terapéutica

para un mismo paciente, el *script* localiza este archivo y autoaplica la matriz de configuración en tiempo de ejecución.

Asimismo, se consolidó la creación de un archivo maestro de seguimiento clínico, denominado `historial_partidas.csv`. Este documento concatena secuencialmente todas las partidas jugadas por el paciente, estructurando los datos en columnas que registran la fecha, el nivel, la dificultad, la duración total, el índice acumulado de fatiga y el tiempo medio de reacción cognitivo-motor.

Adicionalmente a este resumen estadístico, se preparó una subcarpeta específica destinada a almacenar la telemetría biomecánica exhaustiva en crudo (*raw data*) para sustentar futuras expansiones investigativas en el campo de la nefrología. Este sub-registrador captura a una frecuencia controlada de 10 hercios (10 veces por segundo) la posición, rotación y velocidad lineal tanto del visor como de ambos controladores. Como medida de seguridad clínica activa, se integró un algoritmo lógico que detecta aceleraciones anómalas superiores a 3.0 m/s^2; de este modo, si el paciente realiza un movimiento brusco peligroso para su brazo canalizado, el sistema marca automáticamente dicho fotograma en el registro CSV con la etiqueta `MOVIMIENTO_BRUSCO`.

5.10.3. Rediseño del algoritmo de calibración asimétrica y lateralidad

El algoritmo matemático que rige el control de las palas fue rediseñado por completo para soportar un mapeo espacial asimétrico. Esta mejora paramétrica permite que un paciente, jugando con una sola extremidad habilitada (por ejemplo, el brazo derecho), pueda proyectar su pala virtual hasta el extremo izquierdo de la pantalla recorriendo una distancia física mucho menor, evitando así cruzar forzadamente el eje central de su cuerpo. La calibración del Rango de Movimiento (ROM) se ejecuta ahora de manera estrictamente individual para cada controlador a partir del centro corporal (pecho o regazo) que el paciente asigne.

Para integrar esta funcionalidad, se programó una rutina de calibración secuencial. En el caso del modo de juego bi-manual, el sistema solicita al usuario calibrar primero la máxima extensión del brazo izquierdo y, posteriormente, la del derecho. Para garantizar una asimilación cómoda de las instrucciones holográficas en pantalla, el tiempo de respuesta de cada fase de calibración se incrementó holgadamente de tres a cinco segundos.

Durante la fase de pruebas de esta nueva rutina, se detectó un defecto lógico de colisión de datos en la memoria: la última calibración realizada (habitualmente la del brazo derecho) sobrescribía las variables del brazo opuesto. Para subsanar este fallo, se refactorizó la estructura interna de la clase `GestorDatosUsuario`, segmentando y encapsulando de forma estricta las variables de telemetría, alcance y calibración mediante enumeradores de lateralidad (*Left/Right*).

Directamente vinculado a este avance, se implementó un registro dinámico de impactos segmentados por lateralidad. El monitor clínico contabiliza en segundo plano el volumen exacto de veces que el paciente intercepta la pelota utilizando el brazo izquierdo frente al derecho. La extracción de estas dos variables permite a la plataforma web calcular posteriormente el índice de simetría, una métrica de altísimo valor para que el personal médico pueda detectar patrones subconscientes de evitación, cansancio asimétrico o dolor en la extremidad canalizada.

5.10.4. Gamificación explosiva y patrón arquitectónico *Audio Bridge*

En el apartado de juegabilidad, se introdujo una nueva bonificación de tipo "explosiva", cuyo recurso artístico (icono tridimensional) fue generado como material de apoyo mediante herramientas de Inteligencia Artificial. Al ser interceptada por el paciente, el gestor aplica un modificador de estado: todas las pelotas activas en pantalla adquieren propiedades volátiles durante cinco segundos. Al colisionar, el *script* traza una esfera matemática de superposición (`Physics.OverlapSphere`) para causar daño en área (*AoE*) a los bloques colindantes, infligiendo además daño crítico al bloque directamente impactado.

Para comunicar visualmente este estado de peligro al jugador, se programó una subrutina que hace parpadear el material emisivo de la esfera a medida que la mejora está a punto de concluir. Si el paciente recoge el objeto de forma múltiple, la lógica reinicia el temporizador de la Corrutina al máximo, evitando conflictos de superposición temporal.

A nivel de diseño sonoro, se enriqueció la experiencia integrando nuevos recursos auditivos procedentes de los paquetes *Music Jingles* y *Sci-Fi Sounds*, incorporando fanfarrias de victoria, tonos de derrota y graves crujidos de explosión.

Para garantizar la correcta ejecución de estos efectos espaciales, fue imperativo implementar el patrón de diseño *Audio Bridge* en la arquitectura del `GestorArkanoid`. Dado que en el motor Unity los objetos volátiles (como los bloques de cristal o las cápsulas de mejora) se eliminan de la memoria mediante el comando `Destroy()` casi instantáneamente tras colisionar, sus componentes `AudioSource` son destruidos antes de que el archivo de sonido termine de reproducirse. El patrón *Audio Bridge* soluciona este conflicto delegando la ejecución del clip de audio al gestor persistente de la partida (el *Singleton*), asegurando que la cola de sonido se reproduzca íntegra y de forma asíncrona, independientemente del corto ciclo de vida del objeto emisor.

5.10.5. Ergonomía visual envolvente y cuadros de mando interactivos

Con el objetivo de maximizar la inmersión del usuario y proteger su ergonomía visual, se implementó un formato de renderizado en pantalla curva con una amplitud angular de 120 grados, activable dinámicamente desde el menú de ajustes. Esta geometría cilíndrica envolvente garantiza que todos los puntos del panel holográfico se mantengan a la misma distancia óptica focal de las lentes del visor. De esta manera, se minimiza la fatiga ocular por reenfoque y se erradica la distorsión geométrica periférica característica de los paneles planos excesivamente anchos en RV.

Para complementar esta adaptación espacial, se integraron deslizadores paramétricos (*sliders*) que otorgan al enfermero el control absoluto sobre la distancia de la pantalla y el menú flotante, acotados en un rango seguro de 1,5 a 6,0 metros. El algoritmo de proyección se encarga de escalar matemáticamente la pantalla curva de manera proporcional a la distancia seleccionada; este redimensionamiento dinámico logra preservar inalterado el Campo de Visión envolvente (*FOV*) percibido por el paciente. Estas métricas exactas de distanciamiento se persisten y diferencian automáticamente en el archivo JSON personal del usuario.

Por último, se rediseñó desde cero la experiencia de usuario (*UX*) durante la suspensión de la partida. El panel básico de pausa fue sustituido por un cuadro de mando complejo (*Dashboard*). Este panel consulta dinámicamente las variables internas del `GestorArkanoid` en tiempo real para renderizar el nivel actual, la dificultad, el cronómetro (formateado en minutos y segundos), los bloques restantes y la suma total de sus puntos de vida. De forma análoga, al finalizar la sesión, el sistema oculta los textos planos de depuración (*debug text*) y

despliega un panel informativo altamente pulido que presenta las estadísticas finales bajo un enfoque de refuerzo positivo, culminando así el Producto Mínimo Viable (PMV) del módulo de tren superior.

5.11. ITERACIÓN 6: REFINAMIENTO DE UX Y *ARKANOID* VERSIÓN FINAL

Objetivo global de la iteración. Esta sexta iteración constituyó la fase definitiva de pulido y aseguramiento de calidad (QA) para el módulo de rehabilitación del tren superior. El objetivo central fue solucionar los errores lógicos residuales de las versiones previas, implementar un sistema de notificaciones no intrusivo y consolidar una experiencia de usuario (UX) impecable que permitiera dar por concluida la versión candidata a lanzamiento (*Release Candidate 1.0*).

5.11.1. Planificación de tareas

Para coordinar el cierre de este primer gran bloque del Trabajo de Fin de Grado, se identificaron y catalogaron las tareas de mejora visual y corrección de errores críticos. En la Tabla 5.6 se expone la planificación ejecutado durante esta iteración final.

ID	Descripción de la tarea	Prioridad	Estado
UX-05	Sistema de notificaciones flotantes adaptativas (*Lazy HUD*).	Alta	Completada
UX-06	Textos dinámicos de puntuación e inyección de récords personales.	Alta	Completada
CLIN-03	Refinamiento visual de la calibración geométrica y bloqueo de láser.	Media	Completada
GAME-05	Rediseño de la curva de dificultad y reordenación de niveles.	Media	Completada
FIX-04	Corrección del búfer de profundidad (*Z-Ordering*) y menús.	Alta	Completada
FIX-05	Resolución de *bugs* lógicos y *clipping* con la topología del suelo.	Alta	Completada

Tabla 5.6: Tareas planificadas y completadas para el cierre y pulido de la versión final del módulo de tren superior.

5.11.2. Sistema de notificaciones flotantes (Lazy HUD)

En entornos inmersivos de RV, anclar elementos de la interfaz de forma rígida a la cámara del usuario (*Head-Locked HUD*) suele generar severos episodios de fatiga visual y cinetosis. Para informar al paciente sobre controles ergonómicos clave (como el uso de los botones *B* o *Y* para centrar la vista), se diseñó un sistema de notificaciones flotantes adaptativas, comúnmente denominado en la industria como *Lazy HUD*. En la Figura 5.7 se expone el diseño visual de este panel proyectado durante la sesión.

Toda esta lógica matemática se centralizó en el *script* `NotificacionFlotanteVR`. Este componente proyecta el panel de texto de tal forma que no persigue la mirada del usuario de forma rígida e instantánea, sino que interpola su posición de manera fluida. Como se detalla en el Listado 5.3, el algoritmo calcula un punto ideal situado ligeramente por debajo

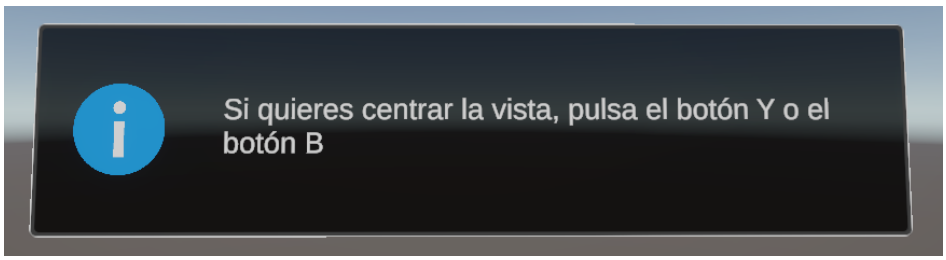


Figura 5.7: Panel de notificación flotante (*Lazy HUD*) informando al paciente sobre los controles ergonómicos de centrado.

de la línea directa de visión del paciente (`desplazamientoAbajo`), evitando así bloquear su campo de visión central durante la partida clínica.

Listado 5.3: Algoritmo de seguimiento ocular suavizado (Lazy HUD) utilizando interpolación matemática escalar y rotacional.

```
void ActualizarHUD()
{
    if (!HUD_Visible) return;

    // Calcular punto objetivo (al frente y ligeramente abajo)
    Vector3 posIdeal = cabeza.posicion
                     + (cabeza.frente * distFrente)
                     + (cabeza.arriba * distAbajo);

    // Interpolar posicion espacial de forma asincrona
    float paso = Time.unscaledDeltaTime * velSeguimiento;
    transform.position = Vector3.Lerp(transform.position, posIdeal, paso);

    // Rotacion esferica para mantener la legibilidad
    Vector3 dirHaciaCabeza = transform.position - cabeza.posicion;

    if (dirHaciaCabeza != Vector3.zero)
    {
        Quaternion rotIdeal = Quaternion.LookRotation(dirHaciaCabeza);
        transform.rotation = Quaternion.Slerp(transform.rotation,
                                              rotIdeal, paso);
    }
}
```

Para garantizar una integración orgánica, estas notificaciones se instancian en pantalla utilizando una rutina de fundido (*Fade-In*), modificando de manera programática el canal de transparencia (*Alpha*) del *CanvasGroup*. Tras permanecer visibles durante los segundos especificados en la llamada del evento, desaparecen progresivamente, asegurando una lectura cómoda y libre de distracciones bruscas.

5.11.3. Refinamiento del orden de renderizado y gamificación

La gestión de la superposición espacial (*Z-Ordering*) fue recalibrada. Se acotaron los valores máximos y mínimos de las coordenadas de profundidad para asegurar que el panel del menú siempre se instancie físicamente más cerca del usuario que la pantalla holográfica del juego, evitando colisiones de polígonos incómodas. Asimismo, el holograma de la partida solo es visible durante el transcurso del propio juego o al entrar en el estado de pausa, ocultándose de forma automática si el paciente navega por menús no relacionados.

Para facilitar una curva de aprendizaje mucho más suave para los pacientes de mayor edad, se rediseñó el catálogo de retos. Los niveles fueron reordenados metodológicamente, reduciendo la densidad de ladrillos en los escenarios iniciales. Adicionalmente, el control de la interfaz flotante se simplificó: se eliminó el deslizador de distancia del menú principal (fijándolo en una coordenada inmutable de 2.5 metros) y se sustituyó por un controlador paramétrico de escala general.

El sistema de retroalimentación de la puntuación recibió una actualización visual de alto impacto con la implementación de un gestor de textos dinámicos (*Dynamic Text Manager*). Este módulo instancia puntuaciones flotantes efímeras en las coordenadas tridimensionales exactas de cada bloque destruido, las cuales se desvanecen ascendiendo por la pantalla. De forma paralela, la previsualización del menú ahora inyecta dinámicamente el récord absoluto del paciente; si logra superar dicha marca, el sistema despliega un mensaje holográfico especial de nuevo récord, incentivando su motivación motriz.

Respecto al sistema de calibración geométrica, se realizaron dos optimizaciones de experiencia de usuario. En primer lugar, se desactivó deliberadamente el *raycasting* de los controladores mientras la rutina de calibración está en progreso, evitando interacciones nulas. En segundo lugar, se inyectaron imágenes descriptivas en la interfaz para cada fase del proceso (reposo y extensión lateral), ilustrando de forma inequívoca la postura corporal requerida al paciente.

5.11.4. Resolución de errores y versión candidata

La fase intensiva de pruebas (QA) reveló diversos defectos lógicos que fueron subsanados. El error más crítico provocaba que el texto flotante de puntuación mostrara siempre los puntos base de los bloques, ignorando el multiplicador paramétrico de la dificultad seleccionada. En el ámbito de la interfaz gráfica, se solucionó un conflicto nativo de la API de entradas (*Input System*) de Unity, el cual provocaba que el leve movimiento inercial de los *joysticks* analógicos alterara involuntariamente los valores de los *sliders* del menú.

Por último, el algoritmo responsable del centrado ergonómico fue recodificado. El nuevo cálculo matemático evalúa la topología inferior e impide de forma restrictiva que la pantalla virtual, especialmente en su variante envolvente curva, pueda atravesar el suelo (*clipping*) al alinearse con la mirada de un paciente altamente recostado sobre la camilla. El éxito en la implementación de todas estas optimizaciones permitió compilar un archivo ejecutable extremadamente estable, sentando las bases definitivas para la candidata de lanzamiento y dando por concluido el desarrollo integral del tren superior.

5.12. ITERACIÓN 7: DESARROLLO DEL TREN INFERIOR: JUEGO DE RITMO

Objetivo global de la iteración. Tras consolidar el ecosistema de extremidades superiores, el proyecto evolucionó hacia su segundo gran hito: el diseño y programación de un "juego serio" orientado al movimiento del tren inferior. El diseño base plantea una terapia de ritmo

donde el usuario debe adoptar distintas posturas de piernas, sincronizando sus movimientos con estímulos audiovisuales. Dada la envergadura técnica de este desarrollo, la iteración abarcó desde la cimentación arquitectónica inicial hasta la creación de herramientas de autoría propias. El desarrollo de este juego se pudo completar en una sola iteración al haber mayor liberación de carga de trabajo en el resto de asignaturas. El nombre que recibió este juego es *Piernas VRítmicas*.

Es necesario mencionar que el juego desarrollado toma gran inspiración de títulos comerciales como *Beat Saber* o *Friday Night Funkin'*, adaptando sus mecánicas principales a los requerimientos de la rehabilitación clínica.

5.12.1. Planificación de tareas

Para estructurar la inmensa carga técnica de esta iteración, se elaboró un desglose de tareas que guio el desarrollo desde la configuración gráfica inicial hasta la refactorización final del servidor central. En la Tabla 5.7 se expone la planificación y las tareas completadas.

ID	Descripción de la tarea	Prioridad	Estado
ARCH-02	Separación del proyecto y empaquetado del núcleo (*Custom Package*).	Alta	Completada
UX-07	Diseño del área de juego y sistema ergonómico de túnel dinámico.	Alta	Completada
VR-03	Algoritmo de telemetría de piernas y validación física.	Alta	Completada
GAME-06	Desarrollo de mecánicas de ritmo, cinemáticas y gamificación visual.	Alta	Completada
TOOL-01	Creación del editor de niveles interno y cuantización de *beats*.	Media	Completada
BACK-04	Refactorización agnóstica del servidor y cumplimiento LOPDGDD.	Alta	Completada

Tabla 5.7: Tareas planificadas y completadas para el desarrollo del ecosistema inmersivo del tren inferior.

5.12.2. Configuración del entorno y reutilización arquitectónica

Para evitar conflictos de dependencias, optimizar el peso del compilado final y mantener una arquitectura de *software* cohesiva, además de mejorar la escalabilidad del proyecto, se tomó la decisión técnica de separar el juego de piernas en un proyecto independiente. Se inicializó un nuevo espacio de trabajo en Unity configurado con el *Universal Render Pipeline* (URP) y parametrizado específicamente para la compilación nativa en Android. Este entorno se vinculó a un nuevo repositorio de control de versiones en GitHub, integrando los archivos de exclusión (*.gitignore*) estandarizados por la industria para omitir archivos temporales y cachés locales.

El mayor desafío arquitectónico de esta fase inicial consistió en importar el núcleo lógico desarrollado durante las iteraciones del módulo superior, evitando "reinventar la rueda". Los *scripts* base de telemetría y persistencia de datos (como el `GestorDatosUsuario` y

el `MonitorClinico`), junto con el sistema de notificaciones flotantes (*Lazy HUD*), fueron empaquetados y exportados mediante la herramienta *Custom Package* de Unity.

Al importar este paquete en el nuevo proyecto, se eliminaron todas las dependencias lógicas directas que hacían referencia al código de *Arkanoid*, consolidando una arquitectura modular que se ilustra detalladamente en la Figura 5.8. En su lugar, las llamadas de estado se enrutaron hacia un nuevo controlador unificado denominado `GestorRitmo`.

Esta decisión estratégica garantizó que el guardado de las sesiones clínicas mantuviera una compatibilidad absoluta con el estándar de archivos JSON y CSV previamente estructurado, permitiendo compartir los expedientes de los pacientes entre ambas aplicaciones del Trabajo de Fin de Grado.

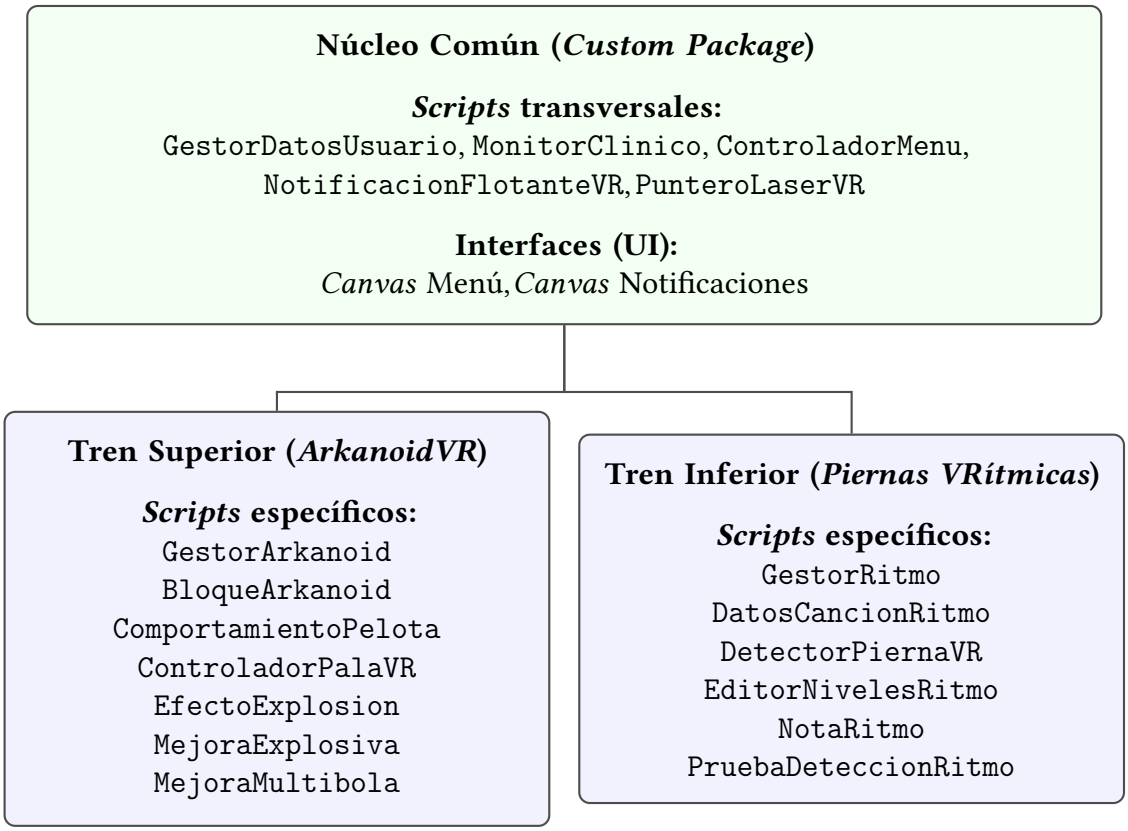


Figura 5.8: Arquitectura de componentes del sistema. Se ilustra la importación y reutilización de los *scripts* e interfaces del núcleo común para la construcción de los dos módulos terapéuticos independientes.

5.12.3. La interfaz de juego y el túnel dinámico adaptativo

A nivel visual, se diseñó un entorno inmersivo bautizado internamente como "Gimnasio Zen". Para su construcción, se emplearon recursos optimizados importados de la tienda oficial, combinando texturas cuadriculadas del *Gridbox Prototype Materials*[4] para el suelo, con el fondo nocturno estrellado del *Stylized Skyboxes*[5].

[4]Paquete Gridbox Prototype Materials: https://assetstore.unity.com/packages/2d/textures-materials/gridbox-prototype-materials-129127

[5]Paquete de Stylized Skyboxes: https://assetstore.unity.com/packages/2d/textures-materials/gridbox-prototype-materials-129127

El principal obstáculo de usabilidad clínica en este entorno residía en la postura de descanso de los pacientes. Al encontrarse recostados en camillas de hemodiálisis con una inclinación lumbar de entre 45 y 90 grados, establecer un horizonte fijo les obligaría a forzar severamente las cervicales. Para resolverlo, se diseñó el concepto de "Túnel Dinámico", cuya interfaz de usuario resultante se expone en la Figura 5.9.

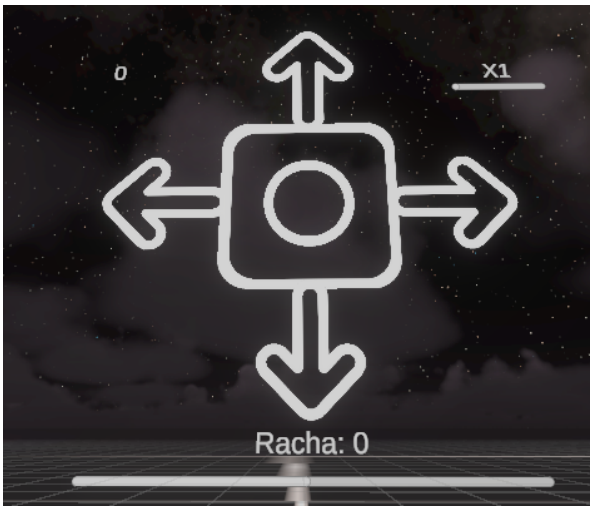


Figura 5.9: Vista inmersiva del túnel dinámico en el entorno "Gimnasio Zen", adaptado a la línea de visión del paciente y mostrando los indicadores de ritmo, puntuación y vida.

En lugar de fijar la pista rítmica al suelo del mundo tridimensional, todo el contenedor principal del juego se encapsuló en un objeto padre móvil. Al iniciar la partida o al pulsar el botón de recalibración de la interfaz, el sistema ejecuta la rutina de alineación detallada en el Listado 5.4. El algoritmo captura el vector frontal (*Forward*) del visor y posiciona la pista levitando exactamente frente a la línea de visión del paciente. Esto le permite jugar de forma totalmente ergonómica, incluso si su mirada se dirige en diagonal hacia el techo del hospital.

5.12.4. Correcciones heredadas de la interfaz y proyecciones visuales

Durante la exportación de los módulos comunes de la interfaz de usuario (*UI*), se aplicaron diversas micro-correcciones de calidad de vida basadas en la retroalimentación del módulo anterior. Se introdujo un margen de tolerancia espacial en el centrado automático del menú, exigiendo que el usuario desplace la cabeza una distancia mínima significativa antes de que la interfaz flote hacia su nueva posición, evitando así movimientos parasitarios molestos producidos por la respiración natural del paciente. Asimismo, se redujo el tamaño escalar por defecto del menú y se ampliaron los márgenes delimitadores del botón de ajustes para facilitar el uso del puntero láser.

Por último, se perfeccionó la lógica geométrica de la pantalla curva virtual heredada del prototipo anterior. Para evitar que los extremos del panel holográfico atravesaran la topología del entorno virtual (*clipping*), se enlazó su altura mínima de forma proporcional a la distancia del jugador. De este modo, si el paciente mira hacia el frente y acerca la interfaz, la pantalla mantiene un ángulo perpendicular estricto respecto al suelo; la inclinación envolvente solo se activa si los giroscopios detectan que el usuario está apuntando activamente su mirada hacia el techo, preservando la coherencia espacial de la habitación.

Listado 5.4: Algoritmo de centrado ergonómico (*Túnel Dinámico*) basado en la proyección del vector direccional de la cabeza.

```
public void CentrarPista()
{
    if (cabeza == null || contenedorPista == null) return;

    Vector3 posCabeza = cabeza.position;
    Vector3 dirMirada = cabeza.forward;

    // Correccion de cabeceo si el paciente mira muy hacia abajo
    if (dirMirada.y < umbralMirarArriba)
    {
        dirMirada = Vector3.ProjectOnPlane(dirMirada, ↩
            ↪ Vector3.up).normalized;
        if (dirMirada == Vector3.zero) dirMirada = ↩
            ↪ Vector3.forward;
    }

    // Proyeccion del contenedor a una distancia paramétrica
    Vector3 posPista = posCabeza + (dirMirada * ↩
        ↪ distanciaPista);
    posPista.y = Mathf.Max(posPista.y, alturaMinSuelo);

    contenedorPista.position = posPista;
    contenedorPista.rotation = ↩
        ↪ Quaternion.LookRotation(dirMirada);
}
```

5.12.5. Biomecánica y detección del tren inferior

El diseño de la captura de movimiento (*Motion Tracking*) para este segundo módulo terapéutico planteó un desafío sin precedentes en el proyecto. Se requería monitorizar de forma precisa cinco posturas objetivo: reposo, inclinación lateral izquierda, inclinación lateral derecha, extensión completa (cuádriceps tenso) y elevación (flexión de cadera). Para lograrlo utilizando el *hardware* disponible, se estableció el acoplamiento de un único controlador físico mediante cintas elásticas a la cara anterior del muslo dominante del paciente, como se observa en la Figura 5.10, aunque dicho muslo podría variar para permitir entrenar las piernas de manera individual.

Durante los primeros prototipos del *script* `DetectorPiernaVR`, se intentó calcular la postura de la extremidad leyendo directamente los ángulos de Euler del giroscopio interno del mando. Sin embargo, esta técnica fracasó en el entorno de pruebas. Dado que el mando se ata a un paciente recostado en una camilla, la inclinación inicial del dispositivo provocaba un acoplamiento crítico de ejes, un fenómeno matemático conocido como bloqueo del cardán (*Gimbal Lock*). Como consecuencia, el sistema interpretaba erróneamente los movimientos rectos como trayectorias diagonales.

Para solucionar este complejo desafío matemático, se descartó por completo la dependencia exclusiva de las rotaciones locales y se diseñó un modelo analítico basado en la Proyección de Vectores Espaciales. El nuevo algoritmo captura la posición física tridimensional exacta del mando en el espacio absoluto. Al moverse la pierna, el sistema calcula el vector de desplazamiento desde el centro de reposo y lo proyecta sobre los ejes anatómicos reales del paciente utilizando el Producto Escalar (*Dot Product*), tal y como se detalla en el

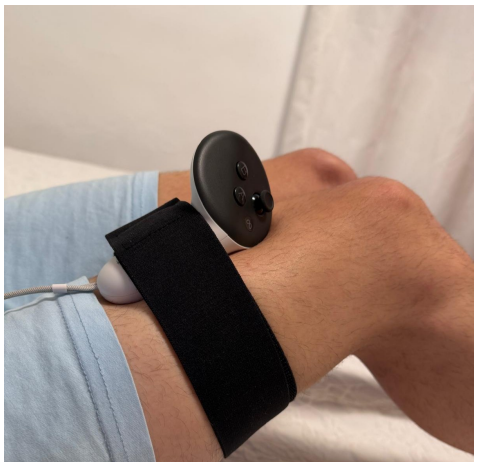


Figura 5.10: Colocación del mando derecho con el uso de la cinta elástica en la cara anterior del muslo, para el juego *Piernas VRítmicas*.

Listado 5.5.

Esta operación matemática traduce el movimiento físico en un movimiento analógico, fluido e intuitivo de un cursor hueco proyectado en la interfaz bidimensional. Para asegurar la máxima fidelidad en la lectura de estos datos y evitar falsos positivos, se implementó una validación física apoyada en la Experiencia de Usuario (UX).

El sistema ha sido programado para registrar el movimiento (cambiando el color del cursor de gris a cian) única y exclusivamente si detecta que el botón lateral de agarre (*Grip*) del mando se encuentra presionado. De esta manera, es la propia tensión de la cinta elástica sobre la pierna la que mantiene pulsado el interruptor, validando que el *hardware* está correctamente acoplado. Si la cinta llegase a aflojarse o el mando se deslizara, el botón se liberaría físicamente y el sistema dejaría de registrar la telemetría errónea al instante.

5.12.6. Calibración dinámica asimétrica y persistencia de seguridad

Debido a que la fisionomía y la movilidad residual varían drásticamente entre pacientes de hemodiálisis, resultó imperativo desarrollar un sistema de calibración paramétrico. Aprovechando que la terapia emplea únicamente el mando dominante sobre la pierna, el paciente o el personal de enfermería puede utilizar el mando secundario libre para invocar un menú oculto de calibración pulsando el botón *X*. Esta manera de calibración sería más tarde cambiada a una calibración mediante el uso del menú de manera obligatoria para comenzar el juego, al apartado de ajustes, si la primera no resultó satisfactoria.

Esta rutina guía al usuario de forma secuencial para registrar las coordenadas y los cuaterniones exactos correspondientes a las cinco posiciones anatómicas clave. Para garantizar la integridad de estos datos, la clase gestora exporta la matriz de posiciones al archivo local `config.json` en el directorio persistente del visor, lo que permitirá en un futuro la extracción de estos perfiles biomecánicos en bruto.

Finalmente, como medida de seguridad arquitectónica, se implementó un constructor de datos personalizado en la clase `PuntoCalibracion`. Este constructor inyecta coordenadas de fábrica (*hardcoded fallbacks*) directamente en el código fuente. Gracias a este respaldo, si ocurriera una pérdida del archivo JSON, un error de lectura, o si se ejecutara una instalación limpia en unas gafas nuevas, la aplicación es capaz de inicializar la cinemática de las piernas sin lanzar excepciones fatales de referencia nula (*NullReferenceException*).

Listado 5.5: Algoritmo de detección biomecánica mediante la proyección vectorial del desplazamiento (Producto Escalar).

```
void CalcularPosturaPierna()
{
    // Vector de desplazamiento desde la postura de reposo
    Vector3 desplazamiento = posActual - calib.reposo.posicion;

    // Proyeccion sobre el eje vertical (elevacion y extension)
    if (desplazamiento.y > 0 && ejeArriba.sqrMagnitude > 0)
        movY = Vector3.Dot(desplazamiento, ejeArriba) / ejeArriba.sqrMagnitude;
    else if (ejeFrente.sqrMagnitude > 0)
        movY = -(Vector3.Dot(desplazamiento, ejeFrente) / ejeFrente.sqrMagnitude);

    // Proyeccion sobre el eje horizontal (inclinacion lateral)
    if (desplazamiento.x > 0 && ejeDerecha.sqrMagnitude > 0)
        movX = Vector3.Dot(desplazamiento, ejeDerecha) / ejeDerecha.sqrMagnitude;
    else if (ejeIzquierda.sqrMagnitude > 0)
        movX = -(Vector3.Dot(desplazamiento, ejeIzquierda) / ejeIzquierda.sqrMagnitude);

    // El resultado traslada el cursor analogico en la interfaz 2D
}
```

Es debido a que estos valores “hardcoded” podrían variar según la postura natural del paciente u otros factores que se impuso la calibración obligatoria por sesión de juego.

5.12.7. Diseño tridimensional y cinemática de las notas

El desarrollo visual de los elementos interactivos se llevó a cabo utilizando el *software* de modelado tridimensional Blender. Para optimizar los recursos del sistema, se diseñaron únicamente dos modelos base fundamentales: una flecha direccional y un modelo circular liso destinado de forma exclusiva a la postura de reposo. Tal y como se ilustra en la Figura 5.11, todas las notas que indican dirección se basan exactamente en el mismo modelo tridimensional de la flecha; el sistema las instancia dinámicamente, las rota sobre su eje Z y les aplica un material distinto según el movimiento requerido (inclinación, extensión o elevación). Para facilitar la asimilación cognitiva del paciente y reducir sus tiempos de reacción, se aplicó una codificación de colores muy contrastada. Este diseño visual permite al jugador asociar de manera mucho más fácil e intuitiva cada exigencia motriz con un color único, reduciendo la carga mental durante la terapia.

Respecto a su comportamiento físico en el entorno virtual, gobernado por el *script* `NotaRitmo`, se programó una cinemática de movimiento no lineal. Al ser instanciadas por el gestor, las notas aparecen al fondo del túnel dinámico ejecutando un *zoom* rápido hacia el jugador. Sin embargo, conforme se acercan al plano espacial de colisión, su velocidad de traslación se reduce de forma drástica. Esta curva de velocidad geométrica proporciona al usuario un margen de lectura visual temprano y un tiempo de reacción ergonómico al llegar a la zona de impacto, mitigando la frustración en pacientes con movilidad reducida.

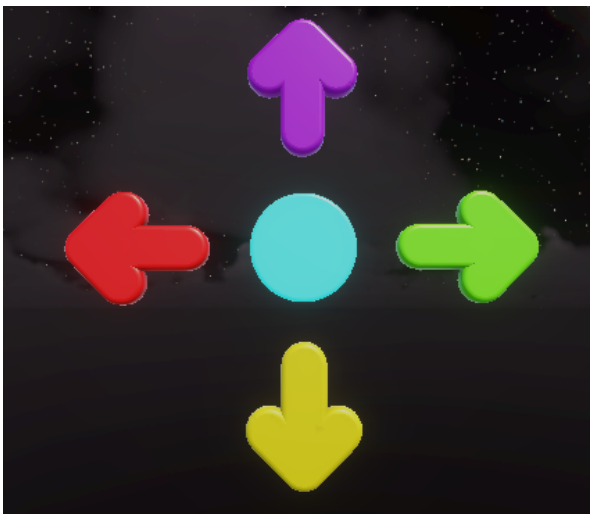


Figura 5.11: Modelos tridimensionales de las notas rítmicas. Se observa la reutilización del modelo de flecha mediante rotación y la codificación de colores asignada a cada postura motriz.

5.12.8. Validación de impactos y retroalimentación visual

Para probar la robustez del algoritmo de detección espacial programado en el `DetectorPiernaVR`, se implementó temporalmente un modo de depuración interno. Mediante la pulsación del botón *X*, el sistema instanciaba una postura aleatoria cada dos segundos, lo que permitió ajustar los radios lógicos de acierto. Las zonas de recepción fijas, donde deben impactar las notas, se construyeron utilizando *Sprites* bidimensionales con materiales emisivos (*Unlit HDR*) en lugar de modelos 3D complejos, una decisión de diseño vital para aligerar la carga poligonal y mantener una tasa de fotogramas por segundo (*FPS*) estable en el visor.

La retroalimentación visual (*feedback*) resultó ser un aspecto crítico para la experiencia de juego. Cuando el paciente acierta una postura en el momento exacto en el que la nota atraviesa el umbral de detección, el *sprite* correspondiente emite un parpadeo lumínico de color verde.

Para flexibilizar la exigencia clínica, se programó un margen de tolerancia espacial o "zona tardía"'. Si el paciente adopta la postura con un ligero retraso de milisegundos, la interacción se contabiliza como "medio acertada" y el panel destella en color amarillo. Por el contrario, si la nota atraviesa toda el área sin que el usuario alcance la postura o mantenga pulsado el botón de validación (*Grip*), el sistema registra un fallo absoluto y el receptor parpadea en color rojo.

5.12.9. Gamificación clínica e interfaz de estado

Para incentivar el esfuerzo físico continuo a lo largo de la sesión terapéutica, se estructuró un sistema de gamificación fuertemente inspirado en títulos comerciales del género rítmico, como *Beat Saber*. Por cada impacto perfecto, el sistema otorga 100 puntos base de recompensa. No obstante, el diseño de la interfaz busca premiar la constancia del movimiento mediante un registro de rachas y un multiplicador paramétrico.

A medida que el paciente encadena aciertos perfectos consecutivos, el multiplicador global de la partida evoluciona escalonadamente de x1 a x2, posteriormente a x4, alcanzando un máximo de x8. Lograr impactar las notas en la zona tardía de tolerancia (color amarillo) otorga la puntuación base para no desmotivar al usuario, pero la lógica del juego dictamina

que ni incrementa la racha actual ni la rompe. Por otro lado, registrar un fallo absoluto (color rojo) penaliza al jugador degradando su multiplicador un nivel de forma inmediata.

Acompañando a estos indicadores numéricos holográficos, se incorporó una barra de vitalidad que fluctúa en tiempo real según el rendimiento. Si el paciente encadena múltiples fallos, la barra se vacía por completo, forzando la conclusión del nivel y el registro de la telemetría, proporcionando así al personal sanitario una métrica visual clara sobre la resistencia límite del usuario.

5.12.10. Herramienta de autoría y diseño de niveles

Para garantizar la escalabilidad del proyecto y agilizar la creación de los escenarios rítmicos, se desarrolló una herramienta de autoría personalizada integrada directamente en el motor gráfico. Mediante el *script* `EditorNivelesRitmo`, el cual hereda de la clase base `EditorWindow`, se programó una ventana gráfica de utilidades nativa para Unity.

Esta interfaz extendida permite al diseñador asignar un archivo de audio, configurar los *beats* por minuto (BPM) de la pista y establecer el desfase temporal inicial (*offset*) del metrónomo. La principal innovación de este módulo es su capacidad de mapeo de notas en tiempo real.

El desarrollador puede reproducir la canción dentro del editor y registrar las notas musicales sobre la marcha utilizando las teclas direccionales y la barra espaciadora del teclado físico. Para asegurar que los impactos tridimensionales coincidan matemáticamente con el compás de la música, se implementó un algoritmo de cuantización, ilustrado en el Listado 5.6. Este sistema intercepta el tiempo físico de la pulsación y lo "autoacopla" redondeándolo a la fracción de ritmo válida más cercana, garantizando un diseño de nivel perfecto y libre de desincronizaciones humanas.

5.12.11. Inteligencia Artificial musical y banda sonora terapéutica

Dado que el proyecto requería pistas musicales instrumentales, con ritmos muy marcados y libres de derechos de autor para su uso académico y de investigación, se recurrió a la Inteligencia Artificial generativa. Utilizando la plataforma *Suno AI*, se sintetizaron seis composiciones originales (una de ellas destinada a la fase de adaptación del tutorial) que conforman la progresión de niveles del juego.

Las instrucciones o *prompts* introducidos en el modelo neuronal se diseñaron meticulosamente para intentar obtener compases estándar de 4/4 con cadencias constantes y atmósferas terapéuticas relajantes. Sin embargo, los actuales modelos de Inteligencia Artificial generativa de audio presentan cierto grado de imprevisibilidad geométrica a la hora de adherirse de manera estricta y determinista a la velocidad (*tempo*) especificada en formato de texto.

Como resultado de este sesgo, se generaron pistas instrumentales que, aunque acertadas en género, abarcan un espectro rítmico que oscila realmente entre los 90 BPM y los 128 BPM. Lejos de ser un inconveniente, estas variaciones empíricas son aprovechadas por el sistema para escalar la dificultad, permitiendo al personal de enfermería adaptar de forma precisa la intensidad del ejercicio cardiovascular a las capacidades individuales de cada paciente.

A continuación, se detallan las instrucciones literales proporcionadas al modelo neuronal, junto con el título final de la composición y la desviación observada respecto a la velocidad real de la pista *a posteriori*[6].

[6]El valor del BPM real de las pistas generadas fue calculado y verificado de forma objetiva extrayendo el

Listado 5.6: Algoritmo de cuantización y grabación de notas en tiempo real para el editor de niveles personalizado.

```
void RegistrarPulsacion(TipoNota notaPulsada, float tiempoAudio)
{
    float tiempoFinal = tiempoAudio;

    // Cuantizacion: auto-acople matematico al ritmo mas cercano
    if (usarCuantizacionActiva)
    {
        tiempoFinal = AcoplarAlRitmoMasCercano(tiempoAudio, bpm);
    }

    // Adicion de la nota a la lista de la dificultad actual
    List<NotaGuardada> listaActual = ObtenerListaActual();
    listaActual.Add(new NotaGuardada
    {
        tiempoAparicion = tiempoFinal,
        tipoNota = notaPulsada
    });

    // Marcado del archivo ScriptableObject para su persistencia
    EditorUtility.SetDirty(cancionActual);
}
```

- **Metronome Milkshake (nivel tutorial):** generado mediante la instrucción `Lofi chillhop, ambient synth, calm and steady groove, 90 BPM, clear bass rum, relaxing, no lead melody, background music for fitness tutorial`. El modelo respetó la velocidad solicitada, resultando en un **BPM real de 90**.
- **Citrus Circuit:** generado mediante la instrucción `[BPM: 105] uplifting synthwave, steady beat, 4/4 time signature, positive, bright synthesizers, retro drum machine, clear rhythm, instrumental`. El sistema entregó una pista ligeramente más lenta, con un **BPM real de 100**.
- **Bamboo Subwoofer:** vreado con el *prompt* `[BPM: 85] lo-fi chillhop, relaxing drum loop, steady rhythm, 4/4 time signature, calming acoustic guitar, warm bass, hopeful atmosphere, instrumental`. La salida presentó una aceleración leve, arrojando un **BPM real de 90**.
- **Sunlit Footsteps:** instruido mediante `[BPM: 115] upbeat dance pop, four on the floor beat, steady rhythm, 4/4 time signature, motivational, bright melodies, clean drums, instrumental`. Resultó en la desviación más acusada de las pruebas, entregando un ritmo de alta intensidad con un **BPM real de 128**.
- **Gutterlight Bloom:** generado con el texto `[BPM: 95] uplifting acoustic folk, steady rhythm, 4/4 time signature, warm acoustic guitar strumming, light clear percussion, positive mood, instrumental`. Entregó un resultado cercano, con un **BPM real de 104**.
- **Cobalt Metronome:** instrucción `[BPM: 100] ambient electronic groove, chillout,`

archivo de audio e introduciéndolo en la herramienta de análisis algorítmico *Vocal Remover Key & BPM Finder*: https://vocalremover.org/key-bpm-finder

`steady deep beat, 4/4 time signature, immersive atmosphere, hopeful, cinematic percussion, instrumental, short song`. El modelo acertó la indicación rítmica, devolviendo un **BPM real de 100**.



Figura 5.12: Vista del menú selector de niveles, con el nombre de la canción, el nombre del artista, la duración y el récord personal del jugador.

5.12.12. Experiencia de usuario final y cumplimiento de la LOPDGDD

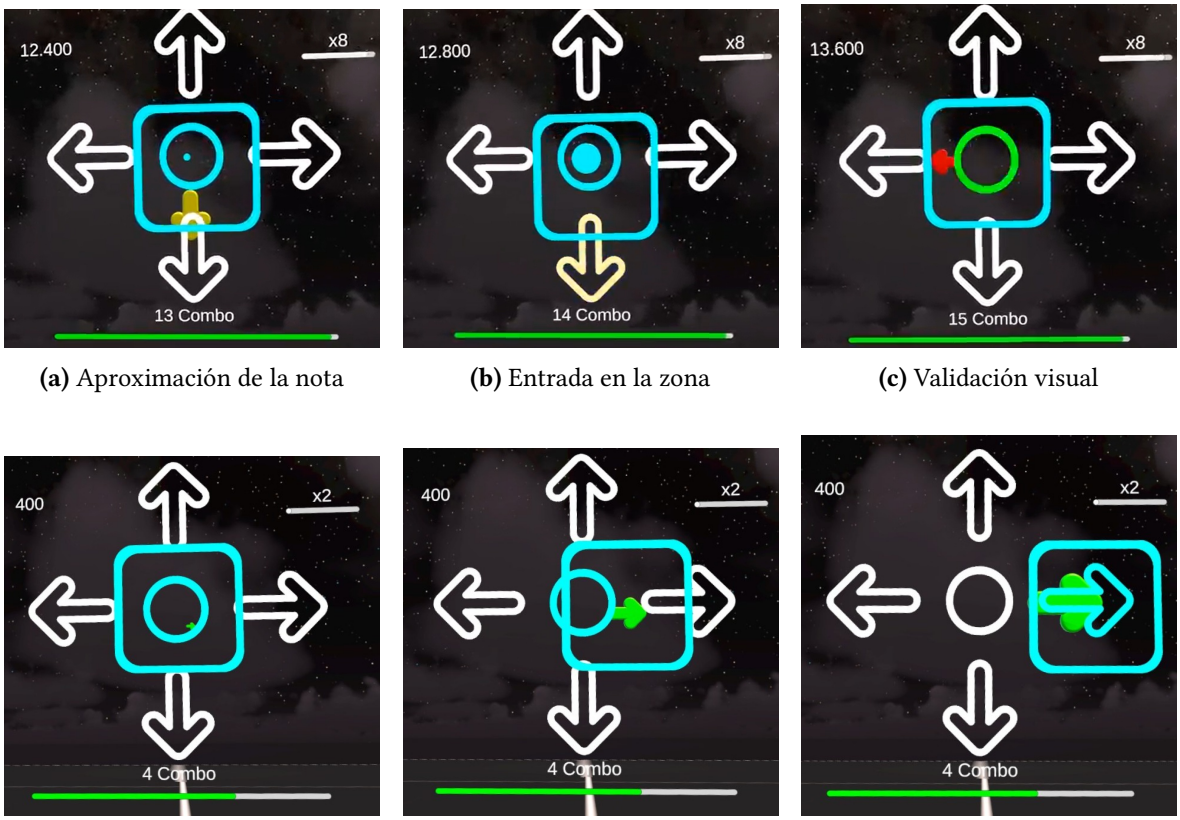
En cuanto al flujo de ejecución del visor, se estableció la calibración biomecánica como un paso estricto y obligatorio antes de iniciar cualquier partida. El sistema despliega una advertencia informativa y fuerza al paciente a realizar el mapeo asimétrico de sus extremidades inferiores. En la interfaz principal previa al juego, se reaprovecharon los paneles modulares de iteraciones anteriores, dotándolos de campos de texto dinámicos que muestran el nombre de la pista generada, la duración total y el récord personal del usuario. Además, se deshabilitaron definitivamente los métodos de instanciación aleatoria usados durante la fase de depuración para entregar un producto final completamente pulido.

Como cierre arquitectónico de esta inmensa iteración, se retomó el código del servidor central (*Backend*) desarrollado en los primeros meses para aplicar una refactorización de seguridad y limpieza. Se eliminaron todas las dependencias lógicas, el manejo de excepciones y las variables que hacían referencia explícita al uso de gafas de RV, logrando un servidor agnóstico que simplemente recibe e ingesta los datos clínicos en la base de datos sin importar qué plataforma los envíe.

Este cambio es consecuencia de la dificultad técnica para dotar de una conexión de red estable a las gafas de RV, por lo que al no poder enviar sus métricas directamente al *Backend*, todo el código de éste planificado para esta idea tuvo que ser desechado.

5.13. ITERACIÓN 8: ECOSISTEMA WEB E INGESTA DE DATOS

Objetivo global de la iteración. Una vez estabilizado el desarrollo de las aplicaciones de RV, el foco del proyecto se trasladó íntegramente a la construcción del *Frontend* web para el personal sanitario utilizando el *framework* Angular. El objetivo principal fue desarrollar una



(a) Aproximación de la nota **(b)** Entrada en la zona **(c)** Validación visual

(d) Postura de reposo **(e)** Inclinación motriz **(f)** Encaje en interfaz

Figura 5.13: Mecánicas del módulo rítmico. En la fila superior, secuencia de aproximación de una nota hacia la zona de colisión. En la fila inferior, desplazamiento del cursor analógico en respuesta a la inclinación física de la pierna del paciente para lograr el encaje.

plataforma integral que permitiera gestionar las identidades, ingerir los historiales clínicos exportados por los visores y visualizar la progresión de los pacientes mediante paneles de métricas interactivos. Para soportar este ecosistema, se llevó a cabo una refactorización profunda de la arquitectura *RESTful* del servidor *Backend*.

5.13.1. Planificación de tareas

Para estructurar el salto arquitectónico hacia el ecosistema *Frontend* y garantizar una integración segura con el servidor, las necesidades de gestión del personal sanitario se tradujeron en un conjunto de requerimientos web y de base de datos. En la Tabla 5.8 se expone la planificación ejecutada durante esta iteración, destacando la construcción de los cuadros de mando interactivos y la severa refactorización polimórfica necesaria para soportar la ingesta masiva de archivos.

ID	Descripción de la tarea	Prioridad	Estado
FRONT-01	Desarrollo del panel de administración (roles y accesos).	Alta	Completada
FRONT-02	Módulo de privacidad y cumplimiento estricto de la LOPDGDD.	Alta	Completada
FRONT-03	Flujo de selección clínica y alta rápida de pacientes con seudónimos.	Media	Completada
FRONT-04	Interfaz de ingesta de datos (Upload) por *Drag & Drop*.	Alta	Completada
BACK-01	Refactorización de BD (herencia JOINED) y *endpoints*.	Alta	Completada
FRONT-05	Dashboard de métricas interactivas con Chart.js y filtrado.	Alta	Completada

Tabla 5.8: Tareas planificadas y completadas durante el desarrollo del ecosistema web y la refactorización del servidor.

5.13.2. Panel de administración y seguridad activa JWT

El desarrollo de la plataforma web arrancó con la implementación de la capa más restrictiva del sistema: el panel de administración. Cumpliendo estrictamente con los requisitos de seguridad trazados en la fase de análisis, esta vista fue protegida mediante guardias de rutas (*Route Guards*) en Angular, limitando su acceso única y exclusivamente a los usuarios que posean el rol *ADMIN*. La Figura 5.14 muestra la interfaz de gestión de personal desarrollada.

Desde este centro de control, el administrador dispone de un formulario unificado para dar de alta a nuevos enfermeros en el hospital, modificar sus credenciales o correo electrónico, alterar sus privilegios de gestión y, finalmente, darlos de baja del ecosistema.

Para evitar brechas de seguridad y conexiones fantasma, se implementó una medida de seguridad activa en la lógica del *Backend*. El filtro `JwtAuthenticationFilter` de Spring Security fue reprogramado para consultar la base de datos MySQL en tiempo real con cada petición. De este modo, si a un enfermero activo se le retiran los permisos o se elimina su cuenta desde el panel, su *token* JWT pierde su validez de forma instantánea. Esta respuesta

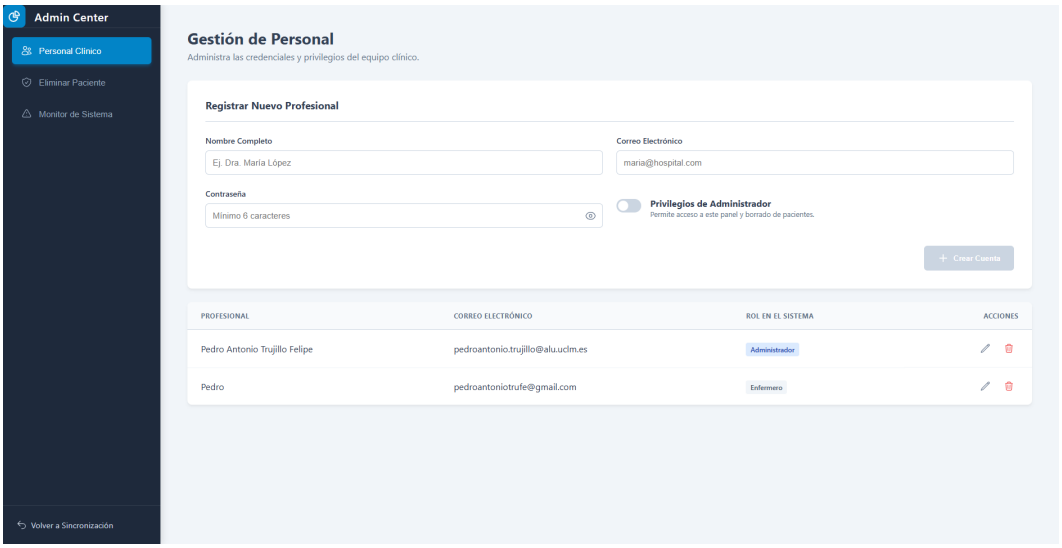


Figura 5.14: Interfaz del panel de administración en Angular para la gestión de enfermeros y privilegios de acceso.

del servidor es interceptada por Angular, forzando un cierre de sesión inmediato en el cliente del enfermero revocado.

5.13.3. Privacidad clínica y derecho al olvido

Para dar cumplimiento al Requisito Funcional 12 ("Eliminar registro de paciente") y garantizar el respeto absoluto por la Ley Orgánica de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD), se integró una subsección específica denominada "Privacidad y LOPDGDD" dentro del panel de administración, ilustrada en la Figura 5.15.

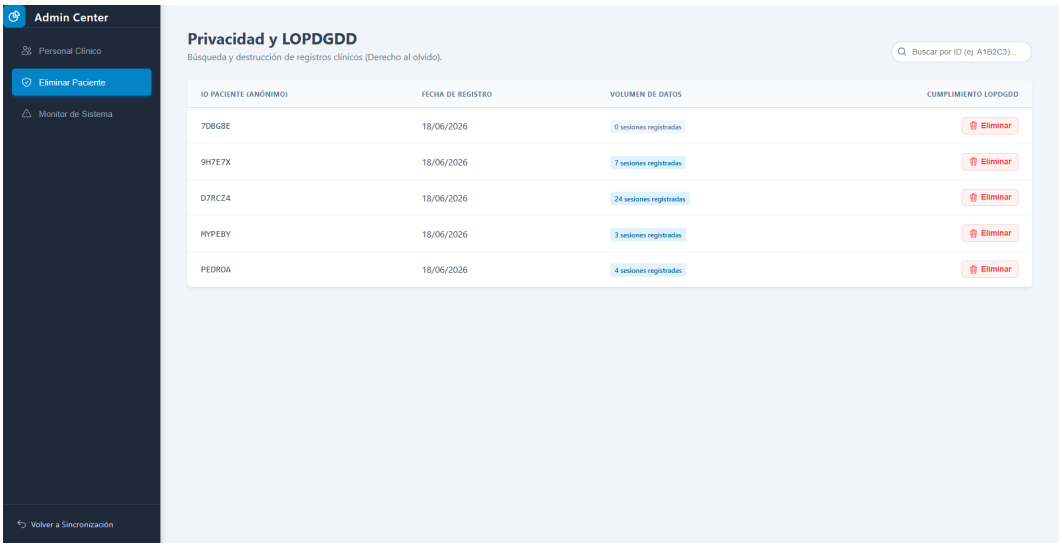


Figura 5.15: Módulo de privacidad y LOPDGDD, mostrando la tabla dinámica de expedientes seudonimizados.

Esta interfaz despliega una tabla de datos dinámica que renderiza la totalidad de los pacientes registrados en el servidor. Preservando la seudonimización clínica, la tabla identifica a los usuarios exclusivamente mediante su ID alfanumérico de seis caracteres, exponiendo además la fecha de creación del expediente y el volumen total de sesiones terapéuticas

asociadas a dicho perfil. Para facilitar la gestión en grandes volúmenes de datos, se programó un motor de búsqueda que filtra las filas en tiempo real.

El aspecto más crítico de este módulo es la gestión del borrado. Al pulsar el botón de eliminación de un expediente, el flujo del programa se interrumpe desplegando un modal de advertencia personalizado. Esta ventana exige una confirmación explícita, advirtiendo al administrador de que la acción provocará una destrucción en cascada (*On Delete Cascade* en base de datos) e irreversible de todo el historial médico del paciente y de los archivos físicos de telemetría alojados en el servidor, garantizando de forma fehaciente el "derecho al olvido" del usuario.

5.13.4. Flujo clínico y gestión de pacientes

Superada la capa de seguridad, el flujo de trabajo principal para el personal de enfermería comienza en la vista central o *Home*. Al iniciar sesión, el usuario es redirigido automáticamente a una pantalla de Selección Clínica. Esta interfaz lista todos los pacientes activos ordenados cronológicamente según su fecha de alta en el sistema, mostrando en cada tarjeta un resumen rápido del volumen de sesiones terapéuticas que han completado. En la Figura 5.16 se ilustra el diseño limpio y accesible de este panel de selección.

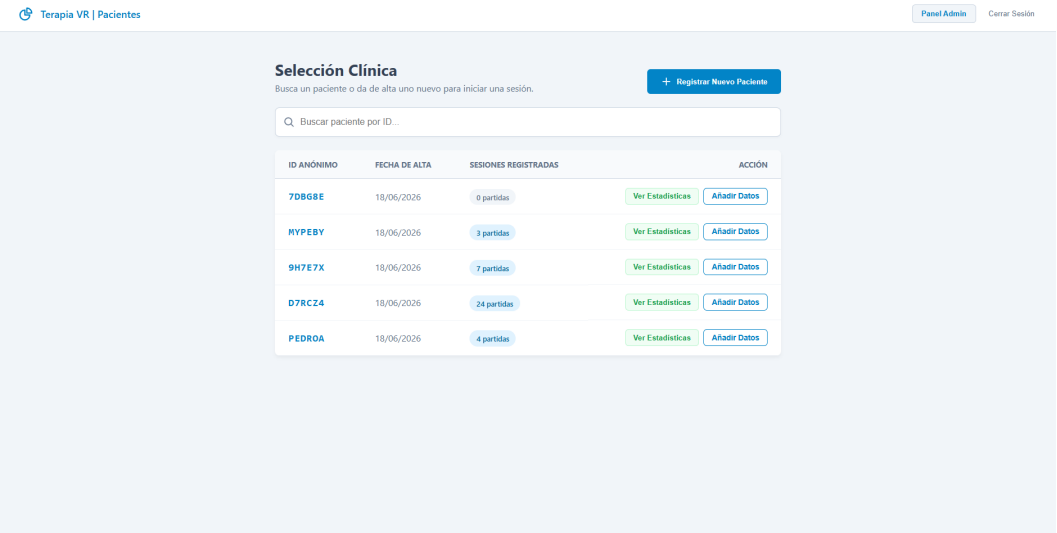


Figura 5.16: Vista principal del personal clínico, mostrando el listado de expedientes y la opción de nuevo registro.

Para dar cobertura al ingreso de nuevos usuarios, se implementó un botón de acción rápida denominado "registrar nuevo paciente". Al ser accionado, el sistema delega la lógica de negocio al servidor, el cual genera y consolida en la base de datos un identificador aleatorio, único y compatible con la LOPDGDD. Inmediatamente después, el *Frontend* captura la respuesta y presenta este nuevo ID al enfermero mediante una tarjeta visual destacada. Este flujo garantiza que el profesional conozca exactamente qué código alfanumérico debe introducir en las gafas Meta Quest antes de iniciar la terapia con el paciente.

5.13.5. Ingesta de datos asíncrona y *Drag & Drop*

Uno de los pilares funcionales de la plataforma web es la centralización de los datos exportados por el visor. Para resolver esta necesidad se construyó la vista de Ingesta de Datos Clínicos (*Upload*). Con el objetivo de minimizar los errores humanos durante la subida

manual de archivos, la interfaz se estructuró mediante un sistema de cuadrícula (*Grid*) que delimita tres zonas de *Drag & Drop* independientes. Estas áreas están destinadas a recibir, respectivamente, el historial de partidas (CSV generales), la telemetría o *tracking* (CSV espaciales) y la configuración (JSON de ajustes del jugador), tal y como se observa en la Figura 5.17.

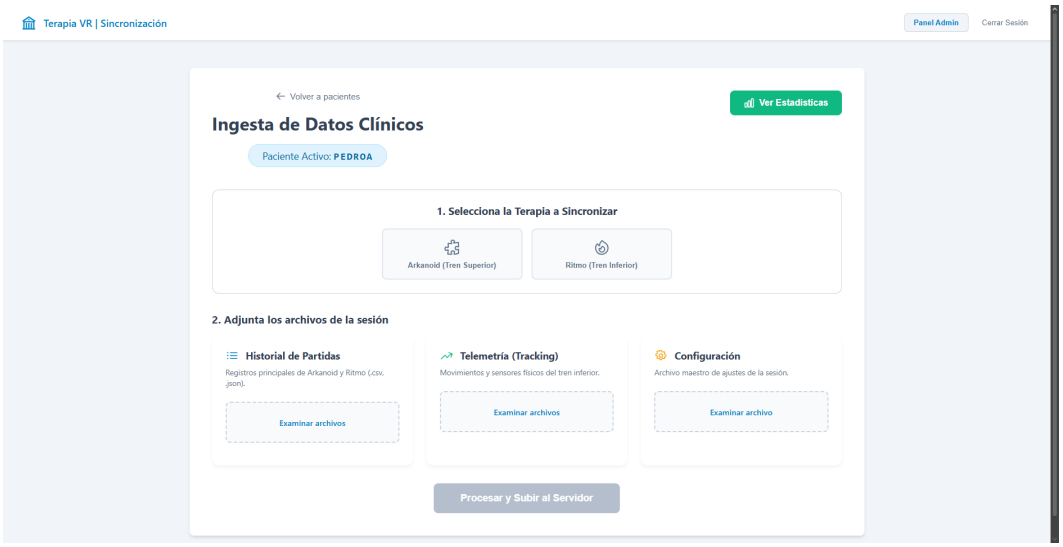


Figura 5.17: Interfaz de subida de archivos mediante zonas de arrastre (*Drag & Drop*) segmentadas por tipo de dato.

Debido a que el ecosistema cuenta ahora con dos terapias distintas (*Arkanoid* y Piernas VRítmicas), se requirió implementar un selector explícito de terapia en la interfaz. El enfermero debe marcar un botón de selección (*Radio Button*) indicando a qué juego pertenecen los archivos arrastrados. Al confirmar la subida, Angular empaqueta los documentos utilizando la interfaz `FormData`, inyectando dinámicamente la variable `tipoJuego` como parámetro adjunto.

Dado el gran volumen de datos a procesar, la aplicación cliente hace uso de la librería reactiva *RxJS*. Empleando el operador `forkJoin`, el *Frontend* dispara las tres peticiones HTTP (*historial*, *telemetría* y *configuración*) de forma rigurosamente paralela.

Durante las pruebas de integración de esta subida múltiple, se detectó un problema de desincronización visual en Angular provocado por la resolución simultánea de las peticiones asíncronas, lo que impedía que la barra de progreso se actualizara correctamente. Para solventar este defecto de Experiencia de Usuario (UX), se inyectó la clase `ChangeDetectorRef` en el componente, forzando un refresco explícito del modelo de objetos del documento para mostrar el *feedback* de éxito de manera instantánea.

5.13.6. Refactorización del servidor y herencia polimórfica

Para soportar esta ingesta masiva, la arquitectura *RESTful* del *Backend* en Spring Boot fue sometida a una refactorización severa. Se abrieron tres puntos de enlace (*endpoints*) dedicados y se reconfiguró el servidor embebido Tomcat, ampliando el límite de recepción de archivos a 50 Megabytes para garantizar el soporte de los posibles densos documentos de telemetría física.

Una vez que los archivos alcanzan el servidor, el servicio `DataIngestionService`, apoyado en la librería *Apache Commons CSV*, asume el control. Este servicio lee el parámetro

`tipoJuego` del *FormData* para dirigir los archivos físicos hacia su directorio correspondiente (`storage/idPaciente/tipoJuego/`). A continuación, examina los historiales, clasifica las métricas y las inserta de manera persistente en las tablas de MySQL, aplicando una lógica de validación que descarta automáticamente cualquier registro duplicado.

Para garantizar la integridad referencial entre el registro estructurado de la base de datos relacional y los masivos archivos físicos de seguimiento guardados en el disco del servidor, se aplicó un diseño orientado a objetos basado en la herencia. Utilizando la anotación `@Inheritance` con la estrategia `JOINED` de *Hibernate*, se creó una entidad abstracta padre denominada `Partida`.

Esta entidad raíz agrupa los atributos universales (identificador del paciente, fecha, duración y la ruta física del archivo asociado). De ella heredan las entidades hijas específicas de cada juego, las cuales almacenan métricas únicas incorporadas en las últimas iteraciones de Unity. Para el tren superior se recogen los impactos segmentados para calcular el Índice de Simetría, mientras que para el tren inferior se persisten la puntuación, la racha máxima y la etiqueta de victoria o derrota. El nexo de unión inquebrantable que enlaza estas tablas SQL con los CSV en crudo es la nueva clave candidata `ID_Sesion`, un valor inmutable forjado a partir de la marca de tiempo exacta (*Timestamp*) en la que las gafas iniciaron la partida.

5.13.7. Expediente clínico y visualización interactiva de métricas

Como culminación del Producto Mínimo Viable (PMV) de la plataforma web, se desarrolló el módulo de "Expediente Clínico". Esta interfaz de analítica avanzada es accesible de forma directa desde el perfil individual de cada paciente. Su propósito fundamental es traducir el inmenso volumen de telemetría y datos crudos ingeridos por el servidor en información visual de alto valor diagnóstico para el personal de nefrología y rehabilitación.

Para nutrir esta vista con información consolidada, se desarrollaron nuevos *endpoints* RESTful en el servidor (tales como `/stats/arkanoid/{id}` y `/stats/ritmo/{id}`). Estas rutas están optimizadas para recuperar el historial médico del paciente ordenado cronológicamente desde la base de datos. En la capa de presentación de Angular, se integró la librería especializada *Chart.js*, permitiendo la renderización de un cuadro de mandos (*Dashboard*) interactivo y responsivo, tal y como se ilustra en la Figura 5.18.

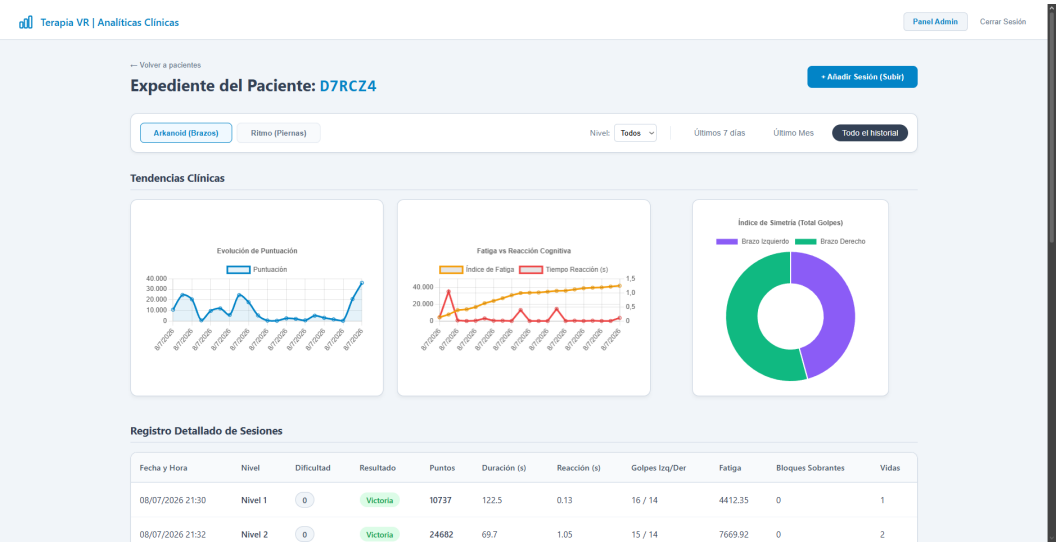


Figura 5.18: Panel interactivo del expediente clínico, mostrando las gráficas de evolución generadas mediante Chart.js y la tabla de datos condicional.

El diseño de este cuadro de mandos se estructuró empleando múltiples representaciones gráficas, adaptando cada formato a la naturaleza matemática de la métrica a analizar. Se implementaron gráficas de líneas estándar para el seguimiento visual de la precisión rítmica y la evolución de las puntuaciones. Para estudios más complejos, se diseñaron gráficas de doble eje (*Dual-Axis Charts*) que permiten contrastar de forma simultánea la curva del índice de fatiga muscular frente a la degradación del tiempo de reacción cognitivo-motor. Asimismo, se incluyeron gráficas de tipo rosquilla (*Doughnut*) enfocadas en ilustrar instantáneamente el índice de asimetría bimanual del paciente.

Toda esta representación gráfica superior se ve respaldada en la parte inferior de la vista por un registro histórico detallado en formato de tabla interactiva. Para optimizar la legibilidad y reducir la carga cognitiva del personal médico durante las rondas hospitalarias, la tabla incorpora un diseño de renderizado condicional. Mediante el uso de etiquetas de color (*badges*), el sistema resalta automáticamente el estado de los resultados de cada sesión, diferenciando de un vistazo las victorias, las derrotas o las configuraciones de dificultad extremas.

Finalmente, para otorgar un rigor científico y analítico a estas estadísticas, se programó un sistema de filtrado cruzado bidireccional. Por un lado, el personal médico puede acotar las métricas aplicando ventanas temporales predefinidas (últimos 7 días, últimos 30 días o el historial completo). Por otro lado, este delimitador de tiempo se combina dinámicamente con un filtrado contextual que extrae los niveles o canciones exactas que ha completado el usuario. La intersección de ambos filtros permite a los especialistas evaluar la progresión real del paciente bajo un nivel de exigencia, estímulo y dificultad estrictamente idénticos a lo largo de las semanas.

5.14. ITERACIÓN 9: EXPERIMENTACIÓN, PRUEBAS Y VALIDACIÓN DEL SISTEMA

Objetivo global de la iteración. Las semanas finales del desarrollo (semanas 15 a 17) se dedicaron a la experimentación técnica y a la validación clínica del ecosistema completo. El objetivo fue certificar la ergonomía del sistema, evaluar su adaptación a las severas restricciones físicas del tratamiento y constatar el potencial de la gamificación para mejorar el estado de ánimo de los pacientes, culminando con la validación oficial por parte del personal sanitario del hospital.

5.14.1. Adaptación clínica y pruebas de viabilidad

La fase de validación se alejó de las tradicionales pruebas de estrés de *software* para centrarse en la viabilidad clínica y psicológica del proyecto. El reto principal consistía en garantizar que los "juegos serios" desarrollados fueran plenamente compatibles con las limitaciones del tratamiento de hemodiálisis, donde la necesidad de mantener el catéter de filtrado exige la inmovilización parcial de un brazo y prohíbe la realización de movimientos bruscos o exagerados.

Paralelamente, se buscaba evaluar si los entornos virtuales resultaban lo suficientemente atractivos como para romper la monotonía de las sesiones clínicas, buscando incidir de forma positiva en el estado emocional y la calidad de vida del paciente. Para consolidar un volumen de datos estadísticamente relevante antes de la evaluación médica, se llevó a cabo una fase de experimentación con usuarios sanos. Un grupo de control conformado por familiares, amistades y el propio desarrollador completó múltiples sesiones de ambas terapias. Esta recolección permitió afinar la sensibilidad paramétrica del código y verificar la correcta inserción de expedientes masivos en la base de datos MySQL.

5.14.2. Pruebas en el entorno hospitalario

El hito más trascendental de esta iteración tuvo lugar el 1 de julio de 2026 en las instalaciones del Hospital General Universitario de Ciudad Real (HGUCR). Se celebró una sesión de experimentación oficial orientada a validar la usabilidad, la funcionalidad y la seguridad del sistema para los pacientes en el contexto físico de una sala de diálisis real.

Siguiendo estrictas directrices éticas y metodológicas, las pruebas prácticas de la jornada fueron ejecutadas exclusivamente por dos miembros especialistas del personal clínico del centro, quienes aportaron su conocimiento para orientar la solución hacia las necesidades reales. En la Figura 5.19 se documentan diversos instantes de esta sesión de validación en las instalaciones del HGUCR.



Figura 5.19: Sesión de experimentación clínica y validación ergonómica llevada a cabo en las instalaciones del Hospital General Universitario de Ciudad Real.

Durante la jornada, los especialistas médicos testearon de forma exhaustiva la adaptación del "túnel dinámico" al ángulo de inclinación de las camillas y evaluaron la precisión de la calibración asimétrica de los brazos. Asimismo, sometieron a prueba el juego de ritmo del tren inferior, verificando que el ejercicio promovido resultaba controlado, moderado y plenamente compatible con las limitaciones del tratamiento.

5.14.3. Certificación institucional y apertura científica

La conclusión obtenida tras estas pruebas de estrés ergonómico fue sumamente positiva: el sistema resulta usable por parte de los pacientes y seguro para su uso en el contexto de la hemodiálisis. Para certificar formalmente este éxito, el proyecto cuenta con un Informe de Colaboración y Validación emitido y firmado por D. Rubén José Bernal Celestino, Supervisor del Área de Investigación, Docencia y Formación del HGUCR. Dicho documento, incluido de manera íntegra en el Anexo A de este Trabajo Fin de Grado, manifiesta la disposición del

hospital a continuar colaborando en esta línea tecnológica para celebrar nuevas sesiones analíticas en próximas fechas.

5.15. RECOPILACIÓN, PUBLICACIÓN Y ESTRUCTURA DE DATOS BIOMÉTRICOS

En un firme compromiso con la transparencia metodológica y los principios de la Ciencia Abierta (*Open Science*), la totalidad de los datos biométricos y de uso recolectados durante la fase de experimentación han sido empaquetados en conjuntos de datos (*datasets*). Estos registros, seudonimizados, han sido depositados en el repositorio público Zenodo, dotando al proyecto de un DOI (*Digital Object Identifier*) que permite a futuros investigadores auditar, analizar y expandir los descubrimientos de este ecosistema.

La información se ha estructurado separando los metadatos de consolidación clínica de las matrices de seguimiento continuo a alta frecuencia. Es necesario recalcar que los datos que podrían ser de mayor utilidad para una investigación sobre los beneficios del ejercicio durante las sesiones de hemodiálisis no pueden ser obtenidos, por lo que solo se miden datos que se pueden obtener desde las gafas de RV.

5.15.1. Módulo de tren superior (*ArkanoidVR*)

Para el ejercicio inmersivo del tren superior, basado en la mecánica de intercepción de esferas y destrucción de bloques, la recolección de datos se categoriza según su naturaleza clínica y funcional, abarcando los siguientes tipos de información:

- **Configuración ergonómica y espacial:** se recogen las preferencias base con las que el paciente afronta la sesión, tales como la `dificultad`, el `volumen` de audio y el `modoMando` (mono-manual o bi-manual). Adicionalmente, se registra la geometría exacta de la interfaz adaptada a la camilla de diálisis, incluyendo la `inclinacionPantallaX`, la activación de la `pantallaCurva` y las diversas métricas paramétricas de renderizado (`tamanoMenu`, `distanciaPlana`, `distanciaCurva`).
- **Calibración biomecánica asimétrica (ROM):** antes de iniciar el juego, el sistema captura el Rango de Movimiento máximo seguro para el paciente, esencial para evitar lesiones en el brazo que porta la vía venosa. Estos datos espaciales se definen de manera independiente para cada extremidad mediante sus coordenadas centrales (`centroX_L`, `centroX_R`) y sus respectivos límites de alcance lateral continuo (`alcanceIzqX_L`, `alcanceDerX_L` para el brazo izquierdo, y sus homólogos para el derecho).
- **Métricas clínicas y de esfuerzo:** este conjunto de datos consolida el impacto físico del ejercicio. Cuantifica de forma aislada el uso de cada extremidad mediante el registro de impactos realizados (`Golpes_IZQ`, `Golpes_DER`) para evaluar la simetría del esfuerzo. Asimismo, se extrae el `IndiceFatiga` acumulado durante toda la sesión (calculado a través del temblor o *jitter*) y el tiempo de `ReaccionMedia(s)` cognitivo-motor frente a los estímulos visuales descendentes.
- **Rendimiento lúdico y progresión:** engloba los resultados inherentes a la partida, fundamentales para la gamificación y la adherencia al tratamiento. Incluye el `Nivel` jugado, la `Duracion(s)` total del ejercicio en segundos, la `Puntuacion` final alcanzada, las `VidasRestantes`, los `BloquesRestantes` al finalizar, y la resolución de la terapia expresada como `Resultado` (victoria o derrota).
- **Cinemática espacial continua (*Tracking*):** constituye la telemetría en crudo muestreada a una frecuencia de 10 Hz a lo largo de toda la sesión, estructurada mediante una marca de `Tiempo(s)` absoluta. Documenta la rotación tridimensional de la cabeza

del paciente (`Head_RotX`, `Y`, `Z`) y captura simultáneamente la posición exacta (`PosX`, `PosY`, `PosZ`) y la velocidad lineal (`Vel(m/s)`) de la mano izquierda (`L_`) y derecha (`R_`). Toda esta matriz física se complementa con un registro de `Evento` que indexa textualmente las colisiones y acciones críticas en su instante exacto de ocurrencia.

5.15.2. Módulo de tren inferior (*Piernas VRítmicas*)

Para la terapia del tren inferior, centrada en la adopción de posturas isométricas y movimientos de las piernas en sincronía con patrones musicales, la información recolectada se clasifica en los siguientes tipos de datos biomecánicos y de rendimiento:

- **Calibración biomecánica absoluta (posturas 3D):** constituye el perfil físico inmutable del paciente para la sesión. El sistema registra las coordenadas tridimensionales de `posicion` (x, y, z) y los cuaterniones de `rotacion` geométrica (x, y, z, w) para las cinco posturas anatómicas clave exigidas durante la terapia: `reposo`, inclinación `izquierda`, inclinación `derecha`, pierna `extendida` y pierna `levantada`. Estos vectores exactos garantizan que la exigencia del juego se acople milimétricamente a la fisionomía y movilidad real del usuario.
- **Métricas clínicas y rendimiento rítmico:** este conjunto evalúa la capacidad de coordinación neuromotora y la tolerancia cardiovascular. Cuantifica la exactitud de los movimientos a través de la `Precision(%)` global de la partida y registra la `RachaMaxima` de posturas acertadas de forma consecutiva (un indicador clave de la concentración y el control motor). Al igual que en el módulo superior, se calcula y persiste un `IndiceFatiga` acumulativo para medir el desgaste a lo largo de la canción.
- **Resultados lúdicos de la sesión:** agrupa los datos descriptivos de la partida para facilitar su catalogación y gamificación. Incluye la pista musical seleccionada (`Cancion`), el nivel de `Dificultad` paramétrica, la `Duracion(s)` en segundos, la `Puntuacion` total obtenida y el `Resultado` clínico de la sesión. Adicionalmente, se registran variables de entorno básicas como el `volumen` de audio preferido por el paciente.
- **Cinemática espacial continua (*Tracking*):** representa la telemetría de alta frecuencia (10 Hz) que documenta el movimiento continuo del paciente, estructurada mediante una marca de `Tiempo(s)` absoluta. A diferencia del tren superior, este módulo captura simultáneamente la posición y rotación tanto de la cabeza (`Head_PosX`, `Y`, `Z` y `Head_RotX`, `Y`, `Z`) como de la extremidad inferior monitorizada (`Leg_PosX`, `Y`, `Z` y `Leg_RotX`, `Y`, `Z`). Este nivel de detalle, sumado a la velocidad lineal de la pierna (`Leg_Vel(m/s)`) y a la columna `Evento` (que marca los instantes de acierto o fallo rítmico), permite a los especialistas reconstruir la sesión tridimensionalmente para auditar la fluidez y el rango articular del paciente.

5.15.3. Acceso a los conjuntos de datos (*Datasets*)

Con el propósito de facilitar la consulta, auditoría y descarga de toda la telemetría biomecánica recopilada, los archivos de datos en crudo y los metadatos de consolidación clínica se encuentran alojados de forma pública y permanente en la plataforma científica Zenodo.

A continuación, se proporcionan los códigos de respuesta rápida (QR) y los enlaces directos (*Digital Object Identifier* - DOI) correspondientes a los repositorios independientes de cada módulo terapéutico:

Dataset *ArkanoidVR*
https://zenodo.org/records/21286393

Dataset *Piernas VRítmicas*
https://zenodo.org/records/21287031

Figura 5.20: Códigos QR y enlaces directos para el acceso a los *datasets* clínicos alojados en Zenodo.

5.16. ACCESO A LOS REPOSITORIOS DEL PROYECTO

En consonancia con el compromiso de transparencia académica y ciencia abierta establecido durante la validación clínica del proyecto, la totalidad del código fuente desarrollado en este Trabajo de Fin de Grado ha sido liberado bajo la licencia de *software* libre MIT.

Para facilitar la labor de auditoría técnica por parte del tribunal evaluador y permitir la futura iteración de estas herramientas por la comunidad investigadora, el ecosistema de *software* se ha segmentado en cuatro repositorios independientes. A continuación, se facilitan los códigos de respuesta rápida (QR) que enlazan de manera directa a cada uno de los módulos alojados en la plataforma GitHub:

1. Servidor Central (Backend)
Arquitectura RESTful en Spring Boot.
https://github.com/Pedro-Antonio-Tr/backend-tfg-pedroantonio

2. Plataforma Clínica (Frontend)
Cliente web desarrollado en Angular.
https://github.com/Pedro-Antonio-Tr/frontend-tfg-pedroantonio

3. VR Tren Superior (Arkanoid)
Proyecto en Unity para el juego terapéutico *Arkanoid*. https://github.com/Pedro-Antonio-Tr/ArkanoidVR-TFG

4. VR Tren Inferior (Piernas VRítmicas)
Proyecto en Unity del módulo de terapia rítmica. https://github.com/Pedro-Antonio-Tr/Piernas-VRitmicas-TFG

Figura 5.21: Códigos QR para el acceso directo a los repositorios de código del ecosistema clínico.

CAPÍTULO 6
Conclusiones

6.1. OBJETIVOS ALCANZADOS

Tras la finalización del periodo de desarrollo e implementación técnica documentado en los capítulos anteriores, es posible afirmar que el **objetivo general** del presente Trabajo de Fin de Grado ha sido superado con éxito. Se ha logrado diseñar, construir y validar un ecosistema de *software* clínico funcional, compuesto por dos aplicaciones inmersivas de Realidad Virtual orientadas al entrenamiento ligero de los miembros superiores e inferiores de los pacientes de hemodiálisis y una plataforma web segura para la recolección de telemetría.

De manera específica, el grado de cumplimiento de los objetivos secundarios definidos en la fase de análisis, y su correspondencia con el ciclo de vida del proyecto, se desglosa a continuación:

- **Análisis y adaptación a restricciones clínicas:** logrado progresivamente desde la toma de requisitos (iteración 0) hasta la validación hospitalaria (iteración 9). Se han superado las barreras ergonómicas impuestas por las camillas de diálisis mediante el desarrollo del sistema de "Túnel Dinámico", el recentrado espacial y los modos de juego accesibles a una sola mano.
- **Desarrollo del módulo inmersivo de relajación:** abordado durante las fases de prototipado inmersivo (iteraciones 3 y 7). Se ha integrado con éxito una sala de espera forestal (*Arkanoid*) y otra de un cielo estrellado (*Piernas VRítmicas*) con efectos visuales y sonoros relajantes, los cuales aíslan al paciente de la estética hospitalaria durante los instantes de mayor estrés médico, como la canalización de la vía.
- **Diseño e implementación del juego serio para el tren superior:** desarrollado y pulido a lo largo de las iteraciones 3 a 6. El módulo interactivo basado en *Arkanoid* es plenamente funcional. Cuenta con físicas de rebote personalizadas, un sistema de dificultad paramétrico dinámico y una herramienta de calibración asimétrica en tiempo real para el rango de movimiento (ROM).
- **Diseño e implementación del juego serio para el tren inferior:** alcanzado de forma íntegra durante la iteración 7. Se superaron los escollos matemáticos del rastreo (*Gimbal Lock*) mediante un algoritmo de producto escalar aplicado al controlador de la pierna. El juego de ritmo terapéutico está completado, incorporando un creador de niveles propio y piezas musicales generadas proceduralmente mediante Inteligencia Artificial.
- **Desarrollo del núcleo de telemetría y persistencia local de datos:** implementado en la iteración 3 y consolidado en la iteración 5. El `MonitorClinico` opera de forma autónoma e invisible, persistiendo el índice de fatiga, la lateralidad, la biometría en

crudo a 10 Hz y los tiempos de reacción en archivos inmutables (JSON y CSV) alojados en el almacenamiento seguro de las gafas Meta Quest 3, mitigando así los posibles cortes de red del hospital.

- **Desarrollo de la plataforma web de gestión clínica e integración de privacidad:** iniciado con la creación y securización del servidor (iteraciones 1 y 2) y culminado con la construcción del *Frontend* (iteración 8). El desarrollo *Full-Stack* garantiza la subida asíncrona de datos, la visualización de *dashboards* analíticos y la seudonimización (mediante IDs de 6 caracteres generados algorítmicamente), respetando escrupulosamente el principio de privacidad desde el diseño (LOPDGDD).
- **Experimentación y validación del sistema en entorno clínico:** materializado en la iteración 9. Las pruebas *in situ* junto al personal sanitario en el Hospital General Universitario de Ciudad Real (HGUCR) certificaron la viabilidad clínica, ergonomía y seguridad de todo el ecosistema tecnológico desarrollado.

6.2. JUSTIFICACIÓN DE COMPETENCIAS ADQUIRIDAS

Durante la elaboración integral de este proyecto, se han puesto en práctica conocimientos transversales y avanzados asimilados a lo largo del Grado en Ingeniería Informática. En cumplimiento con la normativa académica de la ESI-UCLM, en este Trabajo de Fin de Grado se han aplicado y demostrado de forma práctica las competencias correspondientes a la Tecnología Específica de **Tecnologías de la Información (TI)**:

- **TI1:** *Capacidad para comprender el entorno de una organización y sus necesidades en el ámbito de las TIC.*
  Se ha aplicado mediante el estudio exhaustivo del entorno clínico del Hospital General Universitario de Ciudad Real (HGUCR). Esto implicó traducir los requerimientos médicos de la sala de nefrología en especificaciones técnicas de *software*. Al comprender las severas limitaciones operativas de la organización, tales como la imposibilidad de depender de una conexión a internet estable y la estricta confidencialidad exigida por el marco legal, se logró concebir una arquitectura *Offline-First* seudonimizada que responde con exactitud a las necesidades reales del personal sanitario.
- **TI2:** *Capacidad para seleccionar, diseñar, desplegar, integrar, evaluar, construir, gestionar, explotar y mantener las tecnologías de hardware, software y redes.*
  Se justifica a través de la toma de decisiones fundamentada en la selección del ecosistema tecnológico y la posterior construcción de un sistema *Full-Stack* distribuido. Esto abarca desde la explotación del *hardware* inmersivo autónomo (gestión de los sensores espaciales de las gafas Meta Quest 3 y optimización del motor Unity), hasta el despliegue del servidor central en Spring Boot, el modelado relacional en MySQL para la persistencia masiva de telemetría y el ensamblaje de la lógica de cliente en Angular.
- **TI3:** *Capacidad para emplear metodologías centradas en el usuario y la organización para el desarrollo, evaluación y gestión de aplicaciones y sistemas basados en tecnologías de la información que aseguren la accesibilidad, ergonomía y usabilidad de los sistemas.*
  Ha sido demostrada empíricamente al subordinar todo el diseño de los "juegos serios" a las capacidades físicas del paciente de hemodiálisis. Se garantizó la ergonomía clínica mediante el "Túnel Dinámico", que adapta la interfaz a la inclinación de la camilla, y la programación del algoritmo de calibración asimétrica del Rango de Movimiento (ROM) para compensar la inmovilidad parcial. Del mismo modo, la usabilidad se aseguró previniendo la cinetosis mediante interfaces holográficas adaptativas (*Lazy HUD*) y

dotando a los enfermeros de un panel web accesible para la ingesta ágil de ficheros mediante *Drag & Drop*.

- **TI6:** *Capacidad para concebir sistemas, aplicaciones y servicios basados en tecnologías de red, incluyendo Internet, web, comercio electrónico, multimedia, servicios interactivos y computación móvil.*
  Aplicada directamente en la concepción de la arquitectura de servicios. Por un lado, se diseñó la aplicación multimedia inmersiva que opera de forma autónoma (computación móvil). Por otro, se construyó una API REST para la red hospitalaria, implementando protocolos de autenticación seguros y sin estado (*Stateless*) mediante JSON Web Tokens (JWT). Esta competencia culmina con la *Single Page Application* (SPA), capaz de gestionar peticiones asíncronas *multipart* para procesar miles de registros biométricos de manera concurrente y renderizar métricas interactivas en tiempo real.

6.3. TRABAJOS DERIVADOS Y FUTUROS

El desarrollo iterativo de este ecosistema ha consolidado una arquitectura escalable que abre la puerta a diversas expansiones tecnológicas y clínicas. Como líneas de trabajo futuro, se plantean las siguientes evoluciones del sistema:

- **Gestión y comunicación en tiempo real:** sujeto a la disponibilidad de una infraestructura de red estable en el entorno clínico, se propone evolucionar la arquitectura *Offline-First* hacia un sistema de telemetría constante. Esto permitiría a los enfermeros iniciar los juegos remotamente desde la aplicación web y monitorizar las constantes biomecánicas del paciente de forma instantánea.
- **Evolución del seguimiento (*Tracking*) de extremidades inferiores:** Se explorará la sustitución de los controladores físicos de las gafas por *hardware* más ergonómico, valorando la integración de tecnologías o dispositivos *wearables* especializados en el rastreo cinemático de piernas.
- **Expansión del contenido lúdico:** se ampliará la base de datos de niveles disponibles para ambas terapias. En el módulo de ritmo, esto implicará la incorporación de nuevas pistas musicales, mientras que en el módulo de tren superior se diseñarán distribuciones de bloques más complejas.
- **Nuevas mecánicas y mejoras de jugabilidad (*Gameplay*):** para el ecosistema de *Arkanoid*, se prevé la introducción de bloques con patrones de movimiento dinámico y la ampliación del catálogo de bonificaciones. En paralelo, en el juego de piernas se perfeccionará el *feedback* mediante nuevos efectos visuales (*VFX*) para los impactos y se implementará un selector paramétrico de dificultad rítmica.
- **Estabilización y aseguramiento de calidad:** ejecución de tareas de mantenimiento continuo, depuración de código y resolución de errores menores (*bugfixes*) que puedan aflorar durante ciclos de despliegue prolongados en el hospital.

6.4. VALORACIÓN PERSONAL

Abordar el diseño y la construcción de un ecosistema de *software* de esta envergadura ha supuesto un punto de inflexión decisivo en mi trayectoria académica y profesional. Desde siempre había sentido una profunda curiosidad por aprender a desarrollar videojuegos, y la Realidad Virtual era una tecnología que me llamaba poderosamente la atención. Por ello, en cuanto leí la propuesta de este Trabajo de Fin de Grado, supe de inmediato que era el proyecto ideal para mí, aun siendo plenamente consciente de que me enfrentaba a un

reto tecnológico que excedía con creces mi zona de confort. Sin experiencia previa en la programación con el motor gráfico Unity, y siendo este mi primer contacto absoluto con el paradigma de desarrollo inmersivo, la curva de aprendizaje durante las primeras semanas resultó ser extraordinariamente pronunciada.

Sin embargo, culminar este anhelo personal viendo materializados los primeros "videojuegos" desarrollados íntegramente por mí, y comprobar cómo estos operan en armonía con disciplinas tan dispares como el cálculo de vectores físicos, la cinesiterapia clínica y el diseño auditivo, ha sido una experiencia sumamente gratificante. Asumir en solitario la responsabilidad de un desarrollo *Full-Stack* tan profundo me ha dotado de una visión arquitectónica global que resulta invaluable para mi futuro como Ingeniero Informático.

Más allá del desafío del código y la infraestructura, el mayor orgullo que extraigo de este proyecto reside en su propósito humanitario. A lo largo de mis estudios, siempre tuve la ambición de aplicar mis conocimientos en herramientas que ayudasen a la gente de verdad; y qué mejor manera de materializar este deseo que dando respuesta a una necesidad real y directa solicitada por el Hospital General Universitario de Ciudad Real. La oportunidad de trasladar algoritmos abstractos y arquitecturas de red al entorno físico de este hospital, observando de primera mano cómo la tecnología que he diseñado tiene el potencial genuino de mitigar la dureza del tratamiento de hemodiálisis y mejorar la calidad de vida de los pacientes, dota a todo el esfuerzo invertido del mayor de los sentidos.

Bibliografía

[1] Henry Ashista, Alanis Santiago Comas, Taylor Selby, Mohammad Yasir Essar, Jude Alawa, Samar Al-Hajj, and Erica Nelson. An offline-first electronic health record for vulnerable populations: A mixed-methods feasibility study. *PLOS Digital Health*, 5(2):e0001204, 2026.

[2] Karl Becker. Unity statistics 2026. https://gitnux.org/unity-statistics/, Febrero 2026. Último acceso: 12 de julio de 2026.

[3] Simon Brown. The C4 model for visualising software architecture. https://c4model.com/, 2022. Último acceso: 12 de julio de 2026.

[4] Boletín Oficial del Estado. Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales (lopdgdd). BOE núm. 294 https://www.boe.es/eli/es/lo/2018/12/05/3/con, 2018. Último acceso: 12 de julio de 2026.

[5] Sebastian Deterding, Dan Dixon, Rilla Khaled, and Lennart Nacke. From game design elements to gamefulness: defining "gamification". In *Proceedings of the 15th international academic MindTrek conference: Envisioning future media environments*, pages 9–15, 2011.

[6] María José Fernández Lara, José Luis Ibarra Cornejo, Elena Viviana Aguas Alveal, Cesar Eduardo González Tapia, and Diego Galvarino Quidequeo Reffers. Beneficios del ejercicio físico en pacientes con enfermedad renal crónica en hemodiálisis. *Enfermería nefrológica*, 21(2):167–181, 2018.

[7] Yúbal Fernández. El primer simulador VR de la historia tenía forma de recreativa, y se inventó a finales de los 50. Artículo en línea publicado en Xataka https://www.xataka.com/historia-tecnologica/el-primer-simulador-vr-de-la-historia-tenia-forma-de-recreativa-y-se-invento-a-finales-de-los-50 abril 2018. Último acceso: 12 de julio de 2026.

[8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2014.

[9] Vanesa Herrera, David Vallejo, José J Castro-Schez, Dorothy N Monekosso, Ana de los Reyes, Carlos Glez-Morcillo, and Javier Albusac. Rehab-immersive: a framework to support the development of virtual reality applications in upper limb rehabilitation. *SoftwareX*, 23:101412, 2023.

[10] Hernandez AV Herrera-Añazco P, Palacios-Guillén M. Alta tasa de interrupción de hemodiálisis en pacientes del hospital nacional 2 de mayo del perú. https://www.revistarenal.org.ar/index.php/rndt/article/view/110/691, 2014. Último acceso 12 de julio de 2026.

[11] Danielle L Kirkman, Lee D Roberts, Malte Kelm, Jürgen Wagner, Mahdi M Jibani, and Jamie H Macdonald. Interaction between intradialytic exercise and hemodialysis adequacy. *American journal of nephrology*, 38(6):475–482, 2013.

[12] Daniel dos Santos Krug, Rafael Chanin, and Afonso Sales. Exploring the pros and cons of monolithic applications versus microservices. In *Proceedings of the 26th International Conference on Enterprise Information Systems (ICEIS), 2024-Volume 2*, 2024.

[13] Walead Latif. Enfermedad renal crónica. https://medlineplus.gov/spanish/ency/article/000471.htm, 2025. Último acceso 12 de julio de 2026.

[14] Joseph J LaViola Jr, Ernst Kruijff, Ryan P McMahan, Doug Bowman, and Ivan P Poupyrev. *3D user interfaces: theory and practice*. Addison-Wesley Professional, 2017.

[15] Danielle E Levac, M Elise Huber, and Dagmar Sternad. Motor learning in virtual reality environments. *Physical therapy*, 95(10):1415–1428, 2015.

[16] Simon McCallum. Gamification and serious games in healthcare. *Finnish Journal of eHealth and eWelfare*, 4(3):119–132, 2012.

[17] Paul Milgram and Fumio Kishino. A taxonomy of mixed reality visual displays. *IEICE TRANSACTIONS on Information and Systems*, 77(12):1321–1329, 1994.

[18] Centro Otoneurológico. Realidad virtual y vértigo o desequilibrio. https://otoneurologico.com/realidad-virtual-y-vertigo-o-desequilibrio/, 2025. Último acceso 12 de julio de 2026.

[19] Laura Panjwani. Virtual "snowworld" helps burn victims cope with extreme pain. https://www.rdworldonline.com/virtual-snowworld-helps-burn-victims-cope-with-extreme-pain/, 2017. Último acceso 12 de julio de 2026.

[20] Roger S Pressman. *Software engineering: a practitioner's approach*. Palgrave macmillan, 2005.

[21] HMM Rismy and MHM Rizan. Microservices architecture in healthcare system. *International Journal of Computer Applications*, 183(10):25–30, 2021.

[22] Claudio Servin. Espada de damocles. Artículo en línea publicado en la página del proyecto IDIS https://proyectoidis.org/espada-de-damocles/, agosto 1968. Último acceso 12 de julio de 2026.

[23] Mel Slater. Place illusion and plausibility can lead to realistic behaviour in immersive virtual environments. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 364(1535):3549–3557, 2009.

[24] Maximilian Speicher, Brian D Hall, and Michael Nebeling. What is mixed reality? In *Proceedings of the 2019 CHI conference on human factors in computing systems*, pages 1–15, 2019.

[25] Atina Technologies. Unity vs. unreal for vr: Which should you pick in 2026? https://www.atinatechnology.in/unity-vs-unreal-for-vr/, Abril 2026. Último acceso: 12 de julio de 2026.

[26] Michael H Thaut and Volker Hoemberg. *Rhythmic auditory stimulation in rehabilitation of movement disorders*. Taylor & Francis, 2014.

[27] Kinesix XR Therapeutics. Realidad virtual inmersiva en manejo de quemaduras: Tratamiento del dolor y rehabilitación. https://kinesixvr.com/es/blog/rv-quemaduras-tratamiento-dolor/, 2024. Último acceso 12 de julio de 2026.

[28] MindMaze Therapeutics. A treatment ecosystem that's redefining the future of brain health and recovery. Plataforma web oficial, 2026. Último acceso: 12 de julio de 2026.

[29] Unión Europea. Reglamento (ue) 2016/679 del parlamento europeo y del consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (rgpd). Diario Oficial de la Unión Europea, L 119/1 https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016. Último acceso: 12 de julio de 2026.

[30] V Wattanasoontorn, I Boada, R Garcia, and M Sbert. Serious games for health. *Entertainment Computing*, 4(4):231–247, 2013.

[31] Wikipedia. Vpl research. https://en.wikipedia.org/wiki/VPL_Research, 2026. Último acceso 12 de julio de 2026.

[32] Michael Zyda. From visual simulation to virtual reality to games. *Computer*, 38(9):25–32, 2005.

ANEXOS

ANEXO A

Informe de Colaboración y Validación del HGUCR

A continuación, se adjunta el documento oficial emitido por el Hospital General Universitario de Ciudad Real (HGUCR), que acredita la colaboración, experimentación y validación de la seguridad y usabilidad del sistema desarrollado en este Trabajo de Fin de Grado.

Ciudad Real, a 1 de julio de 2026

INFORME DE COLABORACIÓN Y VALIDACIÓN

A quien corresponda:

Por medio del presente escrito, en mi condición de Supervisor del Área de Investigación, Docencia y Formación del Hospital General Universitario de Ciudad Real (HGUCR), hago constar la participación y colaboración de este centro hospitalario en el Trabajo de Fin de Grado (TFG) desarrollado por el estudiante D. Pedro Antonio Trujillo Felipe y sus tutores Javier Alonso Albusac Jiménez y Vanesa Herrera Tirado.

En primer lugar, deseo dejar constancia de que el HGUCR ha participado desde el inicio del proyecto en la especificación de requisitos del trabajo, aportando el conocimiento clínico necesario para orientar la solución hacia las necesidades reales de los pacientes sometidos a tratamiento de hemodiálisis.

Al mismo tiempo, el hospital ha realizado un seguimiento continuado del diseño y desarrollo del software a lo largo de todo el proyecto, sugiriendo mejoras y ajustes orientados específicamente a las particularidades y necesidades de este perfil de pacientes, con el fin de garantizar que el resultado final resultara adecuado al entorno asistencial.

En el marco de esta colaboración, el miércoles 1 de julio de 2026 se celebró en las instalaciones del hospital una sesión de experimentación en la que participaron dos miembros del personal clínico del centro y un paciente. El objetivo de dicha sesión fue validar la usabilidad, la funcionalidad y la seguridad del sistema para los pacientes.

Tras la realización de las pruebas, podemos confirmar que el sistema resulta usable por parte de los pacientes y seguro para su uso en el contexto de la hemodiálisis. Conviene destacar las particularidades de estos pacientes, que presentan una movilidad reducida derivada de la necesidad de mantener el catéter (acceso vascular) empleado para el filtrado de la sangre, lo que impide la realización de movimientos bruscos o exagerados durante la sesión. En este sentido, los juegos serios basados en realidad virtual permiten llevar a cabo un ejercicio controlado y moderado, plenamente compatible con estas limitaciones.

Consideramos, además, que la propuesta contribuirá a mejorar la motivación del paciente, incidiendo positivamente en su estado emocional. La posibilidad de realizar ejercicio moderado durante las largas horas que dura la sesión de hemodiálisis podría repercutir favorablemente en su calidad de vida.

Una vez asegurada la funcionalidad, la usabilidad y la seguridad del sistema, el Hospital General Universitario de Ciudad Real manifiesta su disposición a continuar colaborando en esta línea de trabajo, a participar en el análisis de los datos recogidos y a celebrar nuevas sesiones con un grupo más amplio de pacientes en próximas fechas.

Y para que así conste y surta los efectos oportunos, firmo el presente escrito.

Atentamente,

BERNAL CELESTINO RUBEN JOSE - 05679004M

Firmado digitalmente por BERNAL CELESTINO RUBEN JOSE - 05679004M
Fecha: 2026.07.02 12:20:30 +02'00'

Fdo.: Rubén José Bernal Celestino
Supervisor del Área de Investigación, Docencia y Formación
Hospital General Universitario de Ciudad Real (HGUCR)